

QKD Oracles for Authenticated Key Exchange

Kathrin Hövelmanns¹, Daan Planken^{2,3}, Christian Schaffner^{2,3}, and Sebastian R. Verschoor^{2,3}

¹ Eindhoven University of Technology, The Netherlands

² University of Amsterdam, The Netherlands

³ QuSoft, Amsterdam, The Netherlands
`qkdOraclesForAKE@hoevelmanns.net`

Abstract. Authenticated Key Exchange (AKE) establishes shared (‘symmetric’) cryptographic keys which are essential for secure online communication. AKE protocols can be constructed from public-key cryptography like Key Encapsulation Mechanisms (KEMs). Another approach is to use Quantum Key Distribution (QKD) to establish a symmetric key, which uses quantum communication. *Combining* post-quantum AKE and QKD *appropriately* may provide security against quantum attacks even if only one of the two approaches turns out to be secure.

We provide an extensive review of existing security analyses for combined AKE and their formal security models, and identify some gaps in their treatment of QKD key IDs. In particular, improper handling of QKD key IDs leads to Dependent-Key attacks on AKE.

As our main conceptual contribution, we model QKD as an oracle that closely resembles the standard ETSI 014 QKD interface. We demonstrate the usability of our QKD oracle for cryptographic security analyses by integrating it into a prominent security model for AKE, called CK⁺ model, thereby obtaining a security model for combined AKE that catches Dependent-Key attacks. In this model, we formally prove security of a new protocol that combines QKD with a triple-KEM handshake. This is the first provably secure hybrid protocol that maintains information-theoretic security of QKD.

Keywords: Post-Quantum Cryptography · PQC · Quantum Key Distribution · QKD · Authenticated Key Exchange · AKE · Provable Security · Hybrid Protocols · Combined Security · Dependent-Key attacks

1 Introduction

Today’s secure online communication heavily relies on symmetric cryptography. One way to establish shared symmetric *session keys* is to use Authenticated Key Exchange (AKE) protocols, which are often based on public-key cryptography. AKE guarantees that session keys remain unknown to anyone but the two intended parties. By this guarantee, unauthorized parties cannot obtain these session keys, and thus the honest parties’ communication is protected by symmetric cryptography. It is, however, not completely straightforward how these AKE security guarantees should be formalized – over the past three decades, different security definitions [BR94, CK01, Kra05, LLM07] have been given, all coming with subtle differences and advantages/disadvantages.

Post-quantum AKE. Quantum computers threaten the security of many widely deployed AKE protocols, as Shor’s quantum algorithm [Sho94] breaks the assumptions underlying the current

KH was supported by an NWO VENI grant (Project No. VI.Veni.222.397). DP and SV were supported by the Dutch National Growth Fund (NGF), as part of the Quantum Delta NL programme. CS is (partially) supported by gravitation project Challenges in Cyber Security (CiCS) with file number 024.006.037 which is financed by the Dutch Research Council (NWO). Date: September 15, 2025

public-key schemes. One countermeasure is to replace the current cryptographic algorithms with ones that can be executed on *classical* (non-quantum) hardware, but that resist attacks by an adversary equipped with a quantum computer. This type of cryptography is called Post-Quantum Cryptography (PQC) [BL17]. Recent efforts have led to the development of new digital signatures and Key Encapsulation Mechanisms (KEMs), cryptographic building blocks that can be used as replacements in existing cryptographic protocols like AKE. There has been much scientific debate on which concrete signatures and KEMs should be chosen for deployment, with the ongoing effort accumulating in several (forthcoming) standards [AAC+22, ABC+25].

One concern with novel post-quantum cryptography (and public-key algorithms in general) is that their security relies on the conjectured hardness of certain computational problems. Accordingly, future technological or cryptanalytic advances may turn out to invalidate their security. This is unavoidable because it is impossible to have information-theoretic secure public-key cryptography – an unbounded attacker can always look up the public key and then run a brute-force search for the private key. To hedge against potential future breaks, it is thus widely recommended [Fed21, ANS22, SCN+24] to deploy *hybrid* solutions: hybrid solutions combine multiple algorithms such that the combination remains secure even if all but one of its components are broken. For KEMs, recent research [GHP18, BBF+19] proved that a natural KEM combiner approach – adding several KEM keys together via an exclusive-or (XOR) – is not sufficient to achieve practical (*chosen-ciphertext*) security, and provided non-trivial alternatives that actually meet this security goal.

QKD. Another approach to preventing quantum attacks is to use Quantum Key Distribution (QKD) [BB84, Eke91]. QKD is an interactive protocol which allows Alice to establish a cryptographic key with Bob by encoding information into quantum states that are sent over a quantum channel. QKD provides security guarantees that do not rely on computational hardness assumptions. This is also called information-theoretic security (ITS). However, for QKD to have ITS, the classical communication in a QKD protocol must be authenticated with ITS one-time Message Authentication Codes (MACs), which is commonly achieved by using pre-shared symmetric keys. If these pre-shared keys were directly used for symmetric cryptography, they would be used up quickly, while QKD allows to extend these pre-shared keys – something that is impossible with classical communication alone. In other words, the ITS property of QKD eliminates the threat of future cryptanalytic advances.

On the other hand, QKD requires careful handling of quantum information which makes implementations technologically challenging. As a result, QKD is currently limited in range (to a few hundred kilometers) and in key rates by the quantum communication hardware, and the service might not always be available. Some of these limitations are mitigated in practice by having QKD running continuously in the background, producing keys that are stored in a buffer for later use. Users can request keys via ETSI 014 [ETS19] or a similar interface. The keys are then provided to the user together with an identifier. To ensure that both users retrieve the same key from their devices, the users need to communicate that identifier as part of their protocol. Notably, the key identifier is the only value that binds the QKD key to the session. As we will demonstrate, it is essential to include these identifiers in the higher-level protocol – we describe (in Section 3.3) an attack on protocols that exchange the identifiers out-of-bound. Crucially, that inclusion should not make use of computationally secure primitives (such as cryptographic hash functions), as application of computational primitives to a QKD key would void information-theoretic security, the major selling point of QKD.

While several QKD standardization processes are ongoing [DSL+22, SPP+24], a widely accepted standard for QKD protocols is still lacking. In order to certify QKD deployment in practice, we expect it to be mandatory to combine their key material with post-quantum or current public-key cryptography in a hybrid AKE protocol.

Given the importance of AKE, and the uncertainties surrounding post-quantum approaches and QKD, is it natural to ask:

What is the appropriate way of combining post-quantum cryptography and QKD to achieve a hybrid protocol that remains secure if either the post-quantum cryptography or QKD is secure?

1.1 Our contributions

- Our main conceptual contribution is a tool that allows to formally analyze the security provided by QKD in cryptographic contexts. Concretely, we propose **QKD oracles** whose functionality closely resemble the standard ETSI 014 QKD interface [ETS19], but they can easily be adapted to other QKD device interfaces as well, see Fig. 6 in Section 4. The oracles allow an honest party to retrieve a QKD key (implemented as a uniformly random bitstring) accompanied by a unique identifier. By querying the oracle with that same identifier, the honest peer can retrieve the same key. This interface forces protocols to explicitly communicate the key identifier. We model compromised parties or other attacks on QKD by allowing the adversary to query the oracle, to learn or even change the key.
- These oracles can be integrated into AKE models, as we demonstrate by incorporating them into the CK⁺ security model for AKE [Kra05], resulting in a **novel AKE security model** that captures Dependent-Key attacks, see Section 4. We build upon the work of Hövelmanns, Kiltz, Schäge, and Unruh [HKSU20], who specify the CK⁺ security model in pseudo-code to ensure there is as little ambiguity as possible.
- We propose a **new hybrid AKE protocol** that combines QKD with a Triple-KEM PQC protocol, see Fig. 4 in Section 3. To cryptographically bind the sessions together, we compute two one-time MACs: one with (part of) the QKD key and one with (part of) the PQC key. The session key is given by the XOR of the remainder of both keys. In Section 5, we prove that this protocol is secure in our CK⁺-like model, even if one of its components fails. If QKD is secure, we obtain information-theoretic security, while the security of the PQC protocol is relative to the security of the KEMs and is given in the Quantum Random Oracle Model (QROM).
- As a minor contribution, we demonstrate the importance of explicitly communicating QKD key identifiers by giving an **explicit Dependent-Key attack** against protocols that consider this out-of-bounds, see Section 3.3. We also provide an extensive review of existing literature in Section 1.2, and we highlight an issue in the security proof of ITS KEM combiners.

Outline Our article is structured as follows. In Section 1.2, we provide an extensive review of related work on KEM combiners and QKD integration. Section 2 provides the necessary background on AKE, PQC and QKD. Section 3 presents our hybrid AKE protocol combining QKD with Triple-KEM PQC (see Fig. 4), and Section 3.3 explains potential Dependent-Key attacks when QKD key IDs are not handled properly. Section 4 defines our AKE security model including the formalization of our new QKD oracles in Fig. 6. Section 5 provides the security proof for our protocol. Section 6 concludes the paper and discusses future work.

1.2 Related work

Original work on QKD in an Authenticated Key Exchange (AKE) framework. To model QKD security in an AKE-like fashion, Mosca, Stebila, and Ustaoglu [MSU13] enhanced an AKE security model, called eCK model, with an oracle (called SendQ oracle) that models quantum communication. The authors concluded the work with the question how end-to-end protocols could best combine QKD and classical (non-quantum) AKE keys.

KEM Combiners as ‘hybrid AKE’ (QKD-AKE) combiners? Our end-to-end protocol was designed to combine a QKD exchange with an AKE (a KEM handshake) in a way that preserves security even if one of the components fails. This goal strongly resembles the goal underlying recent KEM combiners [GHP18, BBF⁺19]. There already exist various ‘hybrid AKE’ constructions that combine QKD with classical cryptography, which we will discuss and compare below. What many approaches have in common is that they simply use QKD as input to existing KEM combiners. We want to immediately stress, however, that this raises two independent problems:

1. Firstly, *QKD is not a KEM* — KEMs *and their security* are inherently based on the fact that they use an asymmetric key pair. QKD inherently lacks the input requirements of a KEM combiner. As we will detail in Section 3.1, recent KEM combiners do not mesh well with QKD and finding a suitable QKD-AKE combiner is less straightforward than one might initially think.
2. Secondly, the output of a KEM combiner is *still a KEM* – even if it was ‘proven secure’ (as a KEM), it doesn’t automatically yield a secure *protocol* for *Authenticated Key Exchange*. The resulting (combined, *authenticated*) protocol still needs to be analyzed in a proper AKE-like security model. To that end, QKD needs to be modeled properly via some abstraction such as, e.g., our oracle (see Section 4), or a shared array (as done in HAKE [DHP20a], see below).

We summarize the reviewed literature in Table 1. The given constructions vary widely concerning the availability of a technical security analysis, with the level ranging from non-existent to full formal proofs in a suitable security model. We list the type of cryptographic construction that is being given (‘primitive’), the security definition against which it is analyzed (if any), whether the analysis is given as a formal proof, whether or not the combiner includes the QKD key ID somewhere (which proves important due to the attack described in Section 3.3), and whether or not the combiner retains information-theoretic security (ITS). We substantiate the table by detailing our findings below.

Table 1: Comparison of hybrid constructions that integrate QKD.

	Primitive	Security Notion	Proof	Key ID	ITS
OpenQKD [TLLM18]	PSK/XOR	None	No	No	Maybe
X.1714 [ITU20]	Key combiner	None	No	No	Maybe
3-key combiner [RDM ⁺ 24]	dual PRF	dual PRF ^a	? ^b	Yes	No
QR TLS 1.3 [RCV ⁺ 23]	AKE	None	No	Yes	No
ART [ART24]	AKE	chosen-ciphertext	No	Yes	No
Muckle [DHP20a]	AKE	HAKE	Yes	No	No
Muckle+ [BRS23]	AKE	HAKE	Yes	No	No
Muckle++ [GPH ⁺ 24]	AKE	HAKE	? ^b	Yes	No ^c
Muckle# [BSP ⁺ 24]	AKE	HAKE	? ^d	No	No
VMuckle [BBB ⁺ 25]	AKE	HAKE	Yes	No	No
This work	AKE	CK ⁺ (with QKD oracle)	Yes	Yes	Yes

^a Despite the work suggesting that the construction is chosen-ciphertext secure (see below).

^b Status unclear: proof contains inconsistencies (see below).

^c Despite what is stated in the work (see below).

^d Status unclear: incomplete security definition (see below).

OpenQKD. Tysowski, Ling, Lütkenhaus, and Mosca [TLLM18] suggest to use the QKD key directly as a pre-shared key (PSK) in existing protocols when ITS is not required. Alternatively, they give

an example for a XOR key combination, and substantiate compliance of the XOR combiner with the NIST recommendations by referring to the NIST PQC project FAQ. By now, however, XORing keys is no longer consistent with NIST recommendations – the respective FAQ answer has since been clarified. It now refers to SP 800-56C [BCD20] for a generic composite key establishment scheme that concatenates the keys and then hashes them with a cryptographic hash function. More importantly, however, we would advise against simply XORing the keys since this is vulnerable to Dependent-Key attacks (see Section 3.3).

ITU-T X.1714. recommends [ITU20] to combine QKD with other methods using “key combiners”. By this they mean a function that takes multiple keys as input and outputs another key. We note that their function does not take KEM ciphertexts (or other context) as input. The document provides no formal security definition.

PRF-based combiners. A dual Pseudo-Random Function (PRF) takes two inputs and provides a pseudorandom output if either input is pseudorandom. Aviram, Dowling, Komargodski, Paterson, Ronen, and Yagev [ADK⁺22] give a (proven secure) dual PRF construction (called KeyCombine). Bindel, Brendel, Fischlin, Goncalves and Stebila [BBF⁺19] show that such dual PRFs can be used for KEM combiners, by providing a (proven secure) KEM combiner (called Dual-PRF Combiner) that first applies a dual PRF to the KEM keys, and then uses the result as the key for a PRF computed over the ciphertexts. The authors warn that “the naive approach of directly using a dual PRF” does not preserve practical (chosen-ciphertext) security.

In [ADK⁺22], the authors also gave a generalized construction (called multi-key KeyCombine) with more than two keys as input, and accordingly adapted the security definition. A variant of the multi-key KeyCombine construction, called 3-key combiner, was given by Ricci, Dobias, Malina, Hajny and Jedlicka in [RDM⁺24]. The authors argue chosen-ciphertext security of the 3-key combiner by stating that it achieves the security notion for multi-input PRFs and then referring to the proof of the dual PRF *KEM combiner*. We now argue, however, that there are several independent issues with this. First, in this context, the combiner yields an interactive protocol, not a KEM, and chosen-ciphertext security will thus not be the relevant security property in this context. Assuming it were, we secondly note that the work uses the multi-input PRF ‘naively’ in the sense described by [BBF⁺19], and as [BBF⁺19] warned, this does not achieve chosen-ciphertext security. Thirdly, the combiner expects KEM outputs as its input, whereas the construction in [RDM⁺24] uses QKD and Kyber.AKE [BDK⁺18] as inputs, none of which are KEMs. In conclusion, we believe that chosen-ciphertext security neither holds nor is it the relevant security property.

QR TLS 1.3. In the quantum-resistant TLS 1.3 protocol by Rubio García, Cano Aguilera, Vegas Olmos, Tafur Monroy, and Rommel [RCV⁺23], the authors take the QKD key and concatenate it with the existing Input Keying Material of the TLS 1.3 key schedule. (The QKD key ID is included in the ServerHello message.) The protocol is described as “achieving enhanced security against quantum adversaries”, but the work does not provide a corresponding security definition or (informal) security analysis to clarify this description.

ART. Acquina, Rommel and Tafur Monroy [ART24] construct a protocol that runs two ephemeral KEMs in parallel with QKD and then apply a KEM combiner resembling one from [GHP18]. All protocol messages are both signed (with EdDSA and with Dilithium) and tagged (with Poly1305, using part of the QKD key). Security of the protocol is not treated explicitly – no formal security definition or proof are given. We see two issues with the construction, resembling those of the 3-key combiner: Firstly, the resulting protocol is not a KEM, chosen-ciphertext security of the combiner thus might not be enough. Secondly, QKD is not a KEM, which the combiner would expect as input. The authors argue that QKD could be viewed as a KEM where “public and secret keys play no role” – the QKD step that produces the key and its ID could essentially be viewed as the KEM algorithm

that produces a key and a “ciphertext”, and the QKD step that takes the key ID and looks up the corresponding QKD key could essentially be viewed as the KEM algorithm that recovers shared keys from ciphertexts. We would argue, however, that this would not hold up in terms of security – QKD does not use a public-key pair like a KEM does and is therefore incompatible with the usual security notions for KEMs, including chosen-ciphertext security (as required by the combiner).

HAKE and Muckle. Dowling, Hansen, and Paterson [DHP20a] also combined classical (non-PQC) cryptography, PQC and QKD, accordingly calling the result Hybrid AKE (HAKE). They give a formal security definition, by adapting an AKE security model to combined protocols, reflecting that each of the three components by itself might be vulnerable. QKD is described as being modeled as a shared array of secret bits which honest parties can access to obtain their QKD keys, using an index that would be functionally equivalent to the QKD key ID (also see Section 2.5). We noticed, however, that parties will always get the correct QKD key without having to specify or communicate any index at all, since the model [DHP20b, Appendix C, Fig. 5] fixes a shared key per protocol stage. It thus does not capture attacks where the adversary tricks an honest party into using the wrong key, such as the Dependent-Key attack described in Fig. 5.

We also noticed that the security definition exhibits some ambiguity due to ambiguity of the cleanliness predicate, which in turn stems from a technical issue regarding how QKD keys are shared between sessions. Concretely, the HAKE Create query initializes a fixed key (per session, per stage) in a variable called **esk**. When the adversary triggers subsequent sessions, they can specify if **esk** should be copied from an already initialized session at the peer. Specifically for protocols like Muckle (see below), this power results in ambiguity: the cleanliness predicate (Definition 3: $\text{clean}_{\text{HAKE}}$) only forbids “trivial” leakage of the test session’s QKD key, i.e., leakage either through the test session itself or its matching session. A security proof in this model does not capture that two parties might run a shared session, using different QKD keys (leading to desynchronization that might only be detected at a later stage) and it could happen that a QKD key might be copied over to another session that should in principle be independent. This issue can be resolved by accordingly adjusting the Create query or by modeling QKD through an oracle instead, as we will describe in Fig. 6.

In the same work, the authors present Muckle, a multi-stage protocol where parties share a pre-shared key (PSK), a secret state, and have access to QKD. At each stage, the session generates three ephemeral keys (with a classical KEM, a PQC KEM, and QKD), and authenticates the sent messages with a MAC. The ephemeral keys, secret state, protocol transcript and a counter are processed in a key schedule (a chain of PRFs), which outputs an updated secret state and the session keys. Since Muckle derives the keys with a PRF, it can be computationally secure at best. Furthermore, it is known [CCG16, Theorem 19] that correct one-round protocols cannot achieve full PCS without being vulnerable to denial-of-service attacks, and indeed Muckle can be desynchronized by dropping a single response message. Security is proven in the HAKE model. We note a very minor issue with the main security theorem [DHP20a, Theorem 1] that analyses attacks when the adversary compromised the PQC KEM or QKD (but not both). The proof analyses the two cases separately and then adds the resulting terms, whereas it could (and should) take their minimum.

Muckle+. Bruckner, Ramacher, and Striecks [BRS23] later presented Muckle+, a Sign-and-MAC style protocol which replaces Muckle’s authentication method with a new approach: whereas Muckle MAC’ed the sent messages, using a PSK-based key, Muckle+ takes (the hash of) a partial transcript of the protocol run, signs it, and then MACs it with a key that is derived from the established ephemeral keys and the partial transcript. This makes Muckle+ significantly distinct from Muckle and Krawczyk’s SIGMA protocol [Kra03]. The security analysis is in the HAKE model. We remark that the authors state that they need the QKD keys to be distributed via multi-path techniques, which is not captured by HAKE (and thus not by the proof).

Technically, Muckle+ contains an (easily fixable) bug, stemming from the treatment of the session key. The authors determine [BRS23, Section 3.3] the to-be-attacked session key as a master secret MS . But this creates a vulnerable dependency – in the next stage of the protocol, the session’s internal state is updated to $SecState \leftarrow \text{PRF}(MS, pub)$, where pub is some publicly available information. When attacking the key MS , the adversary gets a value MS' that is either random or equals MS and is tasked with determining which it is. To do so, the adversary can now exploit that the HAKE model allows to reveal $SecState$, the next stage’s state. By checking whether the state fits the rule $SecState = \text{PRF}(MS', pub)$, the adversary immediately knows if MS' equals MS . This can easily be fixed by setting the session key to $(CATS, SATS)$, which are application traffic secrets from which no following values are derived. We reported this issue to the authors and it was resolved in follow-up work (VMuckle, see below).

Muckle++. Muckle++ is a protocol by Garms, Paraïso, Hanley, Khalid, Rafferty, Grant, Newman, Shields, Cid, and O’Neill that modifies Muckle by (optionally) replacing the MACs with signatures, and by replacing the KDF with an ITS primitive [GPH+24]. Specifically, the key schedule replaces the PRF in the key schedule and derivation of the MAC keys with something called a QPRF. The authors write: “To instantiate the QPRF, a keyed-hash MAC (HMAC)-based KDF (HKDF) was modified by xor-ing quantum key material to the PRF output.” It is never specified where this “quantum key material” comes from, nor is it ever provided as input to QPRF in the protocol description. More important, however, is that in the security proof [Theorem 3, Supporting Information], the authors model the QPRF as a dual-PRF, which can never achieve ITS.

Muckle#. Muckle# is a protocol by Battarbee, Striecks, Perret, Ramacher, and Verhaeghe that modifies Muckle+ by replacing the signatures used for authentication with static KEMs [BSP+24]. The security analysis appears to be incomplete – we were unable to verify the proof due to two issues. Firstly, Definition 8 (malicious acceptance) appears to be incomplete, as the sentence contains the phrase “has not been issued”, but it is unclear what this is referring to. It is not stated how this definition is integrated into the HAKE security experiment. Secondly, we note that the cleanliness definition is incomplete: the passive case lists that no **CompromiseQK** query may have been issued for the test session/stage, but the proof of that case considers two identical sub-cases that assume no **CompromiseSK** query has been made. The wording for the cleanliness definition for the active case suggests there should be a list of queries that are disallowed, but no queries are listed.

VMuckle. VMuckle is a protocol by Buruaga, Bugler, Brito, Martin, and Striecks [BBB+25], that adds cryptographic agility to Muckle+ for the purpose of authentication in the MACsec protocol: parties authenticate either using a pre-shared key or with digital signatures. The session key $(CATS, SATS)$ is then used as the Master Session Key input for the MACsec Key Agreement (MKA) protocol.

2 Preliminaries

In this section, we recollect standard notation and background for (post-quantum) cryptography like cryptographic primitives, Authenticated Key Exchange (AKE), and the Quantum Random Oracle Model (QROM), as well as Quantum Key Distribution (QKD).

By $\llbracket b \rrbracket$ we denote the indicator variable of b , which takes value 1 if the boolean statement b is true and 0 if b is false. A list $[x, y]$ contains two elements, $[]$ is an empty list, and to append x to list ℓ , we write $\ell += [x]$. We write $x \leftarrow_{\$} S$ to denote that x is sampled uniformly random from S , we write $x \leftarrow A()$ if x is the output of a probabilistic algorithm A , and we write $x := A()$ if x is the output of a deterministic algorithm A . We write A^H to indicate that an algorithm A has access to an oracle H . Algorithms are efficient with respect to the security parameter, with the exception of

computationally unbounded adversaries. While formally this requires each algorithm to receive the security parameter as (unary) input, we omit this to avoid notational clutter. SECURITY-NOTIONS are written in capitalized sans-serif font. Bookkeeping is denoted in green, using different fonts for **associative arrays**, **CONSTANTS**, and *variables*. Subroutines are written in sans-serif font, and ORACLES in capitals, with red for **ADVERSARY-ORACLES** and purple for **QKD-ORACLES**. The symbol $\perp$ represents failure, for example when indexing an associative array on a key that has not been set yet. For clarity, we explicitly initialize some variables, others are implicitly initialized at a zero or empty value.

Computational and information-theoretic security. Some cryptographic primitives require *computational* security, meaning their security definition should hold against quantum-polynomial time (QPT) adversaries. Other primitives should provide security even against computationally unbounded adversaries, then we say that the primitive provides *information-theoretic* security (ITS).

2.1 Cryptographic Primitives - Key Encapsulation Mechanisms

For a fixed security parameter, a key encapsulation mechanism $\text{KEM} = (\text{KeyGen}, \text{Encaps}, \text{Decaps})$ consists of three algorithms. Key generation $\text{KeyGen}()$ outputs a keypair (pk, sk) . Encapsulation computes $(c, k) \leftarrow \text{Encaps}(pk)$, where $k \in \{0, 1\}^\ell$ is called the key, ℓ is called the key length, and c is called the encapsulation (or sometimes the ciphertext). Decapsulation computes $k' := \text{Decaps}(sk, c)$, where $k' \in \{0, 1\}^\ell \cup \{\perp\}$: the symbol $\perp \notin \{0, 1\}^\ell$ indicates a decapsulation failure.

Definition 2.1 (correctness). A KEM is δ -correct if

$$\Pr[\text{Decaps}(sk, c) \neq k \mid (pk, sk) \leftarrow \text{KeyGen}(); (c, k) \leftarrow \text{Encaps}(pk)] \leq \delta$$

where the probability is taken over the random coins of KeyGen and Encaps .

In our proof we will require a bound on the collision probability of honestly generated KEM values.

Definition 2.2 (KEM collision probabilities). We define the public-key collision probability of a KEM as

$$\mu(\text{KeyGen}) := \Pr[pk = pk' \mid (pk, sk) \leftarrow \text{KeyGen}(), (pk', sk') \leftarrow \text{KeyGen}()] .$$

Additionally, we define the collision probabilities for its ciphertexts and for its encapsulated secrets as

$$\begin{aligned} \mu(\text{Encaps}) &:= \Pr[c = c' \mid (pk, sk) \leftarrow \text{KeyGen}(), (c, k) \leftarrow \text{Encaps}(pk), (c', k') \leftarrow \text{Encaps}(pk)] , \\ \mu(\text{Secret}) &:= \Pr[k = k' \mid (pk, sk) \leftarrow \text{KeyGen}(), (c, k) \leftarrow \text{Encaps}(pk), (c', k') \leftarrow \text{Encaps}(pk)] . \end{aligned}$$

Standard security notions for KEMs. Next, we recall ‘passive’ and ‘active’ security notions for KEMs, **INDistinguishability** under **Chosen-Plaintext Attacks** (IND-CPA) and under **Chosen-Ciphertext Attacks** (IND-CCA). In the IND-CPA game, the game prepares a key pair and a ‘challenge’ encapsulation together with the corresponding secret. It provides the adversary A with the public key, the encapsulation, and either the corresponding secret or a random one. This choice is formalized via a ‘challenge bit’ b . A is tasked with guessing which kind of secret it received, by outputting a guessing bit b' . The advantage of A measures how well A does at distinguishing between the two cases. The IND-CCA game is exactly the same, except that A can additionally request decapsulations of encapsulations of its own choosing (except for the challenge), by interacting with a decapsulation oracle.

Definition 2.3 (IND-CPA, IND-CCA). *Let KEM be a key encapsulation mechanism. For each bit $b \in \{0, 1\}$, we define an IND-CPA game $IND-CPA_b^{KEM,A}$ in Fig. 1, as well as a game $IND-CCA_b^{KEM,A}$. We define the IND-CPA and the IND-CCA advantage of an adversary A against KEM as*

$$\begin{aligned} \text{Adv}_{KEM}^{\text{IND-CPA}}(A) &:= |\Pr[\text{IND-CPA}_0^{KEM,A} \Rightarrow 1] - \Pr[\text{IND-CPA}_1^{KEM,A} \Rightarrow 1]|, \\ \text{Adv}_{KEM}^{\text{IND-CCA}}(A) &:= |\Pr[\text{IND-CCA}_0^{KEM,A} \Rightarrow 1] - \Pr[\text{IND-CCA}_1^{KEM,A} \Rightarrow 1]|. \end{aligned}$$

Game $IND-CPA_b^{KEM,A}$	GAME $IND-CCA_b^{KEM,A}$
01 $(pk, sk) \leftarrow \text{KeyGen}()$	06 $(pk, sk) \leftarrow \text{KeyGen}()$
02 $(c, k_0) \leftarrow \text{Encaps}(pk)$	07 $(c, k_0) \leftarrow \text{Encaps}(pk)$
03 $k_1 \leftarrow_{\$} \{0, 1\}^\ell$	08 $k_1 \leftarrow_{\$} \{0, 1\}^\ell$
04 $b' \leftarrow A(pk, c, k_b)$	09 $b' \leftarrow A^{\text{DEC}(\cdot)}(pk, c, k_b)$
05 return b'	10 return b'
	$\text{DEC}(c')$
	11 if $c' = c$: return $\perp$
	12 return $\text{Decaps}(sk, c')$

Fig. 1: IND-CPA and IND-CCA security games for $KEM = (\text{KeyGen}, \text{Encaps}, \text{Decaps})$.

2.2 Cryptographic Primitives - Message Authentication Codes

A message authentication code (MAC) is a triple $\mathcal{M} = (\text{MKG}, \text{MAC}, \text{MVerify})$, where MKG generates a key k , the tag generation algorithm MAC takes a key k and message m and computes a tag $\tau \leftarrow \text{MAC}_k(m)$, and the (deterministic) verification algorithm MVerify takes a key k , message m and tag τ and outputs a boolean value. We require *correctness*: for each $k \leftarrow \text{MKG}()$ and for each m , it must hold that $\text{MVerify}_k(m, \text{MAC}_k(m)) = \text{true}$.

Canonical MAC. A MAC is called *canonical* if MKG generates a uniformly random bitstring, tag generation (algorithm MAC) is deterministic, and MVerify recomputes the tag and compares it against its input. In this case, we use MAC interchangeably to refer to both the triple $\mathcal{M}$ and the tag generation algorithm. We assume MACs are canonical, because in practice most MACs are canonical, and canonical MAC give slightly tighter bounds in our security proof.

Message integrity. We define message integrity via **One-Time strong Existential UnForgeability** under **Chosen Message Attacks** (OT-sEUF-CMA). In the OT-sEUF-CMA game, the attacker A is granted to request a tag for a message of their own choosing, and subsequently tasked with creating a valid message-tag pair that is different from the obtained one.

Definition 2.4 (OT-sEUF-CMA). *Let $\mathcal{M}$ be a MAC. We define the OT-sEUF-CMA game against $\mathcal{M}$ as in Fig. 2, and the OT-sEUF-CMA advantage of an adversary A against $\mathcal{M}$ as*

$$\text{Adv}_{\mathcal{M}}^{\text{OT-sEUF-CMA}}(A) = \Pr[\text{OT-sEUF-CMA}^{\mathcal{M},A} \Rightarrow 1].$$

GAME OT-sEUF-CMA ^{$\mathcal{M}, A$} 01 $k \leftarrow \text{MKG}()$ 02 $m \leftarrow A_0()$ 03 $\tau \leftarrow \text{MAC}_k(m)$ 04 $(m', \tau') \leftarrow A_1(m, \tau)$ 05 return $\llbracket (m', \tau') \neq (m, \tau) \text{ and } \text{MVerify}_k(m', \tau') \rrbracket$

Fig. 2: OT-sEUF-CMA game for $\mathcal{M} = (\text{MKG}, \text{MAC}, \text{MVerify})$.

2.3 Authenticated Key Exchange

Authenticated Key Exchange (AKE) is arguably one of the most important cryptographic building blocks in modern security systems. Most AKE protocols are based on Public-Key Cryptography (PKC), such as an ad hoc Diffie-Hellman key exchange. That exchange is then authenticated, usually either via digital signatures, non-interactive key exchange, or (sometimes) public-key encryption or key encapsulation mechanisms [BCK98, BCNP08, FSXY12, FSXY13, ABS14, HKSU20].

AKE security models. Research on AKE has made tremendous progress concerning the development of more solid theoretical foundations, by specifying the targeted security guarantees via a formal security model. Notable models are, amongst others, the Bellare-Rogaway (BR) model [BR94], the Canetti-Krawczyk (CK) model [CK01], the CK⁺ model [Kra05], and the extended Canetti-Krawczyk (eCK) model [LLM07].

What they have in common is that they formalize the idea that a key computed between two parties should be suitable for the application of symmetric primitives, by formalizing that the key should be indistinguishable from random to any attacker. Where they differ is how they actually model the threat, in the sense that different models make different assumptions about the attacker's capabilities and about what could go wrong during real-world protocol runs. This is captured by a tailored set of capabilities that is assumed to be held by the attacker. In more detail, the model formalizes these capabilities by granting the attacker with a set of 'query' oracles. Each such query oracle allows to reveal a certain type of secret values. These assumptions have been extended considerably by more recent models, with the differences in definition leading to differences that are cryptographically meaningful.

Another way in which the models differ is the concrete formalization. At times, these can be rather informal descriptions of how attacker queries are handled, which can allow for misinterpretations. There do, however, also exist works like [HKSU20] that formalize these descriptions via pseudo-code, an approach we also follow in this work.

To handle the complexity of these models, proofs (such as ours) typically use game-hopping techniques. A simple but useful observation is called the difference lemma [Sho04]: given events A, B, F it holds that if $A \wedge \neg F \Leftrightarrow B \wedge \neg F$ (i.e. the two events $A \wedge \neg F$ and $B \wedge \neg F$ are identical), then $|\Pr[A] - \Pr[B]| \leq \Pr[F]$.

2.4 (Quantum) Random Oracle Model and One-way to Hiding

The standard way of modeling hash functions in modern cryptography is the Random Oracle Model (ROM). Random oracles are functions to which all algorithms have oracle access, with the output for every input being chosen uniformly at random from the codomain of the function.

Formally, the random oracle is defined as follows: a random oracle $H : \mathcal{X} \rightarrow \mathcal{Y}$ is an empty table of input and output values. As soon as a value $x \in \mathcal{X}$ that is not in the table gets queried, a uniformly random value $y \in \mathcal{Y}$ gets sampled, $H(x)$ is set to y and the pair $(x, H(x))$ is added to

the table. When a value x that is already in the table gets queried, the value $H(x)$ from the table is returned. Using this lazy-sampling approach, any newly queried value is uniformly random, but if a value is queried again, the output is consistent with earlier queries.

In this paper, we will be working in the quantum-accessible version of the ROM, which was introduced as the quantum random oracle model (QROM) in [BDF⁺11]. Following [BDF⁺11], we model the quantum random oracle as follows:

$$\mathcal{O}^H : \sum_{x,y} \alpha_{x,y} |x, y\rangle \mapsto \sum_{x,y} \alpha_{x,y} |x, y \oplus H(x)\rangle$$

The assumptions for the (Q)ROM model are stronger than any actual hash function implementation could ever meet, so a proof in the (Q)ROM is not as strong as a proof in the standard model. It can, however, serve as an indicator that the only possible point of failure is the interplay between hash function and algorithm.

In this paper, we will re-purpose the following QROM Lemma [Unr15, Lemma 6.2], called ‘One-way to Hiding’ in the literature. Intuitively, this lemma states that even a quantum algorithm interacting with a random oracle cannot distinguish an oracle output $H(x)$ from a random co-domain element y (‘hiding’) unless it poses a query with sufficient amplitude on x – in which case x can be measured and a preimage to $H(x)$ has been found (‘one-way’). (We made slight changes for the sake of accessibility, changing only syntax and naming.)

Lemma 2.5 (One-way to Hiding). *Consider an oracle algorithm B that makes at most q_H many (possibly quantum) queries to a (quantum-accessible) random oracle $H : \{0, 1\}^n \rightarrow \{0, 1\}^m$. Let C be an oracle algorithm that on input $x \in \{0, 1\}^n$ does the following: pick $i \leftarrow_{\$} \{1, \dots, q_H\}$ and $y \leftarrow_{\$} \{0, 1\}^m$, run $B^H(x, y)$ until (just before) the i -th query, measure the argument of the query input register in the computational basis, and output the measurement outcome. (When B makes less than i queries, C outputs $\perp \notin \{0, 1\}^n$.)*

Let

$$\begin{aligned} P_{H(x)} &:= \Pr[b' = 1 \mid b' \leftarrow B^H(x, H(x))] , \\ P_y &:= \Pr[b' = 1 \mid y \leftarrow_{\$} \{0, 1\}^m, b' \leftarrow B^H(x, y)] , \\ P_C &:= \Pr[x = x' \mid x' \leftarrow C^H(x)] , \end{aligned}$$

where each probability is also taken over the random oracle choice $H \leftarrow_{\$} (\{0, 1\}^n \rightarrow \{0, 1\}^m)$ and the input choice $x \leftarrow_{\$} \{0, 1\}^n$. Then

$$|P_{H(x)} - P_y| \leq 2q_H \sqrt{P_C} .$$

2.5 Quantum Key Distribution

Ever since the publication of [BB84], quantum key distribution (QKD) has promised to deliver cryptography that is secure under a weaker assumption than any public-key cryptography. More precisely, QKD is information-theoretically secure, meaning that any attempt by an attacker to gain information about the key will be detected with high probability, regardless of the (quantum) computational power to which the attacker has access to.

Since then, several protocols and variants have been proposed, and several manufacturers sell commercially available QKD devices implementing these protocols. Many QKD schemes have been proven to be theoretically secure, however the implementation of a QKD scheme could introduce additional vulnerabilities which are not covered by a theoretical security proof. These vulnerabilities could include implementation errors, side-channel attacks, and bad randomness. Furthermore, the

case needs to be considered where the QKD device itself cannot be trusted, for example because it is acquired from an untrusted vendor, or because of a supply-chain attack.

Concretely, the information-theoretic security of such protocols can be defined as follows: give an unbounded QKD adversary access to an oracle that either outputs the key that was generated during the protocol execution or a uniformly random one. The scheme is information-theoretically secure if the advantage the computationally unbounded adversary has with respect to guessing (based on all the quantum and classical information gathered when attacking the protocol) whether it received an honest key or a uniformly random key is negligible in the security parameter.

The information-theoretic security of QKD ensures that no amount of computation will let the adversary compromise security. However, if we use the QKD key in a primitive that is only computationally secure, then no ITS guarantees can be provided anymore. Moreover if QKD is used in a computationally secure primitive, often there exists an alternative solution built only from classical computational cryptography that can provide the same security guarantees but with lower system complexity and/or with better scalability [ACD⁺25]. This applies when we use QKD directly for data encryption, but also when we use QKD in a combined AKE protocol: if the combiner itself is only computationally secure, there might exist a simpler system using only computationally secure primitives that provides the same security guarantees. Therefore, it is essential to argue that ITS security of the combined AKE protocols is preserved in case QKD is secure, as we do in Theorem 5.5.

QKD interface (ETSI 014) Almost all deployed QKD servers deliver keys to their clients via the Representational State Transfer (REST)-based Application Programmer Interface (API) defined in [ETS19].⁴ The two most important methods are the “Get key” method and the “Get key with key IDs” method. In the former method, the initiating party inputs the identity of their peer and receives a pair consisting of secret random bytes (the key) and a corresponding key ID. Optionally, the requesting party specifies the size and number of keys they require. In the latter method, the peer inputs the identity of the initiating party and the key ID and receives the same key, if this key has been established earlier via the “Get key” method. Both methods require that keys are removed from the server upon delivery (and can therefore only be extracted once per party). No restrictions are put on the generation of the key ID. How the initiator communicates the key ID to their peer is out of scope for the standard, but it is an essential part of any key-exchange protocol, as exemplified by the Dependent-Key attack described in Fig. 5.

A significant part of the ETSI standard deals with multitenancy (multiple clients can request QKD keys from one server), but for our purpose we consider only single tenancy. This simplifies the identity management required when implementing the standard. In our description, a party corresponds to a secure site in ETSI terminology, and a party ID corresponds to either the client ID or the server ID.⁵

The above interface is reflected in our QKD oracle model, specified in Fig. 6. An important detail to notice is that users of the QKD keys do not have access to the QKD protocol execution itself, or even to the transcript that generated the key. In fact the delivered key could be a concatenation of (parts of) multiple QKD protocol executions. Only the key and its ID are available.

3 End-to-end protocol

On a high level, our end-to-end protocol (in Fig. 4) combines a post-quantum (or post-quantum-and-classical ‘hybrid’) triple KEM handshake with a concurrent QKD key exchange, in such a way that the

⁴ Other methods for key delivery exist, such as the ETSI 004 streaming API [ETS20] or proprietary interfaces, but for our purpose of modelling security these behave similarly.

⁵ In ETSI terminology the client would be the *Secure Application Entity* and the server would be the *Key Management Entity*.

output session key is ‘secure’ (indistinguishable from random, see Section 5) even if either the PQC handshake or QKD breaks. We will show that the output key is *statistically* indistinguishable if the QKD protocol is secure, i.e., we obtain ITS security against active attackers that are *computationally unbounded*, see Theorem 5.5 for the formal statement.

One might suspect that it is straightforward how to combine a KEM handshake with a QKD key exchange, e.g., by applying KEM combiners such as the ones proposed in [GHP18, BBF⁺19]. However, QKD is not a KEM! In this section, we first analyze difficulties with applying KEM combiners to our AKE-QKD scenario (see Section 3.1) – in short, most constructions require computational assumptions (and thus violate ITS security), and the only known construction that avoids this, called XtM, seems to be hard to instantiate. The reason is that XtM needs a MAC that meets strong requirements, and we found that the existing MAC instantiations do not meet these requirements.

We conclude that so far, we lack the tools to enable an information-theoretic security proof. We thus construct an alternative protocol in Section 3.2. Our alternative is somewhat similar to XtM, but it avoids the aforementioned gap (by using nested MACs to authenticate the transcript). Lastly, in Section 3.3, we argue that the MAC is in fact cryptographically relevant because it prevents the severely undesirable situation in which the attacker can enforce sessions with related keys (which AKE should definitely avoid).

3.1 Difficulties using KEM combiners

KEM combiners take multiple ‘base’ KEMs as input and transform them into a new KEM that should be secure as long as one of its input (base) KEMs is secure. We will now discuss why the known constructions are not fully suitable for our purposes.

No unconditionally IND-CCA secure constructions in [GHP18]. The KEM combiners introduced by Giacon, Heuer, and Poettering in [GHP18] use some concrete ‘core function’ that transforms the keys and ciphertexts generated by the base KEMs into a single output key. One example for such a core function would be XOR-ing the base keys. While this would provably preserve security against *passive* attackers (IND-CPA security), it does not preserve security against *active* attackers (IND-CCA security), which is the de-facto standard security goal for KEMs. This problem is reflected in a *Mix-and-Match attack* described by Bindel, Brendel, Fischlin, Goncalves, and Stebila in [BBF⁺19]: given a challenge ciphertext (c_1^*, c_2^*) , an adversary A can recover the challenge key $k_1^* \oplus k_2^*$ as follows: create two fresh encapsulations $(c_1, k_1) \leftarrow \text{Encaps}_1$ and $(c_2, k_2) \leftarrow \text{Encaps}_2$, make the two decapsulation queries (c_1, c_2^*) and (c_1^*, c_2) to obtain $k_1 \oplus k_2^*$ and $k_1^* \oplus k_2$, use the responses to recover k_1^* and k_2^* . As we will detail in Section 3.3, this attack informed our Dependent-Key attack on certain types of combined QKD-AKE protocols and thereby our own protocol design.

We note that [GHP18] does contain an IND-CCA-preserving combiner, but it is based on split-key PRFs and thus would introduce a computational assumption.

XOR-then-MAC (XtM) combiner. In [BBF⁺19], the authors also provide a combiner called XOR-then-MAC (XtM) combiner. To the best of our knowledge, this is the only known combiner that does not require computational assumptions to achieve IND-CCA security. Given two KEMs KEM_1 and KEM_2 , the XtM combiner acts as follows: For each KEM index $1 \leq i \leq 2$, obtain an encapsulation $(c_i, k_i) \leftarrow \text{Encaps}_i(pk_i)$, and parse k_i into two separate keys: $k_{\text{kem},i} \parallel k_{\text{mac},i} := k_i$. The XtM combiner’s final output key is the XOR of the respective keys’ first parts: $k := k_{\text{kem},1} \oplus k_{\text{kem},2}$. To prevent chosen-ciphertext attacks, the XtM combiner additionally computes a tag τ over both ciphertexts (which is verified during decapsulation): $\tau := \text{MAC}_{(k_{\text{mac},1}, k_{\text{mac},2})}(c_1 \parallel c_2)$. The combiner’s final ciphertext is (c_1, c_2, τ) .

Difficulties in meeting XtM’s requirements. To enable the proven IND-CCA statement, the combiner requires a MAC that is robust and one-time strong unforgeable. This is defined with

respect to the game Robust-OT-sEUF-CMA as given in Fig. 3: it is like the OT-sEUF-CMA game with a pair of keys and where the adversary can both choose one of the two keys and change it between generation and verification of the tag, capturing that one of the two keys can be broken. Concerning computational security, the authors discuss a solution based on HMAC.

Game Robust-OT-sEUF-CMA ^{M,A}	
01	$(k_1, k_2) \leftarrow \text{MKG}()$
02	$(m^*, b, k_b^*) \leftarrow A_0()$
03	if $b = 1$
04	$k^* := (k_1^*, k_2)$
05	else
06	$k^* := (k_1, k_2^*)$
07	$\tau^* \leftarrow \text{MAC}_{k^*}(m^*)$
08	$(m', \tau', k'_b) \leftarrow A_1^{\text{VER}(\cdot)}(\tau^*)$
09	if $b = 1$
10	$k' := (k'_1, k_2)$
11	else
12	$k' := (k_1, k'_2)$
13	return $\llbracket (m', \tau') \neq (m^*, \tau^*) \wedge \text{MVerify}_{k'}(m', \tau') \rrbracket$

Fig. 3: Security experiment for robust one-time strongly unforgeable MACs, simplified version of $\text{Expt}_{\mathcal{M}}^{\text{XZ-OT-sEUF}}(\mathcal{A})$ [BBF⁺18, Figure 7]. The full version specifies $\text{VER}(\cdot)$: a (possibly quantum) verification oracle where the adversary controls one key k_b , but this oracle is not relevant for our discussion.

The authors suggest four constructions for instantiating the MAC, the first three of which are claimed to be unconditionally secure in [BBF⁺19].

1. **Concatenating** two MACs, each computed under one of the keys: $\tau = (\tau_1, \tau_2) = (\text{MAC}_{k_1}^{(1)}(m), \text{MAC}_{k_2}^{(2)}(m))$ where $\text{MAC}^{(1)}$ and $\text{MAC}^{(2)}$ are both strongly secure MACs.
2. **XOR aggregation** of the two MACs: $\tau = \tau_1 \oplus \tau_2$.
3. **Carter-Wegman**: When using the Carter-Wegman paradigm [WC81], suppose one uses hashing of the form $am + b$ over some finite field, where $k = (a, b)$ is the MAC key. Then one can compute a single MAC over the key $k = k_1 + k_2 = (a_1 + a_2, b_1 + b_2)$, so that $\tau = (a_1 + a_2)m + b_1 + b_2$.
4. **Computationally**: Use HMAC to instantiate the MAC directly, or rely on the HKDF paradigm [Kra10].

We believe the first three constructions are not secure with respect to the Robust-OT-sEUF-CMA game. This is due to the fact that an adversary who can change one of the keys can change the tag accordingly, thus being able to compute a different valid tag on the same message. Formally, we demonstrate the problem by describing Robust-OT-sEUF-CMA-adversaries on each of the respective constructions.

1. The adversary chooses any message m^* and any key k_2^* , then gets $\tau^* = (\tau_1, \tau_2)$ where $\tau_2 = \text{MAC}_{k_2^*}^{(2)}(m^*)$. The adversary chooses any $k'_2 \neq k_2^*$ and computes $\tau'_2 = \text{MAC}_{k'_2}^{(2)}(m^*)$. (In the unlikely case that $\tau_2 = \tau'_2$, the adversary can resample k'_2 .) Now (m^*, τ', k'_2) with $\tau' = (\tau_1, \tau'_2)$ is a successful forgery, since the tags differ and τ' is valid under key k' .
2. Similar to the above, the adversary chooses any m^* , any k_2^* , and any $k'_2 \neq k_2^*$. Since the adversary knows $\tau_2 = \text{MAC}_{k_2^*}^{(2)}(m^*)$ and $\tau'_2 = \text{MAC}_{k'_2}^{(2)}(m^*)$ and is given τ^* , they can create a forgery $\tau' = \tau^* \oplus \tau_2 \oplus \tau'_2$ on message $m' = m^*$.

3. The adversary chooses any $m^* = m'$, any key $k_2^* = (a_2, b_2)$, and chooses $k_2' = (a_2, b_2')$ such that $b_2' \neq b_2$. When given τ^* , the adversary creates forgery $\tau' = \tau^* - b_2 + b_2'$.

We found no issue with the fourth construction, but it does make a computational assumption.

3.2 New combined protocol: protocol with nested MACs

We propose the protocol as shown in Fig. 4, which combines QKD with a post-quantum secure triple KEM AKE. The triple KEM protocol can be seen as a simplification of the FO_{AKE} protocol [HKSU20]. Alice and Bob share a QKD link, from which they can get shared QKD key k_{qkd} using the QKD-GET-KEY and QKD-GET-KEY-WITH-ID methods (we defer the formalization of these methods to Section 4). Triple KEM takes the encapsulations to three keypairs (two static and one ephemeral) and combines them using a KDF to get key k_{pqc} . The (concatenated) protocol transcript consists of the QKD key ID kID , the KEM ciphertexts and the ephemeral KEM public key.

Bob then splits the QKD key ($k_{qkd,m} \parallel k_{qkd,s} := k_{qkd}$) and the PQC key ($k_{pqc,m} \parallel k_{pqc,s} := k_{pqc}$). Using the MAC key $k_{qkd,m}$ he computes a MAC tag τ_1 over the transcript (and over the party IDs). Then he uses MAC key $k_{pqc,m}$ to compute a MAC tag τ_2 over the transcript *and over* τ_1 (and over the party IDs). He outputs session key $k_{sess} = k_{qkd} \oplus k_{pqc}$. Alice splits her keys the same way and if she can verify the tags, she will output the same session key.

Forward secrecy. The proof of security for this protocol is deferred to Section 5, after we introduced the security model. We already note, however, that it is known [Kra05] that pure two-message AKE protocols – like the KEM handshake used in our protocol to establish k_{pqc} – cannot possibly achieve forward secrecy against active attackers. That is, if an attacker actively replaces one of the parties and later also steals the long-term key of the party they replaced, no security could possibly be guaranteed. (In our case, the attacker could get k_{pqc} as follows: create (c_b, pk_e) , knowing k_b and sk_e , steal sk_A to get k_a , and use sk_e to get k_e .) Our combined protocol bypasses this result if QKD is secure and the attacker cannot access k_{qkd} , see Theorem 5.5. In the case that QKD is insecure, we still achieve weak forward secrecy: honestly executed sessions for which the long-term keys leak only later should still remain secure.

3.3 Motivation of our protocol: Dependent-Key attack

We will now argue that the MAC tags – which cryptographically bind the two handshakes – are in fact cryptographically relevant, in the sense that they are necessary to prevent the severely undesirable situation in which attackers can establish sessions with keys that are related to one another. (We will capture this below via a class of attacks we will call *Dependent-Key* attacks.) Crucially, AKE guarantees that an established session key remains unpredictable even if some other session keys leaked – session keys often end up in memory or logs, and AKE should prevent that leakage of one key endangers others, which we now show is not satisfied if the MAC tags are omitted.

Attack scenario. In more detail, the requirement of session key unpredictability even under leakage of session keys was already formalized in the first formal AKE security model [BR94]. It was modeled by allowing the attacker (Mallory) to perform ‘REVEAL’ queries that leak session keys of their choosing. This has two ‘trivial’ exceptions: (1) the session key itself cannot be leaked directly, and (2) if Mallory attacks one of Alice’s session keys, and they learns that key by forcing Bob to leak it on his side, then no security could possibly still be guaranteed. Mallory is thus forbidden from performing a REVEAL query on the ‘matching’ session that happens on Bob’s side.

Attack on protocol without MACs. Now consider the same protocol, but without MAC tags. In other words, the construction essentially becomes the XOR combiner, applied to two key exchanges:

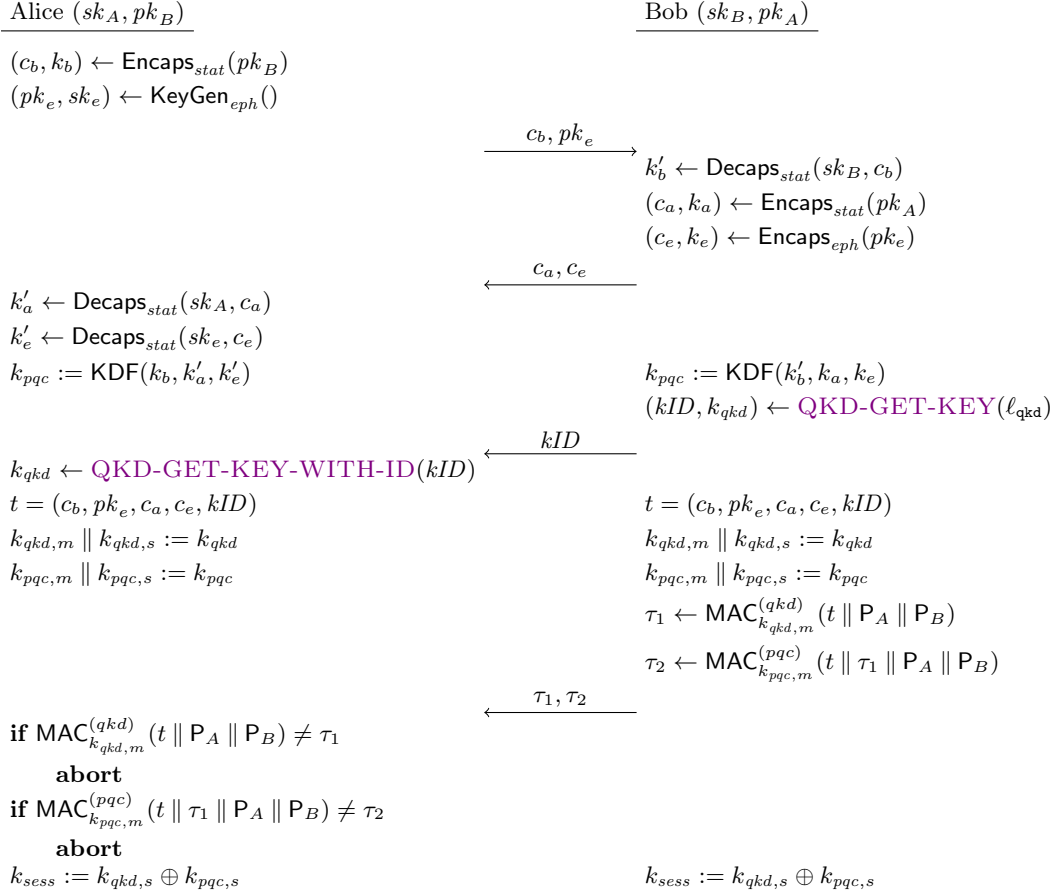


Fig. 4: Our new protocol that combines a triple-KEM handshake and QKD to establish a session key k_{sess} . Alice and Bob each have a private key (resp. sk_A, sk_B) and know each other's public key (resp. pk_A, pk_B) and party identifier (resp. P_A, P_B). They also share a direct QKD connection, accessible via **QKD-GET-KEY** and **QKD-GET-KEY-WITH-ID** (see Fig. 6 for a formalization of these, including session IDs). Key lengths (including ℓ_{qkd}) are public system parameters. Bob can combine his two messages into one, resulting in an efficient two-message protocol.

the session key $k_{sess} = k_{pqc} \oplus k_{qkd}$ is output immediately after Bob sent/Alice received Bob's ciphertext message (c_a, c_e) and the key ID. In Fig. 5, we depict a Dependent-Key attack on that protocol: The adversary, Mallory, sits on the wire between Alice and Bob, modifying Bob's outgoing messages on transit. Concretely, Mallory swaps the two key IDs Bob intended to use for the two sessions, thereby decoupling Alice's input messages from Bob's output messages. Effectively, the first session on Alice's side is thus no longer 'matched' to any session on Bob's side. As a result, *all* session keys are fair game to be leaked, except Alice's to-be-attacked key itself. Mallory is thus allowed to learn Alice's other session key as well as both of Bob's session keys. Suitably XORing these three keys allows them to learn Alice's to-be-attacked key, thereby completely breaking security.

Note that there exist many variants of the attack if Mallory should happen to exhibit additional powers – for example, if Mallory manages to learn Alice's long term key, then they can run session sID'_B directly with Bob himself and so they do not need session sID'_A to learn k'_A . Similarly if Mallory breaks PQC, then they do not need sID_B to learn k_{pqc} , and if they break QKD then they don't need sID'_B to learn k_{qkd} .

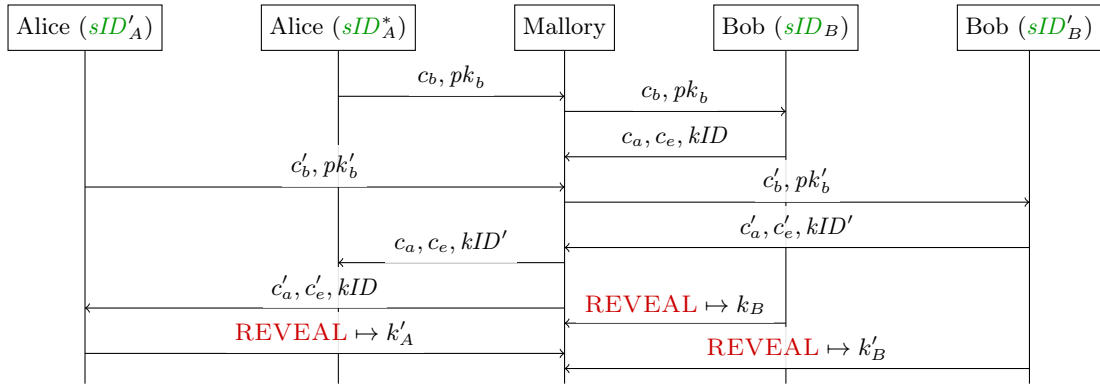


Fig. 5: Dependent-Key attack on the protocol without MACs. Mallory sits on the wire, aiming to attack test session sID_A^* . She modifies the two session's messages from Bob to Alice on transit, swapping the key IDs that were communicated by Bob. She is allowed to request leakage ('REVEAL') from the (non-matching) sessions sID'_A , sID_B and sID'_B , thereby learning the session keys $k'_A = k'_{pqc} \oplus k'_{qkd}$, $k_B = k_{pqc} \oplus k'_{qkd}$, and $k'_B = k'_{pqc} \oplus k_{qkd}$, respectively. She can now easily compute the to-be-attacked session key $k_A^* = k_{pqc} \oplus k_{qkd} = k'_A \oplus k_B \oplus k'_B$.

Preventing the attack via MAC tags. The exchange of MACs in our protocol (see Fig. 4) prevents this kind of Dependent-Key attacks – swapping the key IDs would now lead to differing transcripts on the two sides due to the differing tags, and thus, Alice would reject. We conclude that it is crucial to include the QKD key ID into the transcripts, thereby binding the used QKD key to the AKE session. If key IDs should not be available, e.g. due to QKD keys being delivered via some other interface, then the binding should occur via some other associated value. This could be the session ID or post-processing transcript associated to the QKD session that generated the used key.

4 QKD Oracles for End-to-end security

Following the game-based literature on Authenticated Key Exchange (see, e.g., [BR94, CK01, CK02, BCNP08, Kra05, LLM07, BCNP08, FSXY12, HKSU20]), we model

- security as key indistinguishability, via a game in which the attacker chooses a ‘test’ session whose key *sesKey* it aims to distinguish from random. To that end, the game provides an oracle **TEST** that the attacker can query for a session of its own choosing, and that returns either that session’s key *sesKey* or a uniformly random key. The attacker finishes by outputting a ‘guessing bit’ to indicate whether it believes the key to be real or random.
- adversarial interactions with (potentially many) honest parties by providing the attacker with black-box access to various ‘Send’ oracles that execute the protocol for parties of the attacker’s choosing. This choice reflects that the attacker is granted full control over the network – they could forward honest parties’ output faithfully as well as modify the data in transit. While we could technically do with a single Send query, for clarity we make the queries explicit for the individual messages of the protocol.
- adversarial interactions to extract secret data from the model, by which we model various attacks that may leak information to the adversary. Distinguishing the session key from random becomes easy when too much secret information has been revealed, for example when the adversary directly learned the test session key through a **REVEAL** query. In that case no protocol can be expected to provide any security, so we classify such attacks as *trivial* and we define security to mean that all non-trivial attacks succeed with at most negligible probability.

We detail the setup of the game’s execution environment in Section 4.1 below, after highlighting some key aspects of our model.

DIFFERENCES TO PREVIOUS QKD-RELATED MODELS. A major difference to the HAKE model is that we model the QKD functionality in a different way—unlike works in the HAKE framework [DHP20a, BRS23, GPH⁺24, BSP⁺24], we model the QKD link layer with an ideal functionality that can be accessed via oracle calls, see Fig. 6. With this model, we provide the adversary with explicit methods to both learn *and modify* QKD key material, reflecting the capabilities of an adversary that can compromise QKD. We explicitly model how key identifiers (*kIDs*) are used when interacting with real-world QKD devices [ETS19]. Unlike previous models, this allows us to capture Dependent-Key attacks in our model, which can lead to loss of security as we demonstrate in Section 3.3.

CAPTURING WEAKNESSES VIA ORACLES. Given that we want to investigate hybrid QKD-PKC protocols and to express security even if one of their components fail, our model will provide the attacker with oracles that reveal secret process data (see Section 4.2). We briefly summarize how typical attacks/weaknesses are captured by this model:

1. Man-in-the-middle attacks on end-to-end messages are reflected in the Send oracles.
2. Weak forward secrecy is captured through the **CORRUPT** query: even if the adversary reveals both long-term keys, the key of an honestly executed test session should be indistinguishable from random.
3. Malicious parties are captured via the **CORRUPT** query: after corruption, the adversary can impersonate (the PQC part of) the corrupted party.
4. Key Compromise Impersonation (KCI) is captured via the **CORRUPT** query: an adversary that corrupts a party can impersonate another party *to* the corrupted party, but then the test session key should still be indistinguishable from random.
5. Maximal EXposure (MEX) is captured via **CORRUPT** and **REVEAL-STATE** queries: when the test session contains at least an uncompromised long-term key or a non-revealed ephemeral key for both the initiator and responder, then the test session key should still be indistinguishable from random.
6. Broken QKD implementations are modelled by allowing adversarial access to the idealized QKD functionality, allowing both read-outs via **QKD-LEAK** and overwrites via **QKD-OVERRIDE**.

7. Dependent-Key attacks are captured through the **REVEAL** query: session keys of other sessions (except for a matching session) can be requested to be revealed, and even then, the test session key should still be indistinguishable from random.

4.1 Execution environment

The adversary drives the execution of the protocol. After generation of the long-term public keys and initialization of the QKD oracle, the adversary can interact with the model via oracle queries. Each session is uniquely identified by a session identifier sID . To ensure each session has a unique sID , a global counter sID_{ctr} is incremented whenever a new session is established via the **EST** query. In our model the sID is merely a unique identifier for disambiguating sessions, and it is not accessible by honest parties, unlike some other models where a (partial) local transcript represents the session ID.

STANDARD GAME VARIABLES. We consider a network of n_{pt} many nodes $P_1, \dots, P_{n_{pt}}$ that possibly have many sessions at once. Party IDs are represented by an index in $\{1, \dots, n_{pt}\}$. Each session runs the protocol with access to the involved parties' pre-established QKD/PKC material, while also having its own set of session-specific local variables:

- An integer **owner** $[sID] \in \{1, \dots, n_{pt}\}$ that points to the party running the session.
- An integer **peer** $[sID] \in \{1, \dots, n_{pt}\}$ that points to the session's intended communication partner.
- A string **role** $[sID] \in \{\text{INITIATOR}, \text{RESPONDER}\}$ indicating the party's role in the current session.
- A string **sent** $[sID]$, containing the message sent by the session.⁶
- A string **received** $[sID]$, containing the message received by the session.⁶
- A variable **state** $[sID]$, holding the state that will be revealed to the adversary. When a session accepts, this value is overwritten with the value **ACCEPT**, reflecting secure erasure. When a session aborts, this value is overwritten with the value **REJECT**.
- A variable **sesKey** $[sID]$, holding the session key (if the session is accepted), initialized at $\perp$.

ADVERSARIAL INTERACTION To drive the protocol, the adversary interacts with the model via the following queries.

- **EST** (i, j, ρ) establishes a new session sID with **owner** $[sID] = i$, **peer** $[sID] = j$, and **role** $[sID] = \rho$. The unique sID for this session is returned, which is generated by incrementing a global counter sID_{ctr} .
- **SEND-*** (sID, m) delivers a message m to session sID . The long-term IDs and keys of owner and peer are retrieved, as well as **state** $[sID]$. The message m is recorded in **received** $[sID]$, and then the protocol subroutine is called with the retrieved values, which returns an outgoing message m' and an updated state s' . The message m' is recorded in **sent** $[sID]$, and **state** $[sID]$ is overwritten with s' , then m' is returned to the adversary. If the protocol accepts, the state is erased (overwritten with **ACCEPT**) and **sesKey** $[sID]$ is set with the computed $sesKey$, and the session is *completed*. If the protocol **aborts**, then the state is erased (overwritten with **REJECT**) and no message is returned. Messages and state can be empty ($\perp$) for the first or last step of the protocol.
- **TEST** (b, sID) lets the adversary get a real-or-random key, depending on the game parameter b . The first call to **TEST** defines the test session sID^* , subsequent calls do nothing (the model enforces this by checking if sID^* was already set). If $b = 0$ the adversary gets the real session key **sesKey** $[sID^*]$, otherwise (if $b = 1$) the adversary gets a uniformly random bitstring. The goal for the adversary is to make a guess b' for the value b : if $b' = b$ the adversary wins the game.

⁶ When a protocol has more messages, this is a list of strings.

The above should be familiar to those readers acquainted with the CK^+ model.

MODELLING QKD. We model QKD with an ideal functionality that can be accessed by honest users via (classical) oracle calls **QKD-GET-KEY** and **QKD-GET-KEY-WITH-ID**. In addition we provide the adversary with the **QKD-LEAK** query to reveal QKD keys and **QKD-OVERRIDE** to modify the keys.

The QKD oracle, described in detail in Fig. 6, keeps track of some bookkeeping variables.

- The global counter kID_{ctr} provides a unique key ID per key by incrementing the counter for every new key that is requested. Real QKD devices may instead combine a local counter with a unique device ID, or deliver randomly generated key IDs from a sufficiently large space: as long as the probability of a key ID collision is negligible, our model applies.
- the array **qKey** stores the secret bytes per key ID;
- the array **qStatus** stores per key ID if the key is unknown to the adversary (**HONEST**), leaked to the adversary (**REVEALED**), or even set by the adversary (**CORRUPT**).
- the array **qSentSid** stores per key ID which sID received the key via **QKD-GET-KEY**.
- the array **qRecvSid** stores per key ID a list of $sIDs$, to track which sessions received the key via **QKD-GET-KEY-WITH-ID**. While **HONEST** or **REVEALED** keys will only be delivered once via this method, the oracle may deliver **CORRUPT** keys more than once.

The oracle is initialized with **QkdInit**, which sets kID_{ctr} to zero, and initializes the arrays to the empty value.

An honest session sID gets access to the oracles **QKD-GET-KEY**($sID, \cdot$) and **QKD-GET-KEY-WITH-ID**($sID, \cdot$):

- **QKD-GET-KEY**(sID, ℓ) returns a pair of a unique key identifier kID and a uniformly random key k of length ℓ . The oracle generates a unique kID by incrementing kID_{ctr} and records the calling sID in the **qSentSid** array. It then generates a new key and marks it **HONEST**, but only if **qKey**[kID] was not set before (which can only occur for a **CORRUPT** key). Keys are always available via this query, therefore our model does not capture denial-of-service attacks on QKD.
- **QKD-GET-KEY-WITH-ID**(sID, kID) returns **qKey**[kID], if the QKD keys is indeed shared between the communicating parties, and then erases the stored value. To verify that the key was indeed shared between the relevant parties, the **owner** and **peer** of the calling session sID are compared against the **peer** and **owner** of the session that was recorded in **qSentSid**[kID]. By binding the QKD keys to **owner/peer** arrays of the AKE model, we enforce end-to-end QKD. If a non-empty key is delivered, then the calling sID is appended to **qRecvSid**[kID].

We remark that **HONEST** keys are uniformly random in the QKD oracle, while real QKD keys have some negligibly small (but non-zero) statistical distance (ε) to uniformly random keys. We are already abstracting away many details of QKD, and we expect that this additional simplification does not significantly impact the security of hybrid protocols. For that reason we leave the proper modelling of ε -secure QKD as an open problem for future work.

Modelling QKD directly, including quantum and classical communication [MSU13], provides stronger security guarantees than modelling QKD via an oracle as we do in this work. Integration into an AKE protocol could then use the post-processing transcript as an input to prevent attacks such as the Dependent-Key attack. However, that would also requires that the AKE protocol has access to (the transcript of) QKD post-processing. Instead the typical ETSI 014 interface (see Section 2.5) provides only a key and corresponding ID.

QkdInit() 01 $kID_{ctr} := 0$ 02 $qKey := []$ 03 $qStatus := []$ 04 $qSentSid := []$ 05 $qRecvSid := []$ QKD-KEY-HOLDERS (kID) 06 return ($qSentSid[kID], qRecvSid[kID]$) QKD-LEAK (kID) 07 if $qStatus[kID] = \text{HONEST}$ 08 $qStatus[kID] := \text{REVEALED}$ 09 return $qKey[kID]$ QKD-OVERRIDE (kID, k) 10 $qKey[kID] := k$ 11 $qStatus[kID] := \text{CORRUPT}$	QKD-GET-KEY (sID, ℓ) 12 $kID_{ctr} := kID_{ctr} + 1$ 13 $qSentSid[kID_{ctr}] := sID$ 14 if $qKey[kID_{ctr}] = \perp$ 15 $qKey[kID_{ctr}] \leftarrow_{\$} \{0, 1\}^\ell$ 16 $qStatus[kID_{ctr}] := \text{HONEST}$ 17 $k := qKey[kID_{ctr}]$ 18 return (kID_{ctr}, k) QKD-GET-KEY-WITH-ID (sID, kID) 19 $sID_{qs} := qSentSid[kID]$ 20 if ($owner[sID_{qs}], peer[sID_{qs}] \neq$ ($peer[sID], owner[sID]$)) 21 return $\perp$ 22 if $qKey[kID] = \perp$ 23 return $\perp$ 24 $qRecvSid[kID_{ctr}] += [sID]$ 25 $k := qKey[kID]$ 26 $qKey[kID] := \perp$ 27 return k
---	---

Fig. 6: QKD oracle model. Honest sessions have access to the **QKD-GET-KEY**($sID, \cdot$) and **QKD-GET-KEY-WITH-ID**($sID, \cdot$) oracles (in purple), while the adversary may query **QKD-KEY-HOLDERS**, **QKD-LEAK**, and **QKD-OVERRIDE** (in red). The party calling **QKD-GET-KEY** specifies the key length ℓ . kID is a (predictable) key identifier. Besides the functionality, we add some **bookkeeping values** (in green) that are used in the proofs, but are not available to either the honest user or the adversary. Keys are removed in Line 26 from the oracle when they are requested by the receiver (the party calling **QKD-GET-KEY-WITH-ID**).

4.2 Modeling adversarial interaction

SECRET-REVEALING ORACLES. To account for the potential involvement of public-key algorithms, we assume—like in previous work on AKE—that public-key pairs, if such are used, have been distributed a priori. Given that we want our model to encompass hybrid QKD-PKC protocols, we want to be able to express security even upon failure of one of their components. We will model such failures by leakage of secret values like the involved parties’ long-term secret keys, or secret state information, which the attacker obtains via access to additional oracles:

- **REVEAL**(sID) leaks the (real) session key $\text{sesKey}[sID]$ to the adversary. The model sets the flag $\text{revealed}[sID]$ to record that this session key has been leaked.
- **CORRUPT**(i) leaks the long-term secret key sk_i belonging to party i to the adversary. The model sets the flag $\text{corrupted}[sID]$ to record that this party has been corrupted. The adversary can still interact with corrupted parties (for example via **SEND-*** queries).
- **REVEAL-STATE**(sID) leaks the state $\text{state}[sID]$ to the adversary. The model sets the flag $\text{revState}[sID]$ to record that the state of this session has been leaked.

Malicious interaction with the QKD model is modelled via the following adversary oracle queries:

- **QKD-LEAK**(kID) leaks $\text{qKey}[kID]$ to the adversary. If the status was not already set (to **CORRUPT**), the model sets $\text{qStatus}[kID] := \text{REVEALED}$ to record that the QKD key has been leaked,
- **QKD-OVERRIDE**(kID, k) allows the adversary to overwrite $\text{qKey}[kID] := k$, The model sets $\text{qStatus}[kID] := \text{CORRUPT}$ to record that the QKD key was set by the adversary.
- **QKD-KEY-HOLDERS**(kID) reveals to the adversary which sessions used the QKD key identified by kID . This does not correspond any attack, but rather reflects that we consider this information to be public.

The **QKD-LEAK** query corresponds to passive attacks on QKD where the adversary learns the honestly generated key. For example, this could reflect side-channel leakage from an honest execution of QKD. The **QKD-OVERRIDE** query corresponds to active attacks on QKD, where the adversary has some influence over the output that honest parties receive from the QKD device. For example, this could reflect implementation errors, bad randomness, supply chain attacks, or simply the fact that one of the executing QKD parties was corrupted. Some of these attacks may only provide the adversary with some malleability of the output, but letting the adversary choose these values freely only increases their power.

4.3 Session cleanness and security definition

If we would let the adversary call the above oracles without restrictions, then no protocol can prevent the adversary from guessing b . For example, revealing the test session key directly with **REVEAL**(sID^*) allows the adversary to compare directly against the output from the **TEST** query, thereby revealing the value of b . An attack that would work against *any* protocol is considered a trivial attack, which we formalize using predicates, encoded as subroutines in Fig. 8. The subroutine **TrivialPQC** defines what constitutes a trivial attack against the post-quantum cryptography in a protocol: when the adversary executes an attack such that **TrivialPQC** does *not* hold, then the protocol should be secure as long as the post-quantum primitives that it uses are secure. The subroutine **TrivialQKD** defines what are considered (trivial) attacks against QKD: when **TrivialQKD** does not hold, a protocol should be secure if the used QKD key and other primitives used in the protocol are secure. The notions are independent, such that a protocol should get security from both PQC and QKD against attacks that are neither **TrivialPQC** nor **TrivialQKD**. A session that is not trivially

broken is also called *clean*: we only require security for a clean test session. To define cleanliness, we first require a precise definition of matching sessions.

MATCHING SESSIONS. Two sessions sID_1 and sID_2 are *matching* if

- $(\text{owner}[sID_1], \text{peer}[sID_1]) = (\text{peer}[sID_2], \text{owner}[sID_2])$: they intend to communicate with each other,
- $(\text{sent}[sID_1], \text{received}[sID_1]) = (\text{received}[sID_2], \text{sent}[sID_2])$: they have matching transcripts, and
- $\text{role}[sID_1] \neq \text{role}[sID_2]$: they have different roles.

We are only concerned with sessions that match with the test session. The subroutine $\text{FindMatches}(sID^*)$ (defined in Fig. 8) creates a set *matches* containing all sessions that match with sID^* .

TRIVIALPQC. The **TrivialPQC** notion is taken from [HKSU20], with the exception that they inline FindMatches . **TrivialPQC** excludes certain combinations of queries because no PQC protocol can protect against them, as indicated by the attacks below.

1. If the test session did not accept, then the session key $\perp$ is trivially distinguished from random.
2. If the test session was revealed, then the session key is known to the adversary and easily distinguished from random.
3. If the long-term key and state of the test session were all leaked, then all secret values used to compute the test session key are known to the adversary.
4. If there is no matching session and the peer has been corrupted, then the test session may have been executed directly with the adversary.
5. If there is a matching session, that session may not be revealed.
6. If there is a matching session, that session needs to have at least one secret value (long-term key or state) so the adversary cannot recompute the session key.

In addition to the above list, if the adversary manages to create more than one session that matches with sID^* , then the attack is considered non-trivial. Note that this holds even if the adversary reveals (one of) those matching sessions, so effectively an adversary that can create multiple matching sessions can always win the game. This check formalizes a requirement of the CK^+ model [Kra05], which only considers protocols where the initial message is unique, while allowing us to quantify the security loss induced by lack of randomness in the protocol. Intuitively this check ensures that in a secure protocol the adversary cannot create multiple sessions that output the same key, which might lead to confusion attacks, for example if the adversary swaps different messages encrypted under the same key. The above is formally encoded by subroutine **TrivialPQC** in Fig. 8.

TRIVIALQKD. We can only expect security from QKD if the test session used at least one **HONEST** QKD key. Formally, we require that there exists a kID such that

- $\text{qStatus}[kID] = \text{HONEST}$, and
- $sID^* = \text{qSentSid}[kID]$ or $sID^* \in \text{qRecvSid}[kID]$.

We cannot guarantee security if the adversary revealed the test session or the matching session. However, we also want to reward the adversary if they can create multiple *completed* matching sessions, but only if the test session used an honest QKD key (otherwise a trivial adversary against QKD exists that can create multiple accepting sessions by setting the QKD key with **QKD-OVERRIDE** and delivering the key to multiple recipients).

The above is encoded by the subroutine **TrivialQKD** in Fig. 8. Note that if $\text{qStatus}[kID] = \text{HONEST}$, then $sID^* \in \text{qRecvSid}[kID]$ is equivalent to $[sID^*] = \text{qRecvSid}[kID]$, because $\text{qRecvSid}[kID]$ has length at most one, since **HONEST** keys are deleted after they are delivered once. Also note that

if the adversary manages to create multiple accepting matching sessions (without breaking all QKD keys used), then they are allowed to reveal those sessions and can therefore win the game.

In addition to the above, we also require that the test session accepted, it was not revealed and if there is one matching session then that can also not be revealed.

ATTACKPQC. Our protocol is vulnerable to an attack: the adversary can impersonate the responder without compromising the responder long-term key, instead they reveal the initiator state to reveal k_b . If the adversary also breaks QKD, they can complete the responder session themselves. This is a known attack on the classical protocol (without QKD) [HKSU20], and it is not considered a trivial attack because there exist protocols that do protect against this scenario. Our work aims to integrate QKD into an existing AKE protocol, not to fix the classical protocol. Note that QKD does help: the adversary must break QKD to exploit this attack. This attack is formally encoded by the subroutine **AttackPQC** in Fig. 8.

FORMAL DEFINITION. Figure 7 ties everything together and formalizes the above model. The three games IND-AA-PQC, IND-StAA-PQC, and IND-AA-QKD only differ by which cleanliness/trivial predicate they use. The games are defined relative to a challenge bit b , indicating if the adversary will receive a real or a random key. The challenger generates long-term public keys for each party and initializes the QKD oracle, then adversary receives the public keys and gets to interact with the parties and their secrets via the earlier defined oracles. The adversary then makes a guess b' for the challenge bit, which the game outputs if the adversary is considered non-trivial (and otherwise the game outputs 0). The adversary is said to *win* the game if the game output equals b .

CORRECTNESS DEFINITION. We require the protocol to be correct, which intuitively means that both participating parties will compute the same session key in the presence of a passive adversary. In the AKE literature [Kra05, FSXY12] this requirement is usually formalized by requiring that if two parties complete matching sessions, then they compute the same session key. Since we allow an active adversary to change QKD keys via the **QKD-OVERRIDE** query, we also need to exclude that capability to formally define what we mean by a passive adversary. We enforce this by requiring that all QKD keys used by the matching sessions are not **CORRUPT**.

Definition 4.1 (AKE-correctness). *An AKE protocol Π is δ -correct if the following holds. If sID and $\overline{sID}$ denote two sessions such that $\text{sesKey}[sID] \neq \perp$ and $\text{sesKey}[\overline{sID}] \neq \perp$ (they are complete), $\overline{sID} \in \text{FindMatches}(sID)$ (they are matching), and for all kID such that $kID = \text{qSentSid}$ or $kID \in \text{qRecvSid}$ it holds that $\text{qStatus}[kID] \neq \text{CORRUPT}$ (QKD was not tampered with), then*

$$\Pr[\text{sesKey}[sID] \neq \text{sesKey}[\overline{sID}]] \leq \delta.$$

We emphasize that both sessions need to be completed. Without this requirement, an adversary might duplicate a QKD key ID such that multiple honest sessions will request the QKD key using **QKD-GET-KEY-WITH-ID**, but only one will succeed. We consider this to be an active attack and therefore exclude it from the correctness definition.

SECURITY DEFINITION. For non-**TrivialPQC** attacks, we use one of the two existing notions in our security definition: the stronger notion of “key **IND**istinguishability against **Active Attacks** against **PQC**” can be defined using the IND-AA-PQC game, and the weaker notion of “key **IND**istinguishability against **Active Attacks** against **PQC** without **State** reveal in the test session” is defined using the IND-StAA-PQC game. We include both games in Fig. 7 for completeness, but we only use the (weaker) IND-StAA-PQC game in our analysis of the AKE protocol of Fig. 4. The difference from IND-StAA-PQC is that it calls **TrivialPQC** in Line 08 instead of **AttackPQC** in Line 18.

Definition 4.2 (StAA-PQC Advantage). *Define the game IND-StAA-PQC_b for $b \in \{0, 1\}$ as in Figs. 7 and 8. The advantage of an adversary A against an AKE protocol Π is*

$$\text{Adv}_{\Pi}^{\text{IND-StAA-PQC}}(A) = |\Pr[\text{IND-StAA-PQC}_0(\Pi, A) \Rightarrow 1] - \Pr[\text{IND-StAA-PQC}_1(\Pi, A) \Rightarrow 1]|.$$

GAME IND-AA-PQC_b(Π, A) <pre> 01 $sID_{ctr} := 0$ 02 $sID^* := 0$ 03 for $i \in \{1, \dots, n_{pt}\}$ 04 $(pk_i, sk_i) \leftarrow \text{KeyGen}_{stat}()$ 05 $\text{QkdInit}()$ 06 $b' \leftarrow A^{\mathcal{O}}(pk_1, \dots, pk_{n_{pt}})$ 07 $matches := \text{FindMatches}(sID^*)$ 08 if $\text{Trivial}(matches)$ 09 return 0 // adversary loses 10 return b' </pre> GAME IND-StAA-PQC_b(Π, A) <pre> 11 $sID_{ctr} := 0$ 12 $sID^* := 0$ 13 for $i \in \{1, \dots, n_{pt}\}$ 14 $(pk_i, sk_i) \leftarrow \text{KeyGen}_{stat}()$ 15 $\text{QkdInit}()$ 16 $b' \leftarrow A^{\mathcal{O}}(pk_1, \dots, pk_{n_{pt}})$ 17 $matches := \text{FindMatches}(sID^*)$ 18 if $\text{AttackPQC}(matches)$ 19 return 0 // adversary loses 20 return b' </pre> GAME IND-AA-QKD_b(Π, A) <pre> 21 $sID_{ctr} := 0$ 22 $sID^* := 0$ 23 for $i \in \{1, \dots, n_{pt}\}$ 24 $(pk_i, sk_i) \leftarrow \text{KeyGen}_{stat}()$ 25 $\text{QkdInit}()$ 26 $b' \leftarrow A^{\mathcal{O}}(pk_1, \dots, pk_{n_{pt}})$ 27 $matches := \text{FindMatches}(sID^*)$ 28 if $\text{TrivialQKD}(matches)$ 29 return 0 // adversary loses 30 return b' </pre> TEST(b, sID) <pre> 31 if $sID = 0$ or $sID^* \neq 0$: // only one query 32 return $\perp$ 33 $sID^* := sID$ 34 $k_0^* := \text{sesKey}[sID^*]$ 35 $k_1^* \leftarrow_{\\$} \{0, 1\}^{\ell_{\text{sess}}}$ 36 return k_b^* </pre> REVEAL(sID) <pre> 37 revealed[sID] := true 38 return sesKey[sID] </pre>	CORRUPT(i) <pre> 39 corrupted[i] := true 40 return sk_i </pre> REVEAL-STATE(sID) <pre> 41 revState[sID] := true 42 return state[sID] </pre> EST(i, j, ρ) <pre> 43 $sID_{ctr} := sID_{ctr} + 1$ 44 owner[sID_{ctr}] := i 45 peer[sID_{ctr}] := j 46 role[sID_{ctr}] := ρ 47 return sID_{ctr} </pre> SEND-INIT(sID) <pre> 48 (i, j) := (owner[sID], peer[sID]) 49 (ρ, s) := (role[sID], state[sID]) 50 if $i = \perp$ or $\rho \neq \text{INITIATOR}$ or $s \neq \perp$ 51 abort 52 (m_1, s') $\leftarrow \text{Init}(pk_j)$ 53 sent[sID] := m_1 54 state[sID] := s' 55 return m_1 </pre> SEND-M1(sID, m_1) <pre> 56 (i, j) := (owner[sID], peer[sID]) 57 (ρ, s) := (role[sID], state[sID]) 58 if $i = \perp$ or $\rho \neq \text{RESPONDER}$ or $s \neq \perp$ 59 abort 60 received[sID] := m_1 61 (m_2, k) $\leftarrow \text{SendM1}^{\text{QKD-GET-KEY}(sID, \cdot)}($ 62 i, sk_i, j, pk_j, m_1) 63 sent[sID] := m_2 64 sesKey[sID] := k 65 state[sID] := ACCEPT 66 return m_2 </pre> SEND-M2(sID, m_2) <pre> 66 (i, j) := (owner[sID], peer[sID]) 67 (ρ, s) := (role[sID], state[sID]) 68 if $i = \perp$ or $\rho \neq \text{INITIATOR}$ 69 abort 70 received[sID] := m_2 71 $k \leftarrow \text{SendM2}^{\text{QKD-GET-KEY-WITH-ID}(sID, \cdot)}($ 72 i, sk_i, j, s, m_2) 73 sesKey[sID] := k 74 state[sID] := ACCEPT </pre>
--	---

Fig. 7: Formal security game, adapted from [HKSU20]. Protocol Π is defined by sub-routines (Init, SendM1, SendM2). Adversary A gets access to oracle $\mathcal{O} = (\text{TEST}(b, \cdot), \text{REVEAL}, \text{CORRUPT}, \text{REVEAL-STATE}, \text{EST}, \text{SEND-INIT}, \text{SEND-M1}, \text{SEND-M2}, \text{QKD-KEY-HOLDERS}, \text{QKD-LEAK}, \text{QKD-OVERRIDE})$.

```

AttackPQC(matches)
74 return TrivialPQC(matches) or (matches =  $\emptyset$  and revState[sID∗])

TrivialPQC(matches)
75 if sesKey[sID∗] =  $\perp$ 
76   return true // test session incomplete
77 if revealed[sID∗]
78   return true // test session key revealed directly
79 if corrupted[owner[sID∗]] and revState[sID∗]
80   return true // adversary can complete test session
81 if |matches| > 1
82   return false // multiple matching sessions
83 if matches =  $\emptyset$ 
84   return corrupted[peer[sID∗]] // session completed directly with adversary
85 { sID } := matches // only one match with session ID sID
86 return revealed[sID] or // session key leaked directly via matching session
      (corrupted[owner[sID]] and revState[sID]) // adversary can complete peer session

TrivialQKD(matches)
87 if sesKey[sID∗] =  $\perp$ 
88   return true // test session incomplete
89 if revealed[sID∗]
90   return true // test session key revealed directly
91 complete := Completed(matches)
92 if |complete| = 1
93   { sID } := complete // only one completed match with session ID sID
94   if revealed[sID] // session key leaked directly via matching session
95     return true
96 for 1 ≤ kID ≤ kIDctr
97   if qStatus[kID] = HONEST and (sID∗ = qSentSid[kID] or [sID∗] = qRecvSid[kID])
98     return false // an honest QKD key was delivered to the test session
99 return true // no honest QKD key was delivered to the test session

Completed(matches) // filter out uncomplete sessions (from matches)
100 complete :=  $\emptyset$ 
101 for sID ∈ matches
102   if sesKey[sID] ≠  $\perp$  // session completed
103     complete := complete ∪ { sID }
104 return complete

FindMatches(sID∗) // create set of sessions matching with sID∗
105 matches :=  $\emptyset$ 
106 (i, j) := (owner[sID∗], peer[sID∗])
107 for 1 ≤ sID ≤ sIDctr
108   if owner[sID] = j and peer[sID] = i and // intended partners
      received[sID] = sent[sID∗] and
      sent[sID] = received[sID∗] and // matching transcripts
      role[sID] ≠ role[sID∗] // different roles
109   matches := matches ∪ { sID } // add to set of matching sessions
110 return matches

```

Fig. 8: A protocol is considered trivially broken if too much information is revealed, such that no expectation of security remains for *any* protocol.

Against non-**TrivialQKD** attacks, we define a new security notion: “key **IND**istinguishability against **Active Attacks** against **QKD**”, for which we use the IND-AA-QKD game.

Definition 4.3 (AA-QKD Advantage). Define the game IND-AA-QKD_b for $b \in \{0, 1\}$ as in Figs. 7 and 8. The advantage of an adversary A against an AKE protocol Π is

$$\text{Adv}_{\Pi}^{\text{IND-AA-QKD}}(A) = |\Pr[\text{IND-AA-QKD}_0(\Pi, A) \Rightarrow 1] - \Pr[\text{IND-AA-QKD}_1(\Pi, A) \Rightarrow 1]|.$$

5 Security proof

Fig. 9 formally describes our authenticated key exchange protocol with a QKD oracle from Fig. 4.

Init (pk_j) 01 $(c_b, k_b) \leftarrow \text{Encaps}_{\text{stat}}(pk_j)$ 02 $(pk_e, sk_e) \leftarrow \text{KeyGen}_{\text{eph}}()$ 03 $m_1 := (c_b, pk_e)$ 04 $s' := (k_b, sk_e, m_1)$ 05 return (m_1, s')	SendM2 ^{QKD-GET-KEY-WITH-ID} ($sID, \cdot$)(i, sk_i, j, s, m_2) 16 $(c_a, c_e, kID, \tau'_1, \tau'_2) := m_2$ // or abort 17 $(k_b, sk_e, m_1) := s$ // or abort 18 $k'_a := \text{Decaps}_{\text{stat}}(sk_i, c_a)$ 19 $k'_e := \text{Decaps}_{\text{eph}}(sk_e, c_e)$ 20 $k_{\text{pqc}} := \text{KDF}(k_b, k'_a, k'_e)$ 21 $k_{\text{qkd}} :=$ QKD-GET-KEY-WITH-ID (sID, kID) 22 if $k_{\text{qkd}} = \perp$: abort 23 $t := (m_1, (c_a, c_e, kID))$ 24 $(k_{\text{sess}}, \tau_1, \tau_2) := \text{Combine}(i, j, t, k_{\text{qkd}}, k_{\text{pqc}})$ 25 if $\tau_1 \neq \tau'_1$ or $\tau_2 \neq \tau'_2$: abort 26 return k_{sess}
SendM1 ^{QKD-GET-KEY} ($sID, \cdot$)(i, sk_i, j, pk_j, m_1) 06 $(c_b, pk_e) := m_1$ // or abort 07 $k'_b := \text{Decaps}_{\text{stat}}(sk_i, c_b)$ 08 $(c_a, k_a) \leftarrow \text{Encaps}_{\text{stat}}(pk_j)$ 09 $(c_e, k_e) \leftarrow \text{Encaps}_{\text{eph}}(pk_e)$ 10 $k_{\text{pqc}} := \text{KDF}(k'_b, k_a, k_e)$ 11 $(kID, k_{\text{qkd}}) \leftarrow$ QKD-GET-KEY (sID, ℓ_{qkd}) 12 $t := (m_1, (c_a, c_e, kID))$ 13 $(k_{\text{sess}}, \tau_1, \tau_2) := \text{Combine}(j, i, t, k_{\text{qkd}}, k_{\text{pqc}})$ 14 $m_2 := (c_a, c_e, kID, \tau_1, \tau_2)$ 15 return (m_2, k_{sess})	Combine ($i_I, i_R, t, k_{\text{qkd}}, k_{\text{pqc}}$) 27 $(k_{\text{qkd}, m} \parallel k_{\text{qkd}, s}) := k_{\text{qkd}}$ 28 $(k_{\text{pqc}, m} \parallel k_{\text{pqc}, s}) := k_{\text{pqc}}$ 29 $k_{\text{sess}} := k_{\text{qkd}, s} \oplus k_{\text{pqc}, s}$ 30 $\tau_1 := \text{MAC}_{k_{\text{qkd}, m}}^{(qkd)}((t, i_I, i_R))$ 31 $\tau_2 := \text{MAC}_{k_{\text{pqc}, m}}^{(pqc)}((t, \tau_1, i_I, i_R))$ 32 return $(k_{\text{sess}}, \tau_1, \tau_2)$

Fig. 9: A formal implementation of our AKE protocol $\Pi = (\text{Init}, \text{SendM1}, \text{SendM2})$, using the QKD oracle (**QKD-GET-KEY**, **QKD-GET-KEY-WITH-ID**), in the case of end-to-end QKD.

We use the notation $(x \parallel y)$ to split keys into their required lengths: in all protocols the first part becomes a MAC key, so x should have the length of a MAC key and y should be the remaining bits. We use destructuring assignments to “parse” the incoming messages and state, this should be interpreted as a shorthand notation for checking if the parsed value has the correct format and aborting the session if it has not.

Our protocol uses two MACs, the inner $\text{MAC}^{(qkd)}$ uses a key generated by QKD, and we require statistical security: strong unforgeability (see Definition 2.4) must hold against computationally unbounded adversaries. Indeed such MACs exist [WC81] and are usually canonical, but a non-canonical MAC would suffice. The outer $\text{MAC}^{(pqc)}$ uses a key generated by PQC primitives, this MAC must either have statistical security, or it must be canonical (in which case computational security suffices). This property protects against a (potentially unbounded) adversary that changes

a matching session into a non-matching one by modifying the outer tag exchanged between two honest parties: both statistically secure and canonical MACs will reject such modifications. For simplicity, we assume both MACs are canonical.

Note that the order of nesting MACs is important. Consider the alternative protocol that is identical to Fig. 9, but computes $\text{MAC}^{(pqc)}$ first and $\text{MAC}^{(qkd)}$ second: this protocol is vulnerable to the following attack. An adversary can query **QKD-OVERRIDE** to change k_{qkd} without changing kID . This modification of the key does not change the transcript between honest users, so that $\text{MAC}^{(pqc)}$ does not detect anything, yet the honest parties will compute $\text{MAC}^{(qkd)}$ with a different key (so that the sent tag almost certainly differs from the one that is accepted). The adversary (who knows k_{qkd}) can swap out the QKD tag, so that both parties accept with the same session key, but their sessions are non-matching, and thus the adversary may reveal the peer session key.

This attack does not apply to Fig. 9, because the adversary cannot change k_{pqc} without changing the transcript. The reason is that $\text{MAC}^{(qkd)}$ authenticates the full transcript, and by correctness of the KEMs, both honest parties will (almost certainly) compute the same k_{pqc} and thus the sent tag is the only one that is accepted.

First we prove that the protocol Π defined in Fig. 9 is correct when implemented with correct components.

Theorem 5.1 (AKE-correctness). *Let Π be the AKE protocol defined in Fig. 9, where KEM_{eph} is a δ_{eph} -correct KEM and KEM_{stat} is a δ_{stat} -correct KEM. Then Π is a $(\delta_{eph} + 2\delta_{stat})$ -correct AKE.*

Proof. Let sID and $\overline{sID}$ be complete matching sessions with uncorrupted QKD keys, according to Definition 4.1. Since the sessions match, one of them is an initiator (sID_{init}), the other is a responder (sID_{resp}) and the transcript contains a single QKD key ID (kID^*). Because sID_{init} completed it received a QKD key: $k_{qkd} := \text{QKD-GET-KEY-WITH-ID}(sID_{resp}, kID^*) \neq \perp$, and since $\text{qStatus}[kID^*] \neq \text{CORRUPT}$, both sessions received the same QKD key.

For a matching session, the only other source of error is a decapsulation failure. The protocol contains three decapsulations, and so the probability of any failure is at most the sum of the probability that either one of them fails, so that

$$\Pr[\text{sesKey}[sID] \neq \text{sesKey}[\overline{sID}]] \leq \delta_{eph} + 2\delta_{stat}.$$

□

We now give a bound on the advantage that any (computationally bounded) adversary has on protocol Π (defined in Fig. 9) based on the advantage on its secure PQC components.

Theorem 5.2 (PQC-based security). *Let Π be the protocol of Fig. 9, where KEM_{eph} is δ_{eph} -correct and has key length ℓ_{eph} , and KEM_{stat} is δ_{stat} -correct, has collision probabilities $\mu(\text{Encaps}_{stat})$ and $\mu(\text{Secret}_{stat})$, and has key length ℓ_{stat} . Let A be an IND-StAA-PQC adversary executing Π with n_{pt} parties that establishes n_s sessions that makes at most q_H calls to the key-derivation function KDF (modelled as a QROM). Then there exist IND-CPA adversary B_1 against KEM_{eph} , IND-CPA adversary B_2 against KEM_{stat} , IND-CCA adversary B_3 against KEM_{stat} , and OT-sEUF-CMA adversary B_4 against $\text{MAC}^{(pqc)}$ such that*

$$\begin{aligned} \text{Adv}^{\text{IND-StAA-PQC}}(\Pi, A) \leq & 2n_s^2 \left(\delta_{eph} + \text{Adv}_{\text{KEM}_{eph}}^{\text{IND-CPA}}(B_1) + 2q_H \frac{1}{\sqrt{2^{\ell_{eph}}}} \right) \\ & + 2n_s^2 n_{pt} \left(\delta_{stat} + \text{Adv}_{\text{KEM}_{stat}}^{\text{IND-CCA}}(B_2) + 2q_H \frac{1}{\sqrt{2^{\ell_{stat}}}} \right) + 2 \cdot n_s \cdot \mu(\text{Encaps}_{stat}) \\ & + 8n_s n_{pt} \left(\delta_{stat} + \text{Adv}_{\text{KEM}_{stat}}^{\text{IND-CCA}}(B_3) + n_s^2 \mu(\text{Secret}_{stat}) + n_s \frac{2q_H}{\sqrt{2^{\ell_{stat}}}} + n_s \text{Adv}_{\text{MAC}^{(pqc)}}^{\text{OT-sEUF-CMA}}(B_4) \right) \end{aligned}$$

where the runtime of B_1, B_2, B_3, B_4 is about that of A .

Readers familiar with the KEM-TLS protocol and its security proof [SSW20], might be surprised to see that IND-CPA security suffices for KEM_{eph} in our protocol. The reason our protocol does not require IND-1CCA (a one-time variant of IND-CCA security) is that an initiator session *immediately* aborts when an adversary modifies or replaces the ephemeral encapsulation, because they cannot accompany that same message with the correct MAC tags.

Proof (PQC-based security). We split the proof into two cases. The first case, Case 1, concerns the case $\text{matches} \neq \emptyset$. For the second case (Case 2), we look at $\text{matches} = \emptyset$.

Case 1: passive attack is formally treated in Section 5.1. In this case, a matching session exists. We will prove that with overwhelming probability, this matching session must be unique. Since we are assuming that there are matching sessions, and since the responder session does not have a state, there is no benefit to the attacker in revealing the state of the responder. Therefore, assuming there is exactly one matching session, we can always assume that the state of the responder session is not revealed.

We will split Case 1 into two sub-cases:

- 1a** The state of the initiator has not been revealed. Thus, the ephemeral secret key sk_e has not been revealed to the attacker, and we base security of the final session key on the security of the ephemeral KEM.
- 1b** The initiator has not been corrupted. Thus, the initiator's static key sk_i has not been revealed to the attacker. We base security of the final session key on the security of the initiator static KEM.

Lemma 5.3 (Case 1). *There exist adversaries B_1, B_2 such that*

$$\begin{aligned} & \left| \Pr[\text{IND-StAA-PQC}_1(\Pi, A) \Rightarrow 1 \wedge \text{matches} \neq \emptyset] - \Pr[\text{IND-StAA-PQC}_0(\Pi, A) \Rightarrow 1 \wedge \text{matches} \neq \emptyset] \right| \\ & \leq 2n_s^2 \cdot \left(\delta_{eph} + \text{Adv}_{\text{KEM}}^{\text{IND-CPA}}(B_1) + 2q_H \frac{1}{\sqrt{2^{\ell_{eph}}}} + n_{pt} \left(\delta_{stat} + \text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(B_2) + 2q_H \frac{1}{\sqrt{2^{\ell_{stat}}}} \right) \right) \\ & \quad + 2 \cdot n_s \cdot \mu(\text{Encaps}_{stat}) \end{aligned}$$

and the runtime of B_1, B_2 is about that of A .

Case 2: active attack is formally treated in Section 5.2. This means that no matching session exists. Since we only consider non-trivial attacks, we in particular rule out that the test session's peer has been corrupted. It is thus safe to assume that their static key has not been revealed, and we base security of k_{pqc} on the security of the peer's static KEM. It remains to consider the case that there is a non-matching session that computes the same k_{pqc} , for example if the adversary swapped out kID or changes one of the MAC tags. However, then by the unforgeability of $\text{MAC}^{(pqc)}$, this will be detected by the initiator who will then abort. That means there is only one accepting (responder) session whose session key depends on k_{pqc} , and thus k_{pqc} is independent from the adversary view.

Lemma 5.4 (Case 2). *There exist adversaries B_3, B_4 such that*

$$\begin{aligned} & \left| \Pr[\text{IND-StAA-PQC}_0(\Pi, A) \Rightarrow 1 \wedge \text{matches} = \emptyset] \right. \\ & \quad \left. - \Pr[\text{IND-StAA-PQC}_1(\Pi, A) \Rightarrow 1 \wedge \text{matches} = \emptyset] \right| \\ & \leq 8n_s n_{pt} \left(\delta_{stat} + \text{Adv}_{\text{KEM}_{stat}}^{\text{IND-CCA}}(B_3) + n_s^2 \mu(\text{Secret}_{stat}) + n_s \frac{2q_H}{\sqrt{2^{\ell_{stat}}}} + n_s \text{Adv}_{\text{MAC}^{(pqc)}}^{\text{OT-sEUF-CMA}}(B_4) \right) \end{aligned}$$

and the runtime of B_3, B_4 is about that of A .

In order to bound

$$|\Pr[\text{IND-StAA-PQC}_0(\Pi, A) \Rightarrow 1] - \Pr[\text{IND-StAA-PQC}_1(\Pi, A) \Rightarrow 1]|$$

we combine Lemma 5.3 and Lemma 5.4 by taking the sum of the bounds. This gives

$$\begin{aligned} & |\Pr[\text{IND-StAA-PQC}_0(\Pi, A) \Rightarrow 1] - \Pr[\text{IND-StAA-PQC}_1(\Pi, A) \Rightarrow 1]| \\ & \leq 2n_s^2 \left(\delta_{eph} + \text{Adv}_{\text{KEM}_{eph}}^{\text{IND-CPA}}(B_1) + 2q_H \frac{1}{\sqrt{2^{\ell_{eph}}}} \right) \\ & + 2n_s^2 n_{pt} \left(\delta_{stat} + \text{Adv}_{\text{KEM}_{stat}}^{\text{IND-CCA}}(B_2) + 2q_H \frac{1}{\sqrt{2^{\ell_{stat}}}} \right) + 2 \cdot n_s \cdot \mu(\text{Encaps}_{stat}) \\ & + 8n_s n_{pt} \left(\delta_{stat} + \text{Adv}_{\text{KEM}_{stat}}^{\text{IND-CCA}}(B_3) + n_s^2 \mu(\text{Secret}_{stat}) + n_s \frac{2q_H}{\sqrt{2^{\ell_{stat}}}} + n_s \text{Adv}_{\text{MAC}^{(pqc)}}^{\text{OT-sEUF-CMA}}(B_4) \right). \end{aligned}$$

The proofs of the Lemmas 5.3 and 5.4 can be found in the following sections.

We now give a bound on the advantage that any (potentially unbounded) adversary has on protocol Π (defined in Fig. 9) based on the advantage on its secure QKD components.

Theorem 5.5 (QKD-based security). *Let Π be the protocol of Fig. 9, where KEM_{eph} is δ_{eph} -correct and KEM_{stat} is δ_{stat} -correct. Let A be an IND-AA-QKD adversary executing Π that establishes n_s sessions. Then there exist an OT-sEUF-CMA adversary B_7 against $\text{MAC}^{(qkd)}$ such that*

$$\text{Adv}^{\text{IND-AA-QKD}}(\Pi, A) \leq 2n_s \left(n_s(2\delta_{stat} + \delta_{eph}) + \text{Adv}_{\text{MAC}^{(qkd)}}^{\text{OT-sEUF-CMA}}(B_7) \right)$$

where the runtime of B_7 is potentially unbounded.

Proof sketch (QKD-based security). The formal proof for Theorem 5.5 is given in Section 5.3. The idea is that the QKD key has not leaked to (and has not been set by) the adversary. Our QKD oracle guarantees that honest QKD keys can be handed out at most twice. Thus, we focus on the test session and the session that uses the same QKD key. In our protocol Π , this will always be an initiator/responder pair. We base security of the final session key on the MAC, this ensures that the responder session is a matching session, or the initiator will abort. In both cases, the other session's session key cannot be revealed. Thus, the QKD key (and thus the session key) looks uniformly random to the adversary.

5.1 Security proof for Case 1: passive attack, Lemma 5.3

Formally, we want to upper bound

$$|\Pr[\text{IND-StAA-PQC}_1(\Pi, A) \Rightarrow 1 \wedge \text{matches} \neq \emptyset] - \Pr[\text{IND-StAA-PQC}_0(\Pi, A) \Rightarrow 1 \wedge \text{matches} \neq \emptyset]| ,$$

which we will do via a sequence of games.

In the following section, we will first switch to game G_1 to ensure we are in the correct case, after that the hop to game G_2 ensures that there is precisely one matching session. Game G_3 makes a guess which two sessions will be the initiator and responder, so that in any game hereafter we can talk about ‘the’ initiator and responder sessions.

After that, we split the rest of the proof into two subcases: Case 1a where the ephemeral secret key of the initiator session has not been revealed and Case 1b where the static secret key of initiator has not been revealed.

```

GAME  $G_{0,b} - G_{3,b}$ 
01  $sID_{ctr} := 0$  // session counter
02  $sID^* := 0$  // test session ID
03  $s_{init} \leftarrow_{\$} \{1, \dots, n_s\}$  //  $G_{3,b}$ : guess initiator
04  $s_{resp} \leftarrow_{\$} \{1, \dots, n_s\}$  //  $G_{3,b}$ : guess responder
05 for  $i \in \{1, \dots, n_{pt}\}$ 
06    $(pk_i, sk_i) \leftarrow \text{KeyGen}_{stat}()$ 
07  $\text{QkdInit}()$ 
08  $b' \leftarrow A^{\circ}(pk_1, \dots, pk_{n_{pt}})$ 
09  $matches := \text{FindMatches}(sID^*)$ 
10 if  $matches = \emptyset$ : return 0 //  $G_{1,b}$ : ensure matching session exists
11 if  $|matches| > 1$ : return 0 //  $G_{2,b}$ : ensure at most one matching session
12 if  $\text{AttackPQC}(matches)$ 
13   return 0
14 Pick  $sID_{init}, sID_{resp} \in \{sID^*, \overline{sID}\}$ 
   s.t.  $\text{role}[sID_{init}] = \text{INITIATOR}$  and  $\text{role}[sID_{resp}] = \text{RESPONDER}$  //  $G_{3,b}$ 
15 if  $sID_{init} \neq s_{init}$  or  $sID_{resp} \neq s_{resp}$ 
16   return 0 //  $G_{3,b}$ 
17 return  $b'$ 

```

Fig. 10: $G_{0,b} - G_{3,b}$. Any oracle unchanged is not included.

Games $G_{0,b}$ For each challenge bit $b \in \{0, 1\}$, game $G_{0,b}$ is the original game $\text{IND-StAA-PQC}_b(\Pi, A)$, so

$$\Pr[\text{IND-StAA-PQC}_b(\Pi, A) \Rightarrow 1 \wedge matches \neq \emptyset] = \Pr[G_{0,b} \Rightarrow 1 \wedge matches \neq \emptyset].$$

Games $G_{1,b}$ In this game, we add the check that there is at least one matching session in Line 10, to ensure we are in the right scenario. For each challenge bit $b \in \{0, 1\}$, the new game $G_{1,b}$ only differs from $G_{0,b}$ when $matches = \emptyset$. We're only considering the case where $matches \neq \emptyset$ so we have

$$\Pr[G_{0,b} \Rightarrow 1 \wedge matches \neq \emptyset] = \Pr[G_{1,b} \Rightarrow 1] .$$

Games $G_{2,b}$ For each challenge bit $b \in \{0, 1\}$, we introduce a new game $G_{2,b}$ that returns 0 in Line 11 if there is more than one matching session. Since $G_{1,b}$ and $G_{2,b}$ only differ upon this case, we have

$$|\Pr[G_{1,b} \Rightarrow 1] - \Pr[G_{2,b} \Rightarrow 1]| \leq \Pr[\text{Return in Line 11}] .$$

We now bound $\Pr[\text{Return in Line 11}]$ for each of the two possible roles the test session could have. First, assume that $\text{role}[sID^*] = \text{INITIATOR}$. Having multiple matching sessions of sID^* means that there are two distinct RESPONDER sessions sID_1 and sID_2 that sent the same first response. Formally, this means $\text{sent}[sID_1] = [c_a, c_e, kID, \tau_1, \tau_2] = \text{sent}[sID_2]$. Since all ciphertexts were generated honestly, the probability that both the static and ephemeral ciphertexts are equal for both sessions is at most $\mu(\text{Encaps}_{stat}) \cdot \mu(\text{Encaps}_{eph})$ (see Definition 2.2). Since there are at most $(n_s - 1)$ possible sessions that could create this situation and since $\mu(\text{Encaps}_{eph}) \leq 1$, we have

$$\begin{aligned} \Pr[\text{Return in Line 11} \wedge \text{role}[sID^*] = \text{INITIATOR}] &\leq (n_s - 1) \cdot \mu(\text{Encaps}_{eph}) \cdot \mu(\text{Encaps}_{stat}) \\ &\leq n_s \cdot \mu(\text{Encaps}_{stat}). \end{aligned}$$

Now consider the case that $\text{role}[sID^*] = \text{RESPONDER}$. Having multiple matching sessions of sID^* means there are two distinct INITIATOR sessions sID_1 and sID_2 which sent the same initial message. Formally, this means $\text{sent}[sID_1] = [c_b, pk_e] = \text{sent}[sID_2]$. This means that both sessions

need to generate the same ciphertext to Bob's static key, which happens with probability at most $\mu(\text{Encaps}_{stat})$. Similarly, the probability that both sessions generated the same public key pk_e is at most $\mu(\text{KeyGen}_{eph})$ (see Definition 2.2). Since there are at most $(n_s - 1)$ sessions that could be matching, we have

$$\begin{aligned} \Pr[\text{Return in Line 11} \wedge \text{role}[sID^*] = \text{RESPONDER}] &\leq (n_s - 1) \cdot \mu(\text{Encaps}_{stat}) \cdot \mu(\text{KeyGen}_{eph}) \\ &\leq n_s \cdot \mu(\text{Encaps}_{stat}) . \end{aligned}$$

In total, this gives

$$\Pr[\text{Return in Line 11}] \leq n_s \cdot \mu(\text{Encaps}_{stat}) .$$

So far, we have

$$\begin{aligned} &|\Pr[\text{IND-StAA-PQC}_1(\Pi, A) \Rightarrow 1 \wedge \text{matches} \neq \emptyset] - \Pr[\text{IND-StAA-PQC}_0(\Pi, A) \Rightarrow 1 \wedge \text{matches} \neq \emptyset]| \\ &\leq |\Pr[G_{2,1} \Rightarrow 1] - \Pr[G_{2,0} \Rightarrow 1]| + 2 \cdot n_s \cdot \mu(\text{Encaps}_{stat}) . \end{aligned}$$

If games $G_{0,b} - G_{2,b}$ do not return early, we can safely assume that $|\text{matches}| = 1$. In other words, there is exactly one matching session of sID^* , which we will denote by $\overline{sID}$.

Games $G_{3,b}$ At the start of both games $G_{3,b}$, we guess the session IDs of the init and responder sessions in Line 03 and Line 04, and we check the guesses are correct in Line 14 to Line 16. If the session IDs are guessed correctly, $G_{2,b}$ and $G_{3,b}$ are equal, and if one of the sessions IDs has not been guessed correctly, $G_{3,b}$ outputs 0. Therefore

$$\Pr[G_{2,b} \Rightarrow 1] = n_s^2 \cdot \Pr[G_{3,b} \Rightarrow 1]$$

from which we get

$$\begin{aligned} &|\Pr[\text{IND-StAA-PQC}_1(\Pi, A) \Rightarrow 1 \wedge \text{matches} \neq \emptyset] - \Pr[\text{IND-StAA-PQC}_0(\Pi, A) \Rightarrow 1 \wedge \text{matches} \neq \emptyset]| \\ &\leq n_s^2 \cdot |\Pr[G_{3,1} \Rightarrow 1] - \Pr[G_{3,0} \Rightarrow 1]| + 2 \cdot n_s \cdot \mu(\text{Encaps}_{stat}) . \quad (1) \end{aligned}$$

If the attacker does any action that makes it so that $\text{AttackPQC}(\text{matches})$ returns **true**, he will not be able to distinguish anything. Therefore it is safe to assume that the attacker's actions do not cause $\text{AttackPQC}(\text{matches})$ to return **true**. In the case where $|\text{matches}| = 1$, and we name the matching session $\overline{sID}$, this comes down to:

1. $\text{sesKey}[sID^*] \neq \perp$ (the session actually finished)
2. $\neg \text{revealed}[sID^*]$ (the test session's key was not revealed)
3. $\neg \text{corrupted}[\text{owner}[sID^*]]$ or $\neg \text{revState}[sID^*]$ (either the test session's owner was not corrupted or the test session was not revealed)
4. $\neg \text{revealed}[\overline{sID}]$ (the matching session's key is not revealed)
5. $\neg \text{corrupted}[\text{owner}[\overline{sID}]]$ or $\neg \text{revState}[\overline{sID}]$ (either the matching session's owner was not corrupted or the matching session was not revealed)

Since the responder session has no state, we may assume $\neg \text{revState}[sID_{\text{resp}}]$. And since $\{sID_{\text{init}}, sID_{\text{resp}}\} = \{sID^*, \overline{sID}\}$, the conjunction of properties 3 and 5 is equivalent to $\neg \text{corrupted}[\text{owner}[sID_{\text{init}}]] \vee \neg \text{revState}[sID_{\text{init}}]$. Knowing this, we have to prove the security of the protocol in the following case: $\text{sesKey}[sID^*] \neq \perp$, $\neg \text{revealed}[sID^*]$, $\neg \text{revealed}[\overline{sID}]$, and either $\neg \text{corrupted}[sID_{\text{init}}]$ or $\neg \text{revState}[sID_{\text{init}}]$ where $sID_{\text{init}} \in \{sID^*, \overline{sID}\}$ such that $\text{role}[sID_{\text{init}}] = \text{INITIATOR}$.

We now look at two possible cases:

- Case 1a: where we assume that the state of the initiator has not been revealed.

- Case **1b**: where we assume that the initiator has not been corrupted.

In Case **1a** we assume that $\neg \text{revState}[sID_{\text{init}}]$, where sID_{init} is the session id of the initiator. We denote this case with $\neg \text{st}$. Since the state of the initiator is not revealed, we can assume that with overwhelming probability, the adversary does not know the secret key sk_e and therefore also not k_e . Since k_e is always part of the input of the KDF and since the KDF is modelled as a random oracle, the resulting session key is secure.

In Case **1b** we assume that $\neg \text{corrupted}[sID_{\text{init}}]$, where again sID_{init} is the session id of the initiator. We denote this case with $\neg sk(i)$. Since the long-term secret key of the initiator is not known to the attacker, we can assume that with overwhelming probability they do not know k_a . And again, since k_a is part of the input of the KDF, the resulting session key is secure.

GAME $G_{3,b}^{\neg \text{st}} - G_{7,b}^{\neg \text{st}}$	
01 $sID_{ctr} := 0$	// session counter
02 $sID^* := 0$	// test session ID
03 $s_{init} \leftarrow_{\$} \{1, \dots, n_s\}$	
04 $s_{resp} \leftarrow_{\$} \{1, \dots, n_s\}$	
05 for $i \in \{1, \dots, n_{pt}\}$	
06 $(pk_i, sk_i) \leftarrow \text{KeyGen}_{stat}()$	
07 $\text{QkdInit}()$	
08 $(\widetilde{pk}_e, \widetilde{sk}_e) \leftarrow \text{KeyGen}_{eph}()$	// $G_{5,b}^{\neg \text{st}}$
09 $(\widetilde{c}_e, \widetilde{k}_e) \leftarrow \text{Encaps}_{eph}(\widetilde{pk}_e)$	// $G_{5,b}^{\neg \text{st}}$
10 $\widetilde{k}_e \leftarrow_{\$} \{0, 1\}^{\ell_{eph}}$	// $G_{6,b}^{\neg \text{st}}$
11 $k_{pqc} \leftarrow_{\$} \{0, 1\}^{\ell_{pqc}}$	// $G_{7,b}^{\neg \text{st}}$
12 $b' \leftarrow A^{\circ}(pk_1, \dots, pk_N)$	
13 $matches := \text{FindMatches}(sID^*)$	
14 if $ matches \neq 1$: return 0	
15 if $\text{AttackPQC}(matches)$	
16 return 0	
17 if $\text{revState}[sID^*]$: return 0	// $G_{4,b}^{\neg \text{st}}$
18 Pick $sID_{init}, sID_{resp} \in \{sID^*, \overline{sID}\}$ s.t. $\text{role}[sID_{init}] = \text{INITIATOR}$ and $\text{role}[sID_{resp}] = \text{RESPONDER}$	
19 if $sID_{init} \neq s_{init}$ or $sID_{resp} \neq s_{resp}$	
20 return 0	
21 return b'	

Fig. 11: $G_{3,b}^{\neg \text{st}} - G_{7,b}^{\neg \text{st}}$. Any oracle not included is unchanged.

Case 1a: State not revealed Games $G_{3,b}^{\neg \text{st}} - G_{7,b}^{\neg \text{st}}$ are defined in Fig. 11. Games $G_{3,b}$ and $G_{3,b}^{\neg \text{st}}$ are the same for both b , so

$$|\Pr[G_{3,1} \Rightarrow 1 \wedge \neg \text{st}] - \Pr[G_{3,0} \Rightarrow 1 \wedge \neg \text{st}]| = |\Pr[G_{3,1}^{\neg \text{st}} \Rightarrow 1 \wedge \neg \text{st}] - \Pr[G_{3,0}^{\neg \text{st}} \Rightarrow 1 \wedge \neg \text{st}]|.$$

Games $G_{4,b}^{\neg \text{st}}$ We add the condition $\neg \text{st}$ in game $G_{4,b}^{\neg \text{st}}$ at Line 17, to ensure we are in the correct case. Since

$$\Pr[G_{3,b}^{\neg \text{st}} \Rightarrow 1 \wedge \neg \text{st}] = \Pr[G_{4,b}^{\neg \text{st}} \Rightarrow 1]$$

for both b , which gives

$$|\Pr[G_{3,1}^{\neg \text{st}} \Rightarrow 1 \wedge \neg \text{st}] - \Pr[G_{3,0}^{\neg \text{st}} \Rightarrow 1 \wedge \neg \text{st}]| = |\Pr[G_{4,1}^{\neg \text{st}} \Rightarrow 1] - \Pr[G_{4,0}^{\neg \text{st}} \Rightarrow 1]|.$$

SEND-INIT (sID)	
22 $(i, j, \rho, s) := (\text{owner}[sID], \text{peer}[sID], \text{role}[sID], \text{state}[sID])$	
23 if $i = \perp$ or $\rho \neq \text{INITIATOR}$ or $s \neq \perp$: abort	
24 $(c_b, k_b) \leftarrow \text{Encaps}_{\text{stat}}(pk_j)$	
25 $(pk_e, sk_e) \leftarrow \text{KeyGen}_{\text{eph}}()$	
26 if $sID = s_{\text{init}}$	
27 $(pk_e, sk_e) := (\widetilde{pk_e}, \widetilde{sk_e})$	$\parallel G_{5,b}^{\text{st}}$
28 $m_1 := (c_b, pk_e)$	
29 $\text{sent}[sID] := m_1$	$\parallel$ record outgoing message
30 $\text{state}[sID] := (k_b, sk_e, m_1)$	$\parallel$ update internal state
31 return m_1	
SEND-M1 (sID, m_1)	
32 $(i, j, \rho, s) := (\text{owner}[sID], \text{peer}[sID], \text{role}[sID], \text{state}[sID])$	
33 if $i = \perp$ or $\rho \neq \text{RESPONDER}$ or $s \neq \perp$: abort	
34 $\text{received}[sID] := m_1$	$\parallel$ record incoming message
35 $(c_b, pk_e) := m_1$	$\parallel$ or abort
36 $k'_b := \text{Decaps}_{\text{stat}}(sk_i, c_b)$	
37 $(c_a, k_a) \leftarrow \text{Encaps}_{\text{stat}}(pk_j)$	
38 $(c_e, k_e) \leftarrow \text{Encaps}_{\text{eph}}(pk_e)$	
39 if $sID = s_{\text{resp}}$	
40 $(c_e, k_e) := (\widetilde{c_e}, \widetilde{k_e})$	$\parallel G_{5,b}^{\text{st}}$
41 $k_{pqc} := \text{KDF}(k'_b, k_a, k_e)$	
42 if $sID = s_{\text{resp}}$	
43 $k_{pqc} := k_{pqc}$	$\parallel G_{7,b}^{\text{st}}$
44 $(kID, k_{qkd}) := \text{QKD-GET-KEY}(sID, \ell_{\text{qkd}})$	
45 $t := (m_1, (c_a, c_e, kID))$	
46 $k_{qkd, m} \parallel k_{qkd, s} := k_{qkd}$	
47 $k_{pqc, m} \parallel k_{pqc, s} := k_{pqc}$	
48 $k_{\text{sess}} := k_{qkd, s} \oplus k_{pqc, s}$	
49 $\tau_1 := \text{MAC}_{k_{qkd, m}}^{(qkd)}((t, j, i))$	
50 $\tau_2 := \text{MAC}_{k_{pqc, m}}^{(pqc)}((t, \tau_1, j, i))$	
51 $m_2 = (c_a, c_e, kID, \tau_1, \tau_2)$	
52 $\text{sent}[sID] := m_2$	$\parallel$ record outgoing message
53 $\text{sesKey}[sID] := k_{\text{sess}}$	$\parallel$ set session key
54 $\text{state}[sID] := \text{ACCEPT}$	$\parallel$ accept session
55 return m_2	

Fig. 11: (cont.) $G_{3,b}^{\text{st}}$ - $G_{7,b}^{\text{st}}$. Any oracle not included is unchanged.

```

SEND-M2( $sID, m_2$ )
56  $(i, j, \rho, s) := (\text{owner}[sID], \text{peer}[sID], \text{role}[sID], \text{state}[sID])$ 
57 if  $i = \perp$  or  $\rho \neq \text{INITIATOR}$  : abort                                // session not established
58  $\text{received}[sID] := m_2$                                                 // record incoming message
59  $(c_a, c_e, kID, \tau'_1, \tau'_2) := m_2$                                 // or abort
60  $(k_b, sk_e, m_1) := s$                                                 // or abort
61  $k'_a := \text{Decaps}_{stat}(sk_i, c_a)$ 
62  $k_e := \text{Decaps}_{eph}(sk_e, c_e)$ 
63 if  $sID = s_{init}$ 
64    $k_e := \tilde{k}_e$                                                         //  $G_{5,b}^{-st}$ 
65  $k_{pqc} := \text{KDF}(k_b, k'_a, k'_e)$ 
66 if  $sID = s_{init}$ 
67    $k_{pqc} := k_{pqc}$                                                     //  $G_{7,b}^{-st}$ 
68  $k_{qkd} := \text{QKD-GET-KEY-WITH-ID}(sID, kID)$ 
69 if  $k_{qkd} = \perp$  : abort
70  $t := (m_1, (c_a, c_e, kID))$ 
71  $k_{qkd,m} \parallel k_{qkd,s} := k_{qkd}$ 
72  $k_{pqc,m} \parallel k_{pqc,s} := k_{pqc}$ 
73  $k_{sess} := k_{qkd,s} \oplus k_{pqc,s}$ 
74  $\tau_1 := \text{MAC}_{k_{qkd,m}}^{(qkd)}((t, i, j))$ 
75  $\tau_2 := \text{MAC}_{k_{pqc,m}}^{(pqc)}((t, \tau_1, i, j))$ 
76 if  $\tau_1 \neq \tau'_1$  or  $\tau_2 \neq \tau'_2$  : abort
77  $\text{sesKey}[sID] := k_{sess}$ 
78  $\text{state}[sID] := \text{ACCEPT}$                                             // clear revealable state

```

Fig. 11: (cont.) $G_{3,b}^{-st}$ - $G_{7,b}^{-st}$. Any oracle not included is unchanged.

Games $G_{5,b}^{\neg st}$ For $G_{5,b}^{\neg st}$ we move the calls to KeyGen_{eph} and Encaps_{eph} of the ephemeral KEM to the game itself, Lines 08 and 09. These values then replace the ephemeral public key, secret key, key and ciphertext in Lines 27, 40 and 64. This change is only noticed by the distinguisher if Decaps_{eph} returns the wrong key. Since the probability that this happens is at most δ_{eph} (see Definition 2.1), we have

$$|\Pr[G_{4,b}^{\neg st} \Rightarrow 1] - \Pr[G_{5,b}^{\neg st} \Rightarrow 1]| \leq \delta_{eph}. \quad (2)$$

Games $G_{6,b}^{\neg st}$ In this game, we replace the ephemeral key of the test sessions generated by Encaps_{eph} with a uniformly random key, in Line 10. To justify replacing the ephemeral key k_e with a uniformly random key, we now relate the potential additional advantage to IND-CPA security of the ephemeral KEM. We do this by showing that we can use any adversary C that distinguishes between $G_{5,b}^{\neg st}$ and $G_{6,b}^{\neg st}$ to create an adversary B_1 of the ephemeral KEM that has a runtime about that of C . The adversary C gets access to either $G_{5,b}^{\neg st}$ or $G_{6,b}^{\neg st}$ and then has to output either 0 or 1. In the IND-CPA game, the input to B_1 consists of pk , c and k_i , where pk and c are generated by the KeyGen_{eph} and Encaps_{eph} respectively, and k_i is generated by Encaps_{eph} if $i = 0$, and uniformly random if $i = 1$. This attacker B_1 can use the adversary C as follows: replace the $\widetilde{pk}_e$, $\widetilde{c}_e$ and $\widetilde{k}_e$ after Line 10 with the inputs of B_1 . Since these values were generated with the same algorithms, this does not change their distribution. Now C , with access to these algorithms, outputs 0 or 1. Let B_1 then give the same output as C .

The difference between $G_{5,b}^{\neg st}$ and $G_{6,b}^{\neg st}$ is whether $\widetilde{k}_e$ is generated by Encaps_{eph} or uniformly random. Since C distinguishes between the honestly generated key and a uniformly random key, replacing this key with the key k_i for which we have the same question allows distinguishing an honestly generated key from a uniformly random key for the IND-CPA game. Since this is true for all adversaries C , we have

$$|\Pr[G_{5,b}^{\neg st} \Rightarrow 1] - \Pr[G_{6,b}^{\neg st} \Rightarrow 1]| \leq \text{Adv}_{\text{KEM}_{eph}}^{\text{IND-CPA}}(B_1). \quad (3)$$

Games $G_{7,b}^{\neg st}$ Here, we replace the output of the KDF of the test session with a uniformly random value. The value is generated at Line 11 and used at Lines 43 and 67.

Since we model $\text{KDF} : \{0,1\}^{2\ell_{\text{stat}}+\ell_{\text{eph}}} \rightarrow \{0,1\}^{\ell_{\text{pqc}}}$ as a random oracle, we can use the one-way to hiding Lemma 2.5 to bound the distance between $G_{6,b}^{\neg st}$ and $G_{7,b}^{\neg st}$. Define $H(\cdot) = \text{KDF}(k_a, k_b, \cdot)$ where k_a and k_b are the keys used in session s_{init} . Then for any fixed k_a, k_b we can replace sampling $\text{KDF} \leftarrow_{\$} (\{0,1\}^{2\ell_{\text{stat}}+\ell_{\text{eph}}} \rightarrow \{0,1\}^{\ell_{\text{pqc}}})$ with sampling $H \leftarrow_{\$} (\{0,1\}^{\ell_{\text{eph}}} \rightarrow \{0,1\}^{\ell_{\text{pqc}}})$. Let $B^H(x, y)$ be the execution of $G_{6,b}^{\neg st}$ where we replace k_e of the test session with x , and k_{pqc} of the test session with y . Then

$$\Pr[G_{6,b}^{\neg st} \Rightarrow 1] = \Pr[b' = 1 : b' \leftarrow B^H(x, H(x))]$$

and

$$\Pr[G_{7,b}^{\neg st} \Rightarrow 1] = \Pr[b' = 1 : b' \leftarrow B^H(x, y), y \leftarrow_{\$} \{0,1\}^{\ell_{\text{pqc}}}]$$

where we also take the average over $H \leftarrow_{\$} (\{0,1\}^{\ell_{\text{eph}}} \rightarrow \{0,1\}^{\ell_{\text{pqc}}})$ and $x \leftarrow_{\$} \{0,1\}^{\ell_{\text{eph}}}$. As in Lemma 2.5, let C be an oracle algorithm that on input x does the following: pick $i \leftarrow_{\$} \{1, \dots, q_h\}$ and $y \leftarrow_{\$} \{0,1\}^m$, run $B^H(x, y)$ until just before the i -th query, measure the argument of the query in the computational basis, and output the measurement outcome. We define

$$P_C = \Pr[x = x' : x' \leftarrow C^H(x)]$$

where the probability is also over $H \leftarrow_{\$} (\{0,1\}^{\ell_{\text{eph}}} \rightarrow \{0,1\}^{\ell_{\text{pqc}}})$ and $x \leftarrow_{\$} \{0,1\}^{\ell_{\text{eph}}}$. But since x is not available to C at any point, the best the attacker can do is guess, which means that $P_C = \frac{1}{2^{\ell_{\text{eph}}}}$.

Then by Lemma 2.5 we have

$$|\Pr[G_{6,b}^{\neg st} \Rightarrow 1] - \Pr[G_{7,b}^{\neg st} \Rightarrow 1]| \leq 2q_H \sqrt{P_C} = 2q_H \frac{1}{\sqrt{2^{\ell_{\text{eph}}}}}. \quad (4)$$

Next, we want to bound

$$|\Pr[G_{7,0}^{\neg \text{st}} \Rightarrow 1] - \Pr[G_{7,1}^{\neg \text{st}} \Rightarrow 1]|.$$

Key k_0^* is generated as a part $k_{pqc} \oplus k_{qkd}$, where k_{pqc} is uniformly random. Furthermore, this key is only used in the matching session of the test session, which the adversary is not allowed to reveal. Key k_1^* is generated uniformly random. Therefore the adversary cannot distinguish between $G_{7,0}^{\neg \text{st}}$ and $G_{7,1}^{\neg \text{st}}$, and we have

$$\Pr[G_{7,0}^{\neg \text{st}} \Rightarrow 1] = \Pr[G_{7,1}^{\neg \text{st}} \Rightarrow 1]. \quad (5)$$

Putting equations (2), (3) and (4) together, we get that

$$|\Pr[G_{4,b}^{\neg \text{st}} \Rightarrow 1] - \Pr[G_{7,b}^{\neg \text{st}} \Rightarrow 1]| \leq \delta_{eph} + \text{Adv}_{\text{KEM}_{eph}}^{\text{IND-CPA}}(B_1) + 2q_H \frac{1}{\sqrt{2^{\ell_{eph}}}}.$$

Combining this with (5), we get that

$$|\Pr[G_{3,0}^{\neg \text{st}} \wedge \neg st \Rightarrow 1] - \Pr[G_{3,1}^{\neg \text{st}} \wedge \neg st \Rightarrow 1]| \leq 2\delta_{eph} + 2\text{Adv}_{\text{KEM}_{eph}}^{\text{IND-CPA}}(B_1) + 4q_H \frac{1}{\sqrt{2^{\ell_{eph}}}}. \quad (6)$$

GAME $G_{3,b}^{\neg sk(i)} - G_{7,b}^{\neg sk(i)}$	
01 $sID_{ctr} := 0$	// session counter
02 $sID^* := 0$	// test session ID
03 $s_{init} \leftarrow_{\$} \{1, \dots, n_s\}$	
04 $s_{resp} \leftarrow_{\$} \{1, \dots, n_s\}$	
05 $j' \leftarrow_{\$} \{1, \dots, n_{pt}\}$	// $G_{5,b}^{\neg sk(i)}$
06 for $i \in \{1, \dots, n_{pt}\}$	
07 $(pk_i, sk_i) \leftarrow \text{KeyGen}_{stat}()$	
08 $(pk_{j'}, sk_{j'}) \leftarrow \Pi.\text{KeyGen}_{stat}()$	// $G_{6,b}^{\neg sk(i)}$
09 $(\tilde{c}_a, \tilde{k}_a) \leftarrow \text{Encaps}_{stat}(\tilde{pk}_{j'})$	// $G_{6,b}^{\neg sk(i)}$
10 $\tilde{k}_a \leftarrow_{\$} \{0, 1\}^{\ell_{stat}}$	// $G_{7,b}^{\neg sk(i)}$
11 $\tilde{k}_{pqc} \leftarrow_{\$} \{0, 1\}^{\ell_{pqc}}$	// $G_{8,b}^{\neg sk(i)}$
12 $\text{QkdInit}()$	
13 $b' \leftarrow A^{\circ}(pk_1, \dots, pk_{n_{pt}})$	
14 $matches := \text{FindMatches}(sID^*)$	
15 if $ matches \neq 1$: return 0	
16 if $\text{AttackPQC}(matches)$	
17 return 0	
18 Pick $sID_{init}, sID_{resp} \in \{sID^*, sID\}$ s.t. $\text{role}[sID_{init}] = \text{INITIATOR}$ and $\text{role}[sID_{resp}] = \text{RESPONDER}$	
19 if $sID_{init} \neq s_{init}$ or $sID_{resp} \neq s_{resp}$	
20 return 0	
21 if $j' \neq \text{owner}[sID_{init}]$	
22 return 0	// $G_{5,b}^{\neg sk(i)}$
23 if $\text{corrupted}[sID_{init}]$: return 0	// $G_{4,b}^{\neg sk(i)}$
24 return b'	

Fig. 12: $G_{3,b}^{\neg sk(i)} - G_{8,b}^{\neg sk(i)}$. Any oracle not included is unchanged.

```

SEND-INIT( $sID$ )
25  $(i, j, \rho, s) := (\text{owner}[sID], \text{peer}[sID], \text{role}[sID], \text{state}[sID])$ 
26 if  $i = \perp$  or  $\rho \neq \text{INITIATOR}$  or  $s \neq \perp$  : abort
27  $(c_b, k_b) \leftarrow \text{Encaps}_{stat}(pk_j)$ 
28  $(pk_e, sk_e) \leftarrow \text{KeyGen}_{eph}()$ 
29  $m_1 := (c_b, pk_e)$ 
30  $\text{sent}[sID] := m_1$  // record outgoing message
31  $\text{state}[sID] := (k_b, sk_e, m_1)$  // set state
32 return  $m_1$ 

SEND-M1( $sID, m_1$ )
33  $(i, j, \rho, s) := (\text{owner}[sID], \text{peer}[sID], \text{role}[sID], \text{state}[sID])$ 
34 if  $i = \perp$  or  $\rho \neq \text{RESPONDER}$  or  $s \neq \perp$  : abort
35  $\text{received}[sID] := m_1$  // record incoming message
36  $(c_b, pk_e) := m_1$  // or abort
37  $k'_b := \text{Decaps}_{stat}(sk_i, c_b)$ 
38  $(c_a, k_a) \leftarrow \text{Encaps}_{stat}(pk_j)$ 
39 if  $sID = s_{resp}$ 
40  $(c_a, k_a) := (\tilde{c}_a, \tilde{k}_a)$  //  $G_{6,b}^{-sk(i)}$ 
41  $(c_e, k_e) \leftarrow \text{Encaps}_{eph}(pk_e)$ 
42  $k_{pqc} := \text{KDF}(k'_b, k_a, k_e)$ 
43 if  $sID = s_{resp}$ 
44  $k_{pqc} := k_{pqc}$  //  $G_{8,b}^{-sk(i)}$ 
45  $(kID, k_{qkd}) := \text{QKD-GET-KEY}(sID, \ell_{qkd})$ 
46  $t := (m_1, (c_a, c_e, kID))$ 
47  $k_{qkd, m} \parallel k_{qkd, s} := k_{qkd}$ 
48  $k_{pqc, m} \parallel k_{pqc, s} := k_{pqc}$ 
49  $k_{sess} := k_{qkd, s} \oplus k_{pqc, s}$ 
50  $\tau_1 := \text{MAC}_{k_{qkd, m}}^{(qkd)}((t, j, i))$ 
51  $\tau_2 := \text{MAC}_{k_{pqc, m}}^{(pqc)}((t, \tau_1, j, i))$ 
52  $m_2 = (c_a, c_e, kID, \tau_1, \tau_2)$ 
53  $\text{sent}[sID] := m_2$  // record outgoing message
54  $\text{sesKey}[sID] := k_{sess}$ 
55  $\text{state}[sID] := \text{ACCEPT}$ 
56 return  $m_2$ 

```

Fig. 12: (cont.) $G_{3,b}^{-sk(i)}$ - $G_{7,b}^{-sk(i)}$. Any oracle not included is unchanged.


```

SEND-M2( $sID, m_2$ )
57  $(i, j, \rho, s) := (\text{owner}[sID], \text{peer}[sID], \text{role}[sID], \text{state}[sID])$ 
58 if  $i = \perp$  or  $\rho \neq \text{INITIATOR}$  : abort                                // session not established
59  $\text{received}[sID] := m_2$                                                 // record incoming message
60  $(c_a, c_e, kID, \tau'_1, \tau'_2) := m_2$                                 // or abort
61  $(k_b, sk_e, m_1) := s$                                                 // or abort
62  $k'_a := \text{Decaps}_{\text{stat}}(sk_i, c_a)$ 
63 if  $sID = s_{\text{init}}$  and  $c_a = \tilde{c}_a$ 
64    $k'_a := k_a$                                                         //  $G_{6,b}^{\neg sk(i)}$ 
65 if  $sID = s_{\text{init}}$  and  $c_a \neq \tilde{c}_a$ 
66    $k'_a := \text{Decaps}_{\text{stat}}(sk_i, c_a)$                                 //  $G_{6,b}^{\neg sk(i)}$ 
67  $k_e := \text{Decaps}_{\text{eph}}(sk_e, c_e)$ 
68  $k_{pqc} := \text{KDF}(k_b, k'_a, k'_e)$ 
69 if  $sID = s_{\text{init}}$ 
70    $k_{pqc} := k_{pqc}$                                                   //  $G_{8,b}^{\neg sk(i)}$ 
71  $k_{qkd} := \text{QKD-GET-KEY-WITH-ID}(sID, kID)$ 
72 if  $k_{qkd} = \perp$  : abort
73  $t := (m_1, (c_a, c_e, kID))$ 
74  $k_{qkd, m} \parallel k_{qkd, s} := k_{qkd}$ 
75  $k_{pqc, m} \parallel k_{pqc, s} := k_{pqc}$ 
76  $k_{\text{sess}} := k_{qkd, s} \oplus k_{pqc, s}$ 
77  $\tau_1 := \text{MAC}_{k_{qkd, m}}^{(qkd)}((t, i, j))$ 
78  $\tau_2 := \text{MAC}_{k_{pqc, m}}^{(pqc)}((t, \tau_1, i, j))$ 
79 if  $\tau_1 \neq \tau'_1$  or  $\tau_2 \neq \tau'_2$  : abort
80  $\text{sesKey}[sID] := k_{\text{sess}}$ 
81  $\text{state}[sID] := \text{ACCEPT}$                                             // clear revealable state

```

Fig. 12: (cont.) $G_{3,b}^{\neg sk(i)}$ - $G_{8,b}^{\neg sk(i)}$. Any oracle not included is unchanged.

Case 1b: Initiator not corrupted Games $G_{3,b}^{\neg sk(i)}$ - $G_{8,b}^{\neg sk(i)}$ are defined in Fig. 12. Games $G_{3,b}$ and $G_{3,b}^{\neg sk(i)}$ are the same for both b , so

$$\begin{aligned} & |\Pr[G_{3,1} \Rightarrow 1 \wedge \neg sk(i)] - \Pr[G_{3,0} \Rightarrow 1 \wedge \neg sk(i)]| \\ &= \left| \Pr[G_{3,1}^{\neg sk(i)} \Rightarrow 1 \wedge \neg sk(i)] - \Pr[G_{3,0}^{\neg sk(i)} \Rightarrow 1 \wedge \neg sk(i)] \right|. \end{aligned}$$

Games $G_{4,b}^{\neg sk(i)}$ We add the condition $\neg sk(i)$ in game $G_{4,b}^{\neg sk(i)}$ at Line 23, to ensure we are in the correct case. Since

$$\Pr[G_{3,b}^{\neg sk(i)} \Rightarrow 1 \wedge \neg sk(i)] = \Pr[G_{4,b}^{\neg sk(i)} \Rightarrow 1]$$

for both b , which gives

$$\left| \Pr[G_{3,1}^{\neg sk(i)} \Rightarrow 1 \wedge \neg sk(i)] - \Pr[G_{3,0}^{\neg sk(i)} \Rightarrow 1 \wedge \neg sk(i)] \right| = \left| \Pr[G_{4,1}^{\neg sk(i)} \Rightarrow 1] - \Pr[G_{4,0}^{\neg sk(i)} \Rightarrow 1] \right|. \quad (7)$$

Games $G_{5,b}^{\neg sk(i)}$ For $G_{5,b}^{\neg sk(i)}$ we guess which party is the owner of SID_{init} . In line 05 we add the guess for the owner of SID_{init} , and in line 21 we check if it is correct. If the party is guessed incorrectly, the game returns 0. If the party is guessed correctly, the game returns 1 if and only if $G_{4,b}^{\neg sk(i)}$ would have returned 1. Since the party is guessed uniformly at random, we get

$$\Pr[G_{4,b}^{\neg sk(i)} \Rightarrow 1] = n_{\text{pt}} \cdot \Pr[G_{5,b}^{\neg sk(i)} \Rightarrow 1]. \quad (8)$$

Games $G_{6,b}^{\neg sk(i)}$ For $G_{6,b}^{\neg sk(i)}$ we move the call $\text{Encaps}_{\text{stat}}$ of the static KEM_{stat} to the **EST** oracle, Line 08. These values then replace the static key and ciphertext in Lines 40 and 64. This change is only noticed by the distinguisher if $\text{Decaps}_{\text{stat}}$ returns the wrong key. Since the probability of returning the wrong key is at most δ_{stat} (see Definition 2.1), we have

$$\left| \Pr[G_{5,b}^{\neg sk(i)} \Rightarrow 1] - \Pr[G_{6,b}^{\neg sk(i)} \Rightarrow 1] \right| \leq \delta_{\text{stat}}. \quad (9)$$

Games $G_{7,b}^{\neg sk(i)}$ In this game, we replace Bob's static key of the test sessions generated by $\text{Encaps}_{\text{stat}}$ with a uniformly random key, in Line 10. To justify replacing the static key k_b with a uniformly random key, we now relate the potential additional advantage to IND-CCA security of the static KEM. We do this by showing that we can use any adversary C that distinguishes between $G_{5,b}^{\neg sk(i)}$ and $G_{6,b}^{\neg sk(i)}$ to create an adversary B_2 of the static KEM that has a runtime about that of C . The adversary C gets access to either $G_{5,b}^{\neg sk(i)}$ or $G_{6,b}^{\neg sk(i)}$ and then has to output either 0 or 1. In the IND-CCA game, the input to B_2 consists of pk , c and k_i , where pk and c are generated by the $\text{KeyGen}_{\text{stat}}$ and $\text{Encaps}_{\text{stat}}$ respectively, and k_i is generated by $\text{Encaps}_{\text{stat}}$ if $i = 0$, and uniformly random if $i = 1$. This attacker B_2 can use the distinguisher C as follows: replace the $\widetilde{pk}_b$, $\widetilde{c}_b$ and $\widetilde{k}_b$ after Line 10 with the inputs of B_2 . Since these values were generated with the same algorithms, this does not change their distribution. Furthermore, any call to $\text{Decaps}_{\text{stat}}$ using key $\widetilde{k}_b$ gets replaced by a call to the Decaps oracle of the IND-CCA game. Now C , with access to these algorithms, outputs 0 or 1. Let B_2 then give the same output as C .

The difference between $G_{6,b}^{\neg sk(i)}$ and $G_{7,b}^{\neg sk(i)}$ is whether $\widetilde{k}_b$ is generated by $\text{Encaps}_{\text{stat}}$ or uniformly random. Since C distinguishes between the honestly generated key and a uniformly random key, replacing this key with the key k_i for which we have the same question allows distinguishing an honestly generated key from a uniformly random key for the IND-CCA game. Since this is true for all adversaries C , we have

$$\left| \Pr[G_{6,b}^{\neg sk(i)} \Rightarrow 1] - \Pr[G_{7,b}^{\neg sk(i)} \Rightarrow 1] \right| \leq \text{Adv}_{\text{KEM}_{\text{stat}}}^{\text{IND-CCA}}(B_2). \quad (10)$$

Games $G_{8,b}^{\neg sk(i)}$ Here, we replace the output of the KDF of the test session with a uniformly random value. The value is generated at Line 11 and used at Lines 44 and 70.

Since we model $\text{KDF} : \{0,1\}^{2\ell_{\text{stat}}+\ell_{\text{eph}}} \rightarrow \{0,1\}^{\ell_{\text{pqc}}}$ as a random oracle, we can use one-way to hiding (Lemma 2.5) to bound the distance between $G_{7,b}^{\neg sk(i)}$ and $G_{8,b}^{\neg sk(i)}$. Define $H(\cdot) = \text{KDF}(k_a, \cdot, k_e)$ where k_a and k_e are the keys used in session s_{init} . Then for any fixed k_a, k_e we can replace sampling $\text{KDF} \leftarrow_{\$} (\{0,1\}^{2\ell_{\text{stat}}+\ell_{\text{eph}}} \rightarrow \{0,1\}^{\ell_{\text{pqc}}})$ with sampling $H \leftarrow_{\$} (\{0,1\}^{\ell_{\text{stat}}} \rightarrow \{0,1\}^{\ell_{\text{pqc}}})$. Let $B^H(x, y)$ be the execution of $G_{7,b}^{\neg sk(i)}$ where we replace k_b of the test session with x , and k_{pqc} of the test session with y . Then

$$\Pr[G_{7,b}^{\neg sk(i)} \Rightarrow 1] = \Pr[b' = 1 : b' \leftarrow B^H(x, H(x))]$$

and

$$\Pr[G_{8,b}^{\neg sk(i)} \Rightarrow 1] = \Pr[b' = 1 : y \leftarrow_{\$} \{0,1\}^{\ell_{\text{pqc}}}, b' \leftarrow B^H(x, y)]$$

where we also take the probability over $H \leftarrow_{\$} (\{0,1\}^{\ell_{\text{stat}}} \rightarrow \{0,1\}^{\ell_{\text{pqc}}})$ and $x \leftarrow_{\$} \{0,1\}^{\ell_{\text{stat}}}$. As in Lemma 2.5, let C be an oracle algorithm that on input x does the following: pick $i \leftarrow_{\$} \{1, \dots, q_h\}$ and $y \leftarrow_{\$} \{0,1\}^m$, run $B^H(x, y)$ until just before the i -th query, measure the argument of the query in the computational basis, and output the measurement outcome. We define

$$P_C = \Pr[x = x' : x' \leftarrow C^H(x)]$$

where we also take the probability over $H \leftarrow_{\$} (\{0,1\}^{\ell_{\text{stat}}} \rightarrow \{0,1\}^{\ell_{\text{pqc}}})$ and $x \leftarrow_{\$} \{0,1\}^{\ell_{\text{stat}}}$. But since x is not available to C at any point, the best the attacker can do is guess, which means that $P_C = \frac{1}{2^{\ell_{\text{stat}}}}$.

Then by Lemma 2.5 we have

$$\left| \Pr[G_{7,b}^{\neg sk(i)} \Rightarrow 1] - \Pr[G_{8,b}^{\neg sk(i)} \Rightarrow 1] \right| \leq 2q_H \sqrt{P_C} = 2q_H \frac{1}{\sqrt{2^{\ell_{\text{stat}}}}}. \quad (11)$$

Next, we want to bound

$$\left| \Pr[G_{8,0}^{\neg sk(i)} \Rightarrow 1] - \Pr[G_{8,1}^{\neg sk(i)} \Rightarrow 1] \right|.$$

Key k_0^* is generated as a part $k_{\text{pqc}} \oplus k_{\text{qkd}}$, where k_{pqc} is uniformly random. Furthermore, this key is only used in the matching session of the test session, which the adversary is not allowed to reveal. Key k_1^* is generated uniformly random. Therefore the adversary cannot distinguish between $G_{8,0}^{\neg sk(i)}$ and $G_{8,1}^{\neg sk(i)}$, and we have

$$\Pr[G_{8,0}^{\neg sk(i)} \Rightarrow 1] = \Pr[G_{8,1}^{\neg sk(i)} \Rightarrow 1]. \quad (12)$$

Combining equations (9), (10) and (11), we get that

$$\left| \Pr[G_{5,b}^{\neg sk(i)} \Rightarrow 1] - \Pr[G_{8,b}^{\neg sk(i)} \Rightarrow 1] \right| \leq \delta_{\text{stat}} + \text{Adv}_{\text{KEM}_{\text{stat}}}^{\text{IND-CCA}}(B_2) + 2q_H \frac{1}{\sqrt{2^{\ell_{\text{stat}}}}}.$$

Combining this with (12), we get that

$$\left| \Pr[G_{5,0}^{\neg sk(i)} \Rightarrow 1] - \Pr[G_{5,1}^{\neg sk(i)} \Rightarrow 1] \right| \leq 2\delta_{\text{stat}} + 2\text{Adv}_{\text{KEM}_{\text{stat}}}^{\text{IND-CCA}}(B_2) + 4q_H \frac{1}{\sqrt{2^{\ell_{\text{stat}}}}}$$

which, when combined with equations (7) and (8), gives

$$\begin{aligned} \left| \Pr[G_{3,0}^{\neg sk(i)} \wedge \neg sk(i) \Rightarrow 1] - \Pr[G_{3,1}^{\neg sk(i)} \wedge \neg sk(i) \Rightarrow 1] \right| \\ \leq 2n_{\text{pt}} \left(\delta_{\text{stat}} + \text{Adv}_{\text{KEM}_{\text{stat}}}^{\text{IND-CCA}}(B_2) + 2q_H \frac{1}{\sqrt{2^{\ell_{\text{stat}}}}} \right) \end{aligned} \quad (13)$$

Combining Case 1a and Case 1b Combining Eqs. (6) and (13) gives

$$\begin{aligned} & \left| \Pr[G_{3,0}^{\neg sk(i)} \Rightarrow 1] - \Pr[G_{3,1}^{\neg sk(i)} \Rightarrow 1] \right| \\ & \leq 2\delta_{eph} + 2\text{Adv}_{\text{KEM}_{eph}}^{\text{IND-CPA}}(B_1) + 4q_H \frac{1}{\sqrt{2^{\ell_{eph}}}} + 2n_{\text{pt}} \left(\delta_{stat} + \text{Adv}_{\text{KEM}_{stat}}^{\text{IND-CCA}}(B_2) + 2q_H \frac{1}{\sqrt{2^{\ell_{stat}}}} \right). \end{aligned}$$

If we fill this in Eq. (1), we have proven the claim of Lemma 5.3

$$\begin{aligned} & |\Pr[\text{IND-StAA-PQC}_1(\Pi, A) \Rightarrow 1 \wedge \text{matches} \neq \emptyset] - \Pr[\text{IND-StAA-PQC}_0(\Pi, A) \Rightarrow 1 \wedge \text{matches} \neq \emptyset]| \\ & \leq n_s^2 \cdot |\Pr[G_{3,1} \Rightarrow 1] - \Pr[G_{3,0} \Rightarrow 1]| + 2 \cdot n_s \cdot \mu(\text{Encaps}_{stat}) \\ & \leq 2n_s^2 \cdot \left(\delta_{eph} + \text{Adv}_{\text{KEM}_{eph}}^{\text{IND-CPA}}(B_1) + 2q_H \frac{1}{\sqrt{2^{\ell_{eph}}}} + n_{\text{pt}} \left(\delta_{stat} + \text{Adv}_{\text{KEM}_{stat}}^{\text{IND-CCA}}(B_2) + 2q_H \frac{1}{\sqrt{2^{\ell_{stat}}}} \right) \right) \\ & \quad + 2 \cdot n_s \cdot \mu(\text{Encaps}_{stat}) \end{aligned}$$

5.2 Security proof for Case 2: active attack, Lemma 5.4

We consider the case that $\text{matches} = \emptyset$, so we are proving an upper bound for:

$$\begin{aligned} & |\Pr[\text{IND-StAA-PQC}_0(\Pi, A) \Rightarrow 1 \wedge \text{matches} = \emptyset] \\ & \quad - \Pr[\text{IND-StAA-PQC}_1(\Pi, A) \Rightarrow 1 \wedge \text{matches} = \emptyset]|. \end{aligned}$$

Proof. We consider the initiator and responder session separately, but due to the symmetry in the protocol the proofs are very similar. We consider only adversaries for which $\text{AttackPQC}(\text{matches})$ does not hold. We know $\text{matches} = \emptyset$ by assumption of the case, so that by definition of AttackPQC , we have

$$\begin{aligned} & \neg \text{AttackPQC}(\emptyset) \Leftrightarrow \\ & \quad \neg \text{revState}[sID^*] \wedge \text{sesKey}[sID^*] \neq \perp \wedge \neg \text{revealed}[sID^*] \wedge \neg \text{corrupted}[\text{peer}[sID^*]]. \end{aligned}$$

In other words, the encapsulation to the peer's static key leaks neither directly via a state reveal, nor indirectly via corruption of the peer. We prove this by reduction to the IND-CCA security of KEM_{stat} . There may be a partially matching session that computes the same k_{pqc} (we consider that in Case 2a1). Since we are considering active attacks, the full transcript must differ somewhere between these sessions, so (unless a forgery against $\text{MAC}^{(pqc)}$ was made), the initiator will detect this difference and abort. If the test session is a responder, then this means that its value $k_{pqc,s}$ is independent of the adversary view and therefore the session key is indistinguishable from uniformly random. If there does not exist a partially matching session (we consider this in Case 2a2), then $k_{pqc,s}$ looks random for the adversary and therefore so does the session key.

The game hops are given in Fig. 13. First we formalize the assumption of Case 2: we encode $\text{matches} = \emptyset$ in $G_{1,b}^{\neg ms}$, so that

$$\Pr[\text{IND-StAA-PQC}_b(\Pi, A) \Rightarrow 1 \wedge \text{matches} = \emptyset] = \Pr[G_{1,b}^{\neg ms} \Rightarrow 1]. \quad (14)$$

Next, $G_{2,b}^{\neg ms}$ makes a random guess s' for the test session sID^* , and since $\Pr[s' = sID^*] = 1/n_s$ and we return 0 if the guess was wrong:

$$\Pr[G_{1,b}^{\neg ms} \Rightarrow 1] = n_s \Pr[G_{2,b}^{\neg ms} \Rightarrow 1]. \quad (15)$$

GAME $G_{1,b}^{\neg\text{ms}} - G_{12,b}^{\neg\text{ms}}$	
01 $sID_{ctr} := 0$	
02 $sID^* := 0$	
03 $s' \leftarrow_{\$} \{1, \dots, n_s\}$	$\parallel G_{2,b}^{\neg\text{ms}}$
04 $s'' \leftarrow_{\$} \{1, \dots, n_s\} \setminus \{s'\}$	$\parallel G_{9,b}^{\neg\text{ms},i,k}$
05 $j' \leftarrow_{\$} \{1, \dots, n_{pt}\}$	$\parallel G_{3,b}^{\neg\text{ms}}$
06 for $i \in \{1, \dots, n_{pt}\}$	
07 $(pk_i, sk_i) \leftarrow \text{KeyGen}_{stat}()$	
08 QkdInit ()	
09 $(\tilde{c}_b, \tilde{k}_b) := \text{Encaps}_{stat}(pk_{j'})$	$\parallel G_{5,b}^{\neg\text{ms},i}$
10 $\tilde{k}_b \leftarrow_{\$} \{0, 1\}^{\ell_{stat}}$	$\parallel G_{6,b}^{\neg\text{ms},i}$
11 $\tilde{k}_{pqc} \leftarrow_{\$} \{0, 1\}^{\ell_{pqc}}$	$\parallel G_{11,b}^{\neg\text{ms},i,k}$
12 $b' \leftarrow A^{\circ}(pk_1, \dots, pk_{n_{pt}})$	
13 $matches := \text{FindMatches}(sID^*)$	
14 if $matches \neq \emptyset$: return 0	$\parallel G_{1,b}^{\neg\text{ms}}$
15 if $\text{AttackPQC}(matches)$	
16 return 0	
17 if $s' \neq sID^*$: return 0	$\parallel G_{2,b}^{\neg\text{ms}}$
18 if $j' \neq \text{peer}[sID^*]$: return 0	$\parallel G_{3,b}^{\neg\text{ms}}$
19 if $\text{role}[sID^*] \neq \text{INITIATOR}$	
20 return 0	$\parallel G_{4,b}^{\neg\text{ms},i}$
21 $kdfMatch = \text{false}$	
22 for $sID \in [S] \setminus \{sID^*\}$	
23 if $kdfIn[sID] = kdfIn[sID^*]$	
24 $kdfMatch = \text{true}$	
25 if $\neg kdfMatch$: return 0	$\parallel G_{8,b}^{\neg\text{ms},i,k}$
26 if $kdfIn[s''] \neq kdfIn[sID^*]$	
27 return 0	$\parallel G_{9,b}^{\neg\text{ms},i,k}$
28 for $sID \in \{1, \dots, n_s\} \setminus \{s'', sID^*\}$	
29 if $kdfIn[sID] = kdfIn[s'']$	$\parallel G_{10,b}^{\neg\text{ms},i,k}$
30 return 0	
31 return b'	
SEND-INIT (sID)	
32 $(i, j) := (\text{owner}[sID], \text{peer}[sID])$	
33 $(\rho, s) := (\text{role}[sID], \text{state}[sID])$	
34 if $i = \perp$ or $\rho \neq \text{INITIATOR}$ or $s \neq \perp$	
35 abort	
36 $(c_b, k_b) := \text{Encaps}_{stat}(pk_j)$	
37 if $sID = s'$	
38 $(c_b, k_b) := (\tilde{c}_b, \tilde{k}_b)$	$\parallel G_{5,b}^{\neg\text{ms},i}$
39 $(pk_e, sk_e) := \text{KeyGen}_{eph}()$	
40 $m_1 := (c_b, pk_e)$	
41 $\text{sent}[sID] := m_1$	
42 $\text{state}[sID] := (k_b, sk_e, m_1)$	
43 return m_1	
SEND-M1 (sID, m_1)	
44 $(i, j) := (\text{owner}[sID], \text{peer}[sID])$	
45 $(\rho, s) := (\text{role}[sID], \text{state}[sID])$	
46 if $i = \perp$ or $\rho \neq \text{RESPONDER}$ or $s \neq \perp$	
47 abort	
48 $\text{received}[sID] := m_1$	
49 $(c_b, pk_e) := m_1$	
50 $k'_b := \text{Decaps}_{stat}(sk_i, c_b)$	
51 if $i = j'$ and $c_b = \tilde{c}_b$	
52 $k'_b := \tilde{k}_b$	$\parallel G_{5,b}^{\neg\text{ms},i}$
53 $(c_a, k_a) \leftarrow \text{Encaps}_{stat}(pk_j)$	
54 $(c_e, k_e) \leftarrow \text{Encaps}_{eph}(pk_e)$	
55 $kdfIn[sID] = (k'_b, k_a, k_e)$	$\parallel G_{7,b}^{\neg\text{ms},i}$
56 $k_{pqc} := \text{KDF}(k'_b, k_a, k_e)$	
57 if $sID = s''$	
58 $k_{pqc} := \tilde{k}_{pqc}$	$\parallel G_{11,b}^{\neg\text{ms},i,k}$
59 $(kID, k_{qkd}) := \text{QKD-GET-KEY}(sID, \ell_{qkd})$	
60 $t := (m_1, (c_a, c_e, kID))$	
61 $(k_{pqc,m} \parallel k_{pqc,s}) := k_{pqc}$	
62 $(k_{qkd,m} \parallel k_{qkd,s}) := k_{qkd}$	
63 $k_{sess} := k_{pqc,s} \oplus k_{qkd,s}$	
64 $\tau_1 = \text{MAC}_{k_{qkd,m}}^{(qkd)}((t, j, i))$	
65 $\tau_2 = \text{MAC}_{k_{pqc,m}}^{(pqc)}((t, \tau_1, j, i))$	
66 $m_2 := (c_a, c_e, kID, \tau_1, \tau_2)$	
67 $\text{sent}[sID] := m_2$	
68 $\text{sesKey}[sID] := k_{sess}$	
69 $\text{state}[sID] := \text{ACCEPT}$	
70 return m_2	
SEND-M2 (sID, m_2)	
71 $(i, j) := (\text{owner}[sID], \text{peer}[sID])$	
72 $(\rho, s) := (\text{role}[sID], \text{state}[sID])$	
73 if $i = \perp$ or $\rho \neq \text{INITIATOR}$	
74 abort	
75 $\text{received}[sID] := m_2$	
76 $(c_a, c_e, kID, \tau'_1, \tau'_2) := m_2$	
77 $(k_b, sk_e, m_1) := s$	
78 $k'_a := \text{Decaps}_{stat}(sk_i, c_a)$	
79 $k'_e := \text{Decaps}_{eph}(sk_e, c_e)$	
80 if $sID = s'$	
81 $kdfIn[sID] = (k_b, k'_a, k'_e)$	$\parallel G_{7,b}^{\neg\text{ms},i}$
82 $k_{pqc} := \text{KDF}(k_b, k'_a, k'_e)$	
83 if $sID = s'$	
84 $k_{pqc} := \tilde{k}_{pqc}$	$\parallel G_{11,b}^{\neg\text{ms},i,k}$
85 $k_{qkd} :=$	
QKD-GET-KEY-WITH-ID (sID, kID)	
86 if $k_{qkd} = \perp$: abort	
87 $t := (m_1, (c_a, c_e, kID))$	
88 $(k_{pqc,m} \parallel k_{pqc,s}) := k_{pqc}$	
89 $(k_{qkd,m} \parallel k_{qkd,s}) := k_{qkd}$	
90 $k_{sess} := k_{pqc,s} \oplus k_{qkd,s}$	
91 $\tau_1 = \text{MAC}_{k_{qkd,m}}^{(qkd)}((t, i, j))$	
92 $\tau_2 = \text{MAC}_{k_{pqc,m}}^{(pqc)}((t, \tau_1, i, j))$	
93 if $sID = s'$: abort	$\parallel G_{12,b}^{\neg\text{ms},i,k}$
94 if $\tau_1 \neq \tau'_1$ or $\tau_2 \neq \tau'_2$: abort	
95 $\text{sesKey}[sID] := k_{sess}$	
96 $\text{state}[sID] := \text{ACCEPT}$	

Fig. 13: Game $G_{1,b}^{\neg\text{ms}}$: non-matching sessions, initiator session, matching $kdfIn$ exists

$G_{3,b}^{\text{ms}}$ makes a random guess j' for the test session $\text{peer}[sID^*]$, and since $\Pr[j' = \text{peer}[sID^*]] = 1/n_{\text{pt}}$:

$$\Pr[G_{2,b}^{\text{ms}} \Rightarrow 1] = n_{\text{pt}} \Pr[G_{3,b}^{\text{ms}} \Rightarrow 1]. \quad (16)$$

CASE 2A: TEST SESSION IS AN INITIATOR. We first consider the case that $\text{role}[sID^*] = \text{INITIATOR}$, encoded in $G_{4,b}^{\text{ms},i}$:

$$\Pr[G_{3,b}^{\text{ms}} \Rightarrow 1 \wedge \text{role}[sID^*] = \text{INITIATOR}] = \Pr[G_{4,b}^{\text{ms},i} \Rightarrow 1]. \quad (17)$$

In game $G_{5,b}^{\text{ms},i}$, we sample the test-session encapsulation in advance, and we also replace the decapsulated value with the encapsulated one. The games are identical unless a decapsulation failure occurs in $G_{4,b}^{\text{ms},i}$, which by Definition 2.1 happens with probability at most δ_{stat} , so we have

$$|\Pr[G_{4,b}^{\text{ms},i} \Rightarrow 1] - \Pr[G_{5,b}^{\text{ms},i} \Rightarrow 1]| \leq \delta_{\text{stat}}. \quad (18)$$

Game $G_{6,b}^{\text{ms},i}$ samples a uniformly random $\tilde{k}_b$ and overwrites the encapsulated value. Given an adversary D that distinguishes $G_{5,b}^{\text{ms},i}$ from $G_{6,b}^{\text{ms},i}$ with advantage $\text{Adv}(D)$, we define an adversary B_3 that uses D and wins the IND-CCA game of KEM_{stat} with $\text{Adv}_{\text{KEM}_{\text{stat}}}^{\text{IND-CCA}}(B_3) = \text{Adv}(D)$.

Adversary B_3 gets input $(\hat{p}k, \hat{c}, \hat{k})$ and can query the decapsulation oracle $\text{DEC}(\hat{sk}, \cdot)$ on all inputs except $\hat{c}$. B_3 wins if they can correctly say whether $\hat{c}$ is an encapsulation of $\hat{k}$ or if $\hat{k}$ is a uniformly random value. B_3 runs $G_{5,b}^{\text{ms},i}$ except that they replace the public key of the guessed session $pk_{j'}$ with $\hat{p}k$ in Lines 12 and 36, they replace $(\tilde{c}_b, \tilde{k}_b)$ with $(\hat{c}, \hat{k})$ on Line 09, and instead of decapsulating with $sk_{j'}$ they query the oracle in Line 50 (making sure to skip the query on input $\hat{c}$ and directly output $\hat{k}$ instead). Finally, B_3 outputs whatever D outputs.

If B_3 receives a real encapsulation they are running $G_{5,b}^{\text{ms},i}$, otherwise $\hat{k}$ was random and they are running $G_{6,b}^{\text{ms},i}$. We also see that B_3 is an IND-CCA adversary, so that

$$|\Pr[G_{5,b}^{\text{ms},i} \Rightarrow 1] - \Pr[G_{6,b}^{\text{ms},i} \Rightarrow 1]| \leq \text{Adv}_{\text{KEM}_{\text{stat}}}^{\text{IND-CCA}}(B_3). \quad (19)$$

To prepare for the next case distinction, in $G_{7,b}^{\text{ms},i}$ we record the input to the KDF in the kdfIn associative array. Since this bookkeeping does not change the adversary view, we have

$$\Pr[G_{6,b}^{\text{ms},i} \Rightarrow 1] = \Pr[G_{7,b}^{\text{ms},i} \Rightarrow 1]. \quad (20)$$

CASE 2A1. A KDF INPUT MATCH EXISTS. In this case, we assume that there exists an honest session sID' such that $sID' \neq sID^*$ and $\text{kdfIn}[sID^*] = \text{kdfIn}[sID']$. We formalize this condition in $G_{8,b}^{\text{ms},i}$ on Line 25.

$$\Pr[G_{7,b}^{\text{ms},i} \Rightarrow 1 \wedge \exists sID' \neq sID^* : \text{kdfIn}[sID^*] = \text{kdfIn}[sID']] = \Pr[G_{8,b}^{\text{ms},i,k} \Rightarrow 1]. \quad (21)$$

Game $G_{9,b}^{\text{ms},i,k}$ makes a guess s'' for the session such that $\text{kdfIn}[s''] = \text{kdfIn}[sID^*]$, and we return 0 if our guess turned out to be wrong. Since $G_{8,b}^{\text{ms},i,k}$ established that such a session must exist and our guess is uniformly random over all sessions other than sID^* , we have that

$$\Pr[G_{8,b}^{\text{ms},i,k} \Rightarrow 1] = (n_s - 1) \Pr[G_{9,b}^{\text{ms},i,k} \Rightarrow 1] \quad (22)$$

Game $G_{10,b}^{\text{ms},i,k}$ establishes that s'' is unique. Game $G_{9,b}^{\text{ms},i,k}$ and $G_{10,b}^{\text{ms},i,k}$ are identical unless $G_{10,b}^{\text{ms},i,k}$ returns 0 on Line 30. A collision in kdfIn requires at least a collision in k_a , but by Definition 2.2 we know that such a collision occurs with probability at most $\mu(\text{Secret}_{\text{stat}})$. There are at

most $n_s - 2$ sessions that can collide, so (also by the difference lemma) we get

$$\begin{aligned} & |\Pr[G_{9,b}^{-\text{ms},i,k} \Rightarrow 1] - \Pr[G_{10,b}^{-\text{ms},i,k} \Rightarrow 1]| \\ & \leq \Pr[G_{10,b}^{-\text{ms},i,k} \text{ returns on Line 30}] \leq (n_s - 2)\mu(\text{Secret}_{\text{stat}}) \leq n_s\mu(\text{Secret}_{\text{stat}}). \end{aligned} \quad (23)$$

Game $G_{11,b}^{-\text{ms},i,k}$ replaces k_{pqc} with the same uniformly random sampled value in both sID^* and s'' . Since we model $\text{KDF} : \{0,1\}^{2\ell_{\text{stat}}+\ell_{\text{eph}}} \rightarrow \{0,1\}^{\ell_{\text{pqc}}}$ as a random oracle, we can use the one-way to hiding Lemma 2.5. Similarly to the hop from $G_{4,b}^{\text{st}}$ to $G_{5,b}^{\text{st}}$, we define $H(\cdot) = \text{KDF}(k_a, \cdot, k_e)$ where k_a and k_e are the keys used in $\text{kdfIn}[s'']$.

Then for any fixed k_a, k_e we can replace sampling $\text{KDF} \leftarrow_{\$} (\{0,1\}^{2\ell_{\text{stat}}+\ell_{\text{eph}}} \rightarrow \{0,1\}^{\ell_{\text{pqc}}})$ with sampling $H \leftarrow_{\$} (\{0,1\}^{\ell_{\text{stat}}} \rightarrow \{0,1\}^{\ell_{\text{pqc}}})$. Let $B^H(x, y)$ be the execution of $G_{10,b}^{-\text{ms},i,k}$ where we replace $\tilde{k}_b$ of the test session with x , and $\tilde{k}_{\text{pqc}}$ of the test session with y . Then

$$\Pr[G_{10,b}^{-\text{ms},i,k} \Rightarrow 1] = \Pr[b' = 1 : b' \leftarrow B^H(x, H(x))]$$

and

$$\Pr[G_{11,b}^{-\text{ms},i,k} \Rightarrow 1] = \Pr[b' = 1 : b' \leftarrow B^{H'}(x, y), y \leftarrow_{\$} \{0,1\}^{\ell_{\text{pqc}}}]$$

where we also take the average over $H \leftarrow_{\$} (\{0,1\}^{\ell_{\text{stat}}} \rightarrow \{0,1\}^{\ell_{\text{pqc}}})$ and $x \leftarrow_{\$} \{0,1\}^{\ell_{\text{stat}}}$. As in Lemma 2.5, let C be an oracle algorithm that on input x does the following: pick $i \leftarrow_{\$} \{1, \dots, q_h\}$ and $y \leftarrow_{\$} \{0,1\}^{\ell_{\text{pqc}}}$, run $B^H(x, y)$ until just before the i -th query, measure the argument of the query in the computational basis, and output the measurement outcome.

We define

$$P_C = \Pr[x = x' : x' \leftarrow C^H(x)]$$

where the probability is also over $H \leftarrow_{\$} (\{0,1\}^{\ell_{\text{stat}}} \rightarrow \{0,1\}^{\ell_{\text{pqc}}})$ and $x \leftarrow_{\$} \{0,1\}^{\ell_{\text{stat}}}$. But since x is not available to C at any point, the best the attacker can do is guess, which means that $P_C = \frac{1}{2^{\ell_{\text{stat}}}}$.

Then by Lemma 2.5 we have

$$\left| \Pr[G_{10,b}^{-\text{ms},i,k} \Rightarrow 1] - \Pr[G_{11,b}^{-\text{ms},i,k} \Rightarrow 1] \right| \leq 2q_H \sqrt{P_C} = \frac{2q_H}{\sqrt{2^{\ell_{\text{stat}}}}}. \quad (24)$$

Game $G_{12,b}^{-\text{ms},i,k}$ always aborts the test session. To prove this is indistinguishable from $G_{11,b}^{-\text{ms},i,k}$ we consider two cases, based on whether the transcript matches on all values before the second MAC tag. Specifically, let $\text{sent}[sID^*] = m_1^*$ and $\text{received}[sID^*] = m_2^* = (c_a^*, c_e^*, kID^*, \tau_1^*, \tau_2^*)$ be the transcript of the test session, then we denote $t^* = (m_1^*, (c_a^*, c_e^*, kID^*))$. Similarly we write $(t'', \tau_1'', \tau_2'')$ for the values recorded in $\text{sent}[s'']$ and $\text{received}[s'']$.

CASE 2A1A. In this case the session s'' *almost* matches the test session sID^* : the transcript matches up to and including the first tag, and both sessions have indicated each other as their peers. Specifically in this case $(t^*, \tau_1^*, \text{owner}[sID^*], \text{peer}[sID^*]) = (t'', \tau_1'', \text{peer}[s''], \text{owner}[s''])$. The tag $\tau_2'' = \text{MAC}_{k_{\text{pqc}}, m}^{(\text{pqc})}((t'', \tau_1'', \text{peer}[s''], \text{owner}[s'']))$ was generated in session s'' . Since s'' is not a matching session of sID^* , it must hold that sID^* received a different tag: $\tau_2^* \neq \tau_2''$. But since k_{pqc} (and therefore $k_{\text{pqc}, m}$) has the same value in both sID^* and s'' , both the key and message of the $\text{MAC}^{(\text{pqc})}$ are identical and sID^* will recompute τ_2'' during verification, and therefore reject τ_2^* . In this case, sID^* will therefore abort with certainty:

$$\begin{aligned} \Pr[G_{11,b}^{-\text{ms},i,k} \Rightarrow 1 \wedge (t^*, \tau_1^*, \text{owner}[sID^*], \text{peer}[sID^*]) = (t'', \tau_1'', \text{peer}[s''], \text{owner}[s''])] \\ = \Pr[G_{12,b}^{-\text{ms},i,k} \Rightarrow 1]. \end{aligned} \quad (25)$$

CASE 2A1B. In this case the session s'' differs from sID^* somewhere in the partial transcript, or the sessions do not indicate each other as peers. This means the input to $\text{MAC}_{k_{pqc},m}^{(pqc)}$ will be different, from which we can give a reduction to the OT-sEUF-CMA security of $\text{MAC}^{(pqc)}$. Specifically in this case $(t^*, \tau_1^*, \text{owner}[sID^*], \text{peer}[sID^*]) \neq (t'', \tau_1'', \text{peer}[s''], \text{owner}[s''])$. Note that $G_{11,b}^{-\text{ms},i,k}$ and $G_{12,b}^{-\text{ms},i,k}$ are identical, unless the failure event occurs where $G_{11,b}^{-\text{ms},i,k}$ does *not* abort session sID^* on Line 94, so we bound the probability of this failure event. Let D be the adversary that triggers the failure event. We construct the following adversary against the OT-sEUF-CMA $^{\text{MAC}^{(pqc)}, B_4}$ game. $B_4 = (B_{4,0}, B_{4,1})$ runs $G_{11,b}^{-\text{ms},i,k}$ using D as a subroutine, but with the following modifications: in session s'' on Line 65, instead of computing $\text{MAC}^{(pqc)}$, $B_{4,0}$ outputs $m = (t'', \tau_1'', \text{peer}[s''], \text{owner}[s''])$ and gets back a tag τ from the OT-sEUF-CMA game: B_4 sets $\tau_2'' = \tau$ in $G_{11,b}^{-\text{ms},i,k}$. Then on SEND-M2(s', m_2), $B_{4,1}$ outputs $(m', \tau') = ((t^*, \tau_1^*, \text{owner}[sID^*], \text{peer}[sID^*]), \tau_2^*)$. Note that both $k_{pqc,m}$ in $G_{11,b}^{-\text{ms},i,k}$ and k in the OT-sEUF-CMA game are uniformly random keys, that $m \neq m'$ by the assumption of Case 2a1b, and that B_4 created a valid forgery exactly when $G_{11,b}^{-\text{ms},i,k}$ would not abort, so that

$$\Pr[G_{11,b}^{-\text{ms},i,k} \text{ does not abort on Line 94} \wedge (t^*, \tau_1^*, \text{owner}[sID^*], \text{peer}[sID^*]) \neq (t'', \tau_1'', \text{peer}[s''], \text{owner}[s''])] = \text{Adv}_{\text{MAC}^{(pqc)}}^{\text{OT-sEUF-CMA}}(B_4)$$

and thus by the difference lemma

$$\left| \Pr[G_{11,b}^{-\text{ms},i,k} \Rightarrow 1 \wedge (t^*, \tau_1^*, \text{owner}[sID^*], \text{peer}[sID^*]) \neq (t'', \tau_1'', \text{peer}[s''], \text{owner}[s''])] - \Pr[G_{12,b}^{-\text{ms},i,k} \Rightarrow 1] \right| \leq \text{Adv}_{\text{MAC}^{(pqc)}}^{\text{OT-sEUF-CMA}}(B_4). \quad (26)$$

We combine disjoint Cases 2a1a and 2a1b by taking the sum over Eqs. (25) and (26), and we get

$$|\Pr[G_{11,b}^{-\text{ms},i,k} \Rightarrow 1] - \Pr[G_{12,b}^{-\text{ms},i,k} \Rightarrow 1]| \leq \text{Adv}_{\text{MAC}^{(pqc)}}^{\text{OT-sEUF-CMA}}(B_4). \quad (27)$$

Since both $G_{12,0}^{-\text{ms},i,k}$ and $G_{12,1}^{-\text{ms},i,k}$ always abort the test session, they are indistinguishable:

$$\Pr[G_{12,0}^{-\text{ms},i,k} \Rightarrow 1] = \Pr[G_{12,1}^{-\text{ms},i,k} \Rightarrow 1] = 0. \quad (28)$$

Combining Eqs. (21) to (24), (27) and (28) concludes Case 2a1, we get

$$\begin{aligned} & |\Pr[G_{7,0}^{-\text{ms},i} \Rightarrow 1 \wedge \exists sID' \neq sID^* : \text{kdfIn}[sID^*] = \text{kdfIn}[sID']]| \\ & - \Pr[G_{7,1}^{-\text{ms},i} \Rightarrow 1 \wedge \exists sID' \neq sID^* : \text{kdfIn}[sID^*] = \text{kdfIn}[sID']]| \\ & \leq 2(n_s - 1) \left(n_s \mu(\text{Secret}_{\text{stat}}) + \frac{2q_H}{\sqrt{2^{\ell_{\text{stat}}}}} + \text{Adv}_{\text{MAC}^{(pqc)}}^{\text{OT-sEUF-CMA}}(B_4) \right) \end{aligned} \quad (29)$$

CASE 2A2. NO KDF INPUT MATCH EXISTS.

If no KDF input match exists, then $\text{kdfIn}[sID] \neq \text{kdfIn}[sID^*]$ for all $sID \neq sID^*$. We encode this in $G_{8,b}^{-\text{ms},i,\neg k}$ in Fig. 14, so that

$$\Pr[G_{7,b}^{-\text{ms},i} \Rightarrow 1 \wedge \forall sID \neq sID^* : \text{kdfIn}[sID] \neq \text{kdfIn}[sID^*]] = \Pr[G_{8,b}^{-\text{ms},i,\neg k} \Rightarrow 1]. \quad (30)$$

Game $G_{9,b}^{-\text{ms},i,\neg k}$ replaces k_{pqc} with a random value $\widetilde{k_{pqc}}$. Since this step replaces a single QROM output, identical to the gamehop from $G_{12,b}^{-\text{ms},i,k}$ to $G_{13,b}^{-\text{ms},i,k}$, we have

$$\begin{aligned} |\Pr[G_{8,b}^{-\text{ms},i,\neg k} \Rightarrow 1] - \Pr[G_{9,b}^{-\text{ms},i,\neg k} \Rightarrow 1]| &= |\Pr[G_{10,b}^{-\text{ms},i,k} \Rightarrow 1] - \Pr[G_{11,b}^{-\text{ms},i,k} \Rightarrow 1]| \\ &\leq 2q_H \sqrt{P_C} = \frac{2q_H}{\sqrt{2^{\ell_{\text{stat}}}}}. \end{aligned} \quad (31)$$

GAME $G_{7,b}^{\neg\text{ms},i} - G_{12,b}^{\neg\text{ms},i,\neg k}$	
01 $sID_{ctr} := 0$	
02 $sID^* := 0$	
03 $s' \leftarrow_{\$} \{1, \dots, n_s\}$	// $G_{2,b}^{\neg\text{ms}}$
04 $j' \leftarrow_{\$} [n_{pt}]$	// $G_{3,b}^{\neg\text{ms}}$
05 for $i \in \{1, \dots, n_{pt}\}$	
06 $(pk_i, sk_i) \leftarrow \text{KeyGen}_{stat}()$	
07 $\text{QkdInit}()$	
08 $(\tilde{c}_b, \tilde{k}_b) := \text{Encaps}_{stat}(pk_{j'})$	// $G_{5,b}^{\neg\text{ms},i}$
09 $\tilde{k}_b \leftarrow_{\$} \{0, 1\}^{\ell_{stat}}$	// $G_{6,b}^{\neg\text{ms},i}$
10 $\tilde{k}_{pgc} \leftarrow_{\$} \{0, 1\}^{\ell_{pgc}}$	// $G_{9,b}^{\neg\text{ms},i,\neg k}$
11 $\tilde{k}_{sess} \leftarrow_{\$} \{0, 1\}^{\ell_{sess}}$	// $G_{10,b}^{\neg\text{ms},i,\neg k}$
12 $b' \leftarrow A^{\circ}(pk_1, \dots, pk_{n_{pt}})$	
13 $matches := \text{FindMatches}(sID^*)$	
14 if $matches \neq \emptyset$: return 0	// $G_{1,b}^{\neg\text{ms}}$
15 if $\text{AttackPQC}(matches)$	
16 return 0	
17 if $s' \neq sID^*$: return 0	// $G_{2,b}^{\neg\text{ms}}$
18 if $j' \neq \text{peer}[sID^*]$: return 0	// $G_{3,b}^{\neg\text{ms}}$
19 if $\text{role}[sID^*] \neq \text{INITIATOR}$: return 0	// $G_{4,b}^{\neg\text{ms},i}$
20 for $sID \in \{1, \dots, n_s\} \setminus \{sID^*\}$	
21 if $\text{kdfIn}[sID] = \text{kdfIn}[sID^*]$	
22 return 0	// $G_{8,b}^{\neg\text{ms},i,\neg k}$
23 return b'	
SEND-INIT (sID)	
24 $(i, j) := (\text{owner}[sID], \text{peer}[sID])$	
25 $(\rho, s) := (\text{role}[sID], \text{state}[sID])$	
26 if $i = \perp$ or $\rho \neq \text{INITIATOR}$ or $s \neq \perp$	
27 abort	
28 $(c_b, k_b) := \text{Encaps}_{stat}(pk_j)$	
29 if $sID = s'$	
30 $(c_b, k_b) := (\tilde{c}_b, \tilde{k}_b)$	// $G_{5,b}^{\neg\text{ms},i}$
31 $(pk_e, sk_e) := \text{KeyGen}_{eph}()$	
32 $m_1 := (c_b, pk_e)$	
33 $\text{sent}[sID] := m_1$	
34 $\text{state}[sID] := (k_b, sk_e, m_1)$	
35 return m_1	
SEND-M1 (sID, m_1)	
36 $(i, j) := (\text{owner}[sID], \text{peer}[sID])$	
37 $(\rho, s) := (\text{role}[sID], \text{state}[sID])$	
38 if $i = \perp$ or $\rho \neq \text{RESPONDER}$ or $s \neq \perp$	
39 abort	
40 $\text{received}[sID] := m_1$	
41 $(c_b, pk_e) := m_1$	
42 $k'_b := \text{Decaps}_{stat}(sk_i, c_b)$	
43 if $i = j'$ and $c_b = \tilde{c}_b$	
44 $k'_b := \tilde{k}_b$	// $G_{5,b}^{\neg\text{ms},i}$
45 $(c_a, k_a) \leftarrow \text{Encaps}_{stat}(pk_j)$	
46 $(c_e, k_e) \leftarrow \text{Encaps}_{eph}(pk_e)$	
47 $\text{kdfIn}[sID] = (k'_b, k_a, k_e)$	// $G_{7,b}^{\neg\text{ms},i}$
48 $k_{pgc} := \text{KDF}(k'_b, k_a, k_e)$	
49 $(kID, k_{qkd}) := \text{QKD-GET-KEY}(sID, \ell_{qkd})$	
50 $t := (m_1, (c_a, c_e, kID))$	
51 $(k_{pgc,m} \parallel k_{pgc,s}) := k_{pgc}$	
52 $(k_{qkd,m} \parallel k_{qkd,s}) := k_{qkd}$	
53 $k_{sess} := k_{pgc,s} \oplus k_{qkd,s}$	
54 $\tau_1 = \text{MAC}_{k_{qkd,m}}^{(qkd)}((t, j, i))$	
55 $\tau_2 = \text{MAC}_{k_{pgc,m}}^{(pgc)}((t, \tau_1, j, i))$	
56 $m_2 := (c_a, c_e, kID, \tau_1, \tau_2)$	
57 $\text{sent}[sID] := m_2$	
58 $\text{sesKey}[sID] := k_{sess}$	
59 $\text{state}[sID] := \text{ACCEPT}$	
60 return m_2	
SEND-M2 (sID, m_2)	
61 $(i, j) := (\text{owner}[sID], \text{peer}[sID])$	
62 $(\rho, s) := (\text{role}[sID], \text{state}[sID])$	
63 if $i = \perp$ or $\rho \neq \text{INITIATOR}$	
64 abort	
65 $\text{received}[sID] := m_2$	
66 $(c_a, c_e, kID, \tau_1, \tau_2) := m_2$	
67 $(k_b, sk_e, m_1) := s$	
68 $k'_a := \text{Decaps}_{stat}(sk_i, c_a)$	
69 $k'_e := \text{Decaps}_{eph}(sk_e, c_e)$	
70 if $sID = s'$	
71 $\text{kdfIn}[sID] = (k_b, k'_a, k'_e)$	// $G_{7,b}^{\neg\text{ms},i}$
72 $k_{pgc} := \text{KDF}(k_b, k'_a, k'_e)$	
73 if $sID = s'$	
74 $k_{pgc} := \tilde{k}_{pgc}$	// $G_{9,b}^{\neg\text{ms},i,\neg k}$
75 $k_{qkd} :=$	
QKD-GET-KEY-WITH-ID (sID, kID)	
76 if $k_{qkd} = \perp$: abort	
77 $t := (m_1, (c_a, c_e, kID))$	
78 $(k_{pgc,m} \parallel k_{pgc,s}) := k_{pgc}$	
79 $(k_{qkd,m} \parallel k_{qkd,s}) := k_{qkd}$	
80 $k_{sess} := k_{pgc,s} \oplus k_{qkd,s}$	
81 if $sID = s'$	
82 $k_{sess} := \tilde{k}_{sess}$	// $G_{10,b}^{\neg\text{ms},i,\neg k}$
83 $\tau_1 = \text{MAC}_{k_{qkd,m}}^{(qkd)}((t, i, j))$	
84 $\tau_2 = \text{MAC}_{k_{pgc,m}}^{(pgc)}((t, \tau_1, i, j))$	
85 if $\tau_1 \neq \tau'_1$ or $\tau_2 \neq \tau'_2$: abort	
86 $\text{sesKey}[sID] := k_{sess}$	
87 $\text{state}[sID] := \text{ACCEPT}$	

Fig. 14: Game $G_{7,b}^{\neg\text{ms},i,\neg k}$: non-matching sessions, initiator session, matching kdfIn does not exist

From here we could prove that the initiator will abort, unless the adversary creates a forgery against $\text{MAC}^{(pqc)}$. However, even a forgery will not help the adversary, because the key that would be accepted is uniformly random, which we prove next.

Game $G_{10,b}^{\neg\text{ms},i,\neg k}$ replaces k_{sess} with a random value. Since $k_{pqc,s}$ acts as a one-time pad on $k_{kd,s}$, this replacement does not change the view of the adversary, so that

$$\Pr[G_{9,b}^{\neg\text{ms},i,\neg k} \Rightarrow 1] = \Pr[G_{10,b}^{\neg\text{ms},i,\neg k} \Rightarrow 1]. \quad (32)$$

Both k_0^* and k_1^* are uniformly random keys in $G_{10,b}^{\neg\text{ms},i,\neg k}$, so that

$$\Pr[G_{10,0}^{\neg\text{ms},i,\neg k} \Rightarrow 1] = \Pr[G_{10,1}^{\neg\text{ms},i,\neg k} \Rightarrow 1]. \quad (33)$$

So that for Case **2a2**, we combine Eqs. (30) to (33) to get

$$\begin{aligned} & |\Pr[G_{7,0}^{\neg\text{ms},i} \Rightarrow 1 \wedge \forall sID \neq sID^* : \text{kdfIn}[sID] \neq \text{kdfIn}[sID^*]] \\ & \quad - \Pr[G_{7,1}^{\neg\text{ms},i} \Rightarrow 1 \wedge \forall sID \neq sID^* : \text{kdfIn}[sID] \neq \text{kdfIn}[sID^*]]| \leq \frac{4q_H}{\sqrt{2^{\ell_{\text{stat}}}}}. \end{aligned} \quad (34)$$

To conclude mutually exclusive Cases **2a1** and **2a2**, we take the sum over Eqs. (29) and (34) and get

$$\begin{aligned} & |\Pr[G_{7,0}^{\neg\text{ms},i} \Rightarrow 1] - \Pr[G_{7,1}^{\neg\text{ms},i} \Rightarrow 1]| \\ & \leq 2(n_s - 1) \left(n_s \mu(\text{Secret}_{\text{stat}}) + \frac{2q_H}{\sqrt{2^{\ell_{\text{stat}}}}} + \text{Adv}_{\text{MAC}^{(pqc)}}^{\text{OT-sEUF-CMA}}(B_4) \right) + \frac{4q_H}{\sqrt{2^{\ell_{\text{stat}}}}} \\ & \leq 2n_s \left(n_s \mu(\text{Secret}_{\text{stat}}) + \frac{2q_H}{\sqrt{2^{\ell_{\text{stat}}}}} + \text{Adv}_{\text{MAC}^{(pqc)}}^{\text{OT-sEUF-CMA}}(B_4) \right). \end{aligned} \quad (35)$$

so that for Case **2a** we combine Eqs. (17) to (20) and (35) to conclude

$$\begin{aligned} & |\Pr[G_{3,0}^{\neg\text{ms}} \Rightarrow 1 \wedge \text{role}[sID^*] = \text{INITIATOR}] - \Pr[G_{3,1}^{\neg\text{ms}} \Rightarrow 1 \wedge \text{role}[sID^*] = \text{INITIATOR}]| \\ & \leq 2\delta_{\text{stat}} + 2\text{Adv}_{\text{KEM}_{\text{stat}}}^{\text{IND-CCA}}(B_3) + 2n_s \left(n_s \mu(\text{Secret}_{\text{stat}}) + \frac{q_H}{\sqrt{2^{\ell_{\text{stat}}}}} + \text{Adv}_{\text{MAC}^{(pqc)}}^{\text{OT-sEUF-CMA}}(B_4) \right). \end{aligned} \quad (36)$$

CASE 2B: TEST SESSION IS A RESPONDER. The proof is mostly analogous to Case **2a**, except that the responder itself will never abort. Instead, we will require that any non-matching initiator session that computes the same k_{pqc} will abort. Also, Case **2b1** requires one extra gamehop: $G_{13,b}^{\neg\text{ms},r,k}$, encoded in Lines **12** and **55**.

Formally, Case **2b** occurs when $\text{role}[sID^*] = \text{RESPONDER}$, this is encoded in $G_{4,b}^{\neg\text{ms},r}$ (see Fig. **15**):

$$\Pr[G_{3,b}^{\neg\text{ms}} \Rightarrow 1 \wedge \text{role}[sID^*] = \text{RESPONDER}] = \Pr[G_{4,b}^{\neg\text{ms},r} \Rightarrow 1]. \quad (37)$$

In game $G_{5,b}^{\neg\text{ms},r}$, we sample the test-session static encapsulation in advance, and in initiator sessions we replace the decapsulated value with the encapsulated one. The games are identical unless a decapsulation failure occurs in $G_{4,b}^{\neg\text{ms},r}$, which by Definition **2.1** is at most δ_{stat} , so we have

$$|\Pr[G_{4,b}^{\neg\text{ms},r} \Rightarrow 1] - \Pr[G_{5,b}^{\neg\text{ms},r} \Rightarrow 1]| \leq \delta_{\text{stat}}. \quad (38)$$

Game $G_{6,b}^{\neg\text{ms},r}$ samples a uniformly random $\widetilde{k_a}$ and overwrites the encapsulated value. Given an adversary D that distinguishes $G_{5,b}^{\neg\text{ms},r}$ from $G_{6,b}^{\neg\text{ms},r}$ with advantage $\text{Adv}(D)$, we define an adversary B_5 that uses D and wins the IND-CCA game of KEM_{stat} with $\text{Adv}_{\text{KEM}_{\text{stat}}}^{\text{IND-CCA}}(B_5) = \text{Adv}(D)$.

GAME $G_{.,b}^{\neg\text{ms},r}$		
01 $sID_{ctr} := 0$		
02 $sID^* := 0$		
03 $s' \leftarrow_{\$} \{1, \dots, n_s\}$	$// G_{2,b}^{\neg\text{ms}}$	
04 $s'' \leftarrow_{\$} \{1, \dots, n_s\} \setminus \{s'\}$	$// G_{9,b}^{\neg\text{ms},r,k}$	
05 $j' \leftarrow_{\$} \{1, \dots, n_{pt}\}$	$// G_{3,b}^{\neg\text{ms}}$	
06 for $i \in \{1, \dots, n_{pt}\}$		
07 $(pk_i, sk_i) \leftarrow \text{KeyGen}_{stat}()$		
08 $\text{QkdInit}()$		
09 $(\tilde{c}_a, k_a) := \text{Encaps}_{stat}(pk_{j'})$	$// G_{5,b}^{\neg\text{ms},r}$	
10 $\tilde{k}_a \leftarrow_{\$} \{0, 1\}^{\ell_{stat}}$	$// G_{6,b}^{\neg\text{ms},r}$	
11 $\tilde{k}_{pqc} \leftarrow_{\$} \{0, 1\}^{\ell_{pqc}}$	$// G_{11,b}^{\neg\text{ms},r,k}$	
12 $\tilde{k}_{sess} \leftarrow_{\$} \{0, 1\}^{\ell_{sess}}$	$// G_{13,b}^{\neg\text{ms},r,k}$	
13 $b' \leftarrow A^{\odot}(pk_1, \dots, pk_{n_{pt}})$		
14 $\text{matches} := \text{FindMatches}(sID^*)$		
15 if $\text{matches} \neq \emptyset$: return 0	$// G_{1,b}^{\neg\text{ms}}$	
16 if $\text{AttackPQC}(\text{matches})$		
17 return 0		
18 if $s' \neq sID^*$: return 0	$// G_{2,b}^{\neg\text{ms}}$	
19 if $j' \neq \text{peer}[sID^*]$: return 0	$// G_{3,b}^{\neg\text{ms}}$	
20 if $\text{role}[sID^*] \neq \text{RESPONDER}$		
21 return 0	$// G_{4,b}^{\neg\text{ms},r}$	
22 $\text{kdfMatch} := \text{false}$		
23 for $sID \in [S] \setminus \{sID^*\}$		
24 if $\text{kdfIn}[sID] = \text{kdfIn}[sID^*]$		
25 $\text{kdfMatch} := \text{true}$		
26 if $\neg \text{kdfMatch}$: return 0	$// G_{8,b}^{\neg\text{ms},r,k}$	
27 if $\text{kdfIn}[s''] \neq \text{kdfIn}[sID^*]$	$// G_{9,b}^{\neg\text{ms},r,k}$	
28 return 0		
29 for $sID \in \{1, \dots, n_s\} \setminus \{s'', sID^*\}$		
30 if $\text{kdfIn}[sID] = \text{kdfIn}[s'']$	$// G_{10,b}^{\neg\text{ms},r,k}$	
31 return 0		
32 return b'		
SEND-M1 (sID, m_1)		
33 $(i, j) := (\text{owner}[sID], \text{peer}[sID])$		
34 $(\rho, s) := (\text{role}[sID], \text{state}[sID])$		
35 if $i = \perp$ or $\rho \neq \text{RESPONDER}$ or $s \neq \perp$		
36 abort		
37 $\text{received}[sID] := m_1$		
38 $(c_b, pk_e) := m_1$		
39 $k'_b := \text{Decaps}_{stat}(sk_i, c_b)$		
40 $(c_a, k_a) \leftarrow \text{Encaps}_{stat}(pk_j)$		
41 if $sID = s'$		
42 $(c_a, k_a) := (\tilde{c}_a, \tilde{k}_a)$	$// G_{5,b}^{\neg\text{ms},r}$	
43 $(c_e, k_e) \leftarrow \text{Encaps}_{eph}(pk_e)$		
44 if $sID = s'$		
45 $\text{kdfIn}[sID] = (k'_b, k_a, k_e)$	$// G_{7,b}^{\neg\text{ms},r}$	
46 $k_{pqc} := \text{KDF}(k'_b, k_a, k_e)$		
47 if $sID = s'$		
48 $k_{pqc} := \tilde{k}_{pqc}$	$// G_{11,b}^{\neg\text{ms},r,k}$	
49 $(kID, k_{qkd}) := \text{QKD-GET-KEY}(sID, \ell_{qkd})$		
50 $t := (m_1, (c_a, c_e, kID))$		
51 $(k_{pqc,m} \parallel k_{pqc,s}) := k_{pqc}$		
52 $(k_{qkd,m} \parallel k_{qkd,s}) := k_{qkd}$		
53 $k_{sess} := k_{pqc,s} \oplus k_{qkd,s}$		
54 if $sID = s'$		
55 $k_{sess} = \tilde{k}_{sess}$	$// G_{13,b}^{\neg\text{ms},r,k}$	
56 $\tau_1 = \text{MAC}_{k_{qkd,m}}^{(qkd)}((t, j, i))$		
57 $\tau_2 = \text{MAC}_{k_{pqc,m}}^{(pqc)}((t, \tau_1, j, i))$		
58 $m_2 := (c_a, c_e, kID, \tau_1, \tau_2)$		
59 $\text{sent}[sID] := m_2$		
60 $\text{sesKey}[sID] := k_{sess}$		
61 $\text{state}[sID] := \text{ACCEPT}$		
62 return m_2		
SEND-M2 (sID, m_2)		
63 $(i, j) := (\text{owner}[sID], \text{peer}[sID])$		
64 $(\rho, s) := (\text{role}[sID], \text{state}[sID])$		
65 if $i = \perp$ or $\rho \neq \text{INITIATOR}$: abort		
66 $\text{received}[sID] := m_2$		
67 $(c_a, c_e, kID, \tau'_1, \tau'_2) := m_2$		
68 $(k'_b, sk_e, m_1) := s$		
69 $k'_a := \text{Decaps}_{stat}(sk_i, c_a)$		
70 if $i = j'$ and $c_a = \tilde{c}_a$		
71 $k'_a := \tilde{k}_a$	$// G_{5,b}^{\neg\text{ms},r}$	
72 $k'_e := \text{Decaps}_{eph}(sk_e, c_e)$		
73 $\text{kdfIn}[sID] = (k'_b, k'_a, k'_e)$	$// G_{7,b}^{\neg\text{ms},r}$	
74 $k_{pqc} := \text{KDF}(k'_b, k'_a, k'_e)$		
75 if $sID = s''$		
76 $k_{pqc} := \tilde{k}_{pqc}$	$// G_{11,b}^{\neg\text{ms},r,k}$	
77 $k_{qkd} :=$		
$\text{QKD-GET-KEY-WITH-ID}(sID, kID)$		
78 if $k_{qkd} = \perp$: abort		
79 $t := (m_1, (c_a, c_e, kID))$		
80 $(k_{pqc,m} \parallel k_{pqc,s}) := k_{pqc}$		
81 $(k_{qkd,m} \parallel k_{qkd,s}) := k_{qkd}$		
82 $k_{sess} := k_{pqc,s} \oplus k_{qkd,s}$		
83 $\tau_1 = \text{MAC}_{k_{qkd,m}}^{(qkd)}((t, i, j))$		
84 $\tau_2 = \text{MAC}_{k_{pqc,m}}^{(pqc)}((t, \tau_1, i, j))$		
85 if $sID = s''$: abort	$// G_{12,b}^{\neg\text{ms},r,k}$	
86 if $\tau_1 \neq \tau'_1$ or $\tau_2 \neq \tau'_2$: abort		
87 $\text{sesKey}[sID] := k_{sess}$		
88 $\text{state}[sID] := \text{ACCEPT}$		

Fig. 15: Game $G_{.,b}^{\neg\text{ms},r}$: non-matching sessions, responder test session

Adversary B gets input $(\hat{pk}, \hat{c}, \hat{k})$ and can query the decapsulation oracle $\text{DEC}(\hat{sk}, \cdot)$ on all inputs except $\hat{c}$. B_5 wins if they can correctly say whether $\hat{c}$ is an encapsulation of $\hat{k}$ or if $\hat{k}$ is a uniformly random value. B_5 runs $G_{5,b}^{\text{ms},r}$ except that they replace the public key of the guessed session $pk_{j'}$ with $\hat{pk}$ in Lines 13 and 40, they replace $(\tilde{c}_a, \tilde{k}_a)$ with $(\hat{c}, \hat{k})$ on Line 09, and instead of decapsulating with $sk_{j'}$ they query the oracle in Line 69 (making sure to skip the query on input $\hat{c}$ and directly output $\hat{k}$ instead). Finally, B_5 outputs whatever D outputs.

If B_5 receives a real encapsulation they are running $G_{5,b}^{\text{ms},i}$, otherwise $\hat{k}$ was random and they are running $G_{6,b}^{\text{ms},i}$. We also see that B_5 is an IND-CCA adversary, so that

$$|\Pr[G_{5,b}^{\text{ms},r} \Rightarrow 1] - \Pr[G_{6,b}^{\text{ms},r} \Rightarrow 1]| \leq \text{Adv}_{\text{KEM}_{\text{stat}}}^{\text{IND-CCA}}(B_5). \quad (39)$$

To prepare for the next case distinction, $G_{7,b}^{\text{ms},r}$ records the input to the KDF in the kdfIn associative array. Since this bookkeeping does not change the adversary view, we have

$$\Pr[G_{6,b}^{\text{ms},r} \Rightarrow 1] = \Pr[G_{7,b}^{\text{ms},r} \Rightarrow 1]. \quad (40)$$

CASE 2B1. A KDF INPUT MATCH EXISTS. In this case, we assume that there exists an honest session sID' such that $sID' \neq sID^*$ and $\text{kdfIn}[sID^*] = \text{kdfIn}[sID']$. We formalize this condition in $G_{8,b}^{\text{ms},r}$:

$$\Pr[G_{7,b}^{\text{ms},r} \Rightarrow 1 \wedge \exists sID' \neq sID^* : \text{kdfIn}[sID^*] = \text{kdfIn}[sID']] = \Pr[G_{8,b}^{\text{ms},r,k} \Rightarrow 1]. \quad (41)$$

Game $G_{9,b}^{\text{ms},r,k}$ makes a guess s'' for the session such that $\text{kdfIn}[s''] = \text{kdfIn}[sID^*]$, and we return 0 if our guess turned out to be wrong. Since $G_{8,b}^{\text{ms},r,k}$ established that such a session must exist and our guess is uniformly random over all sessions other than sID^* , we have that

$$\Pr[G_{8,b}^{\text{ms},r,k} \Rightarrow 1] = (n_s - 1) \Pr[G_{9,b}^{\text{ms},r,k} \Rightarrow 1]. \quad (42)$$

Game $G_{10,b}^{\text{ms},r,k}$ establishes that s'' is unique. Game $G_{9,b}^{\text{ms},r,k}$ and $G_{10,b}^{\text{ms},r,k}$ are identical unless $G_{10,b}^{\text{ms},r,k}$ returns on Line 31. A collision in kdfIn requires at least a collision in k_b , but by Definition 2.2, we know that such a collision occurs with probability at most $\mu(\text{Secret}_{\text{stat}})$. There are at most $n_s - 2$ sessions that can collide, so (also by the difference lemma) we get

$$\begin{aligned} |\Pr[G_{9,b}^{\text{ms},r,k} \Rightarrow 1] - \Pr[G_{10,b}^{\text{ms},r,k} \Rightarrow 1]| &\leq \Pr[G_{10,b}^{\text{ms},r,k} \text{ returns on Line 31}] \\ &\leq (n_s - 2)\mu(\text{Secret}_{\text{stat}}) \leq n_s\mu(\text{Secret}_{\text{stat}}). \end{aligned} \quad (43)$$

Game $G_{11,b}^{\text{ms},r,k}$ replaces k_{pqc} with the same uniformly random sampled value in both sID^* and s'' . We are replacing an output of the QROM, which is analogous to the gamehop from $G_{11,b}^{\text{ms},i,k}$ to $G_{12,b}^{\text{ms},i,k}$, so we have

$$\begin{aligned} |\Pr[G_{10,b}^{\text{ms},r,k} \Rightarrow 1] - \Pr[G_{11,b}^{\text{ms},r,k} \Rightarrow 1]| &= |\Pr[G_{10,b}^{\text{ms},i,k} \Rightarrow 1] - \Pr[G_{11,b}^{\text{ms},i,k} \Rightarrow 1]| \\ &\leq 2q_H\sqrt{P_C} = \frac{2q_H}{\sqrt{2^{\ell_{\text{stat}}}}}. \end{aligned} \quad (44)$$

Game $G_{12,b}^{\text{ms},r,k}$ always aborts initiator session s'' . To prove this is indistinguishable from $G_{11,b}^{\text{ms},r,k}$ we consider two cases, based on whether the transcript matches on all values before the second

MAC tag. Specifically, let $\text{received}[sID^*] = m_1^*$ and $\text{sent}[sID^*] = m_2^* = (c_a^*, c_e^*, kID^*, \tau_1^*, \tau_2^*)$ be the transcript of the test session, then we denote $t^* = (m_1^*, (c_a^*, c_e^*, kID^*))$. Similarly we write $(t'', \tau_1'', \tau_2'')$ for the values recorded in $\text{sent}[s'']$ and $\text{received}[s'']$.

CASE 2B1A. In this case the session s'' *almost* matches the test session sID^* : the transcript matches up to and including the first tag, and both sessions have indicated each other as their peers. Specifically in this case $(t^*, \tau_1^*, \text{peer}[sID^*], \text{owner}[sID^*]) = (t'', \tau_1'', \text{owner}[s''], \text{peer}[s''])$. The tag $\tau_2^* = \text{MAC}_{k_{pqc,m}}^{(pqc)}((t^*, \tau_1^*, \text{peer}[sID^*], \text{owner}[sID^*]))$ was generated in session sID^* . Since s'' is not a matching session of sID^* , it must hold that s'' received a different tag: $\tau_2'' \neq \tau_2^*$. But since k_{pqc} (and therefore $k_{pqc,m}$) has the same value in both sID^* and s'' , both the key and message of the $\text{MAC}^{(pqc)}$ are identical and sID'' will recompute τ_2^* during verification, and therefore reject τ_2'' . In this case, s'' will therefore abort with certainty:

$$\begin{aligned} \Pr[G_{11,b}^{\neg \text{ms},r,k} \Rightarrow 1 \wedge (t^*, \tau_1^*, \text{peer}[sID^*], \text{owner}[sID^*]) = (t'', \tau_1'', \text{owner}[s''], \text{peer}[s''])] \\ = \Pr[G_{12,b}^{\neg \text{ms},r,k} \Rightarrow 1]. \end{aligned} \quad (45)$$

CASE 2B1B. In this case the session s'' differs from sID^* somewhere in the partial transcript, or the sessions do not indicate each other as peers. This means the input to $\text{MAC}_{k_{pqc,m}}^{(pqc)}$ will be different, from which we can give a reduction to the OT-sEUF-CMA security of $\text{MAC}^{(pqc)}$. Specifically in this case $(t^*, \tau_1^*, \text{peer}[sID^*], \text{owner}[sID^*]) \neq (t'', \tau_1'', \text{owner}[s''], \text{peer}[s''])$. Note that $G_{11,b}^{\neg \text{ms},r,k}$ and $G_{12,b}^{\neg \text{ms},r,k}$ are identical, unless the failure event occurs where $G_{11,b}^{\neg \text{ms},r,k}$ does *not* abort session s'' on Line 86, so we bound the probability of this failure event. Let D be the adversary that triggers the failure event. We construct the following adversary against the OT-sEUF-CMA $_{\text{MAC}^{(pqc)}, B_6}$ game. $B_6 = (B_{6,0}, B_{6,1})$ runs $G_{11,b}^{\neg \text{ms},r,k}$ using D as a subroutine, but with the following modifications: in session s' (which equals sID^*) on Line 57, instead of computing $\text{MAC}^{(pqc)}$, $B_{6,0}$ outputs $m = (t^*, \tau_1^*, \text{peer}[sID^*], \text{owner}[sID^*])$ and gets back a tag τ from the OT-sEUF-CMA game: B_6 sets $\tau_2^* = \tau$ in $G_{11,b}^{\neg \text{ms},r,k}$. Then on **SEND-M2**(s'', m_2), $B_{6,1}$ outputs $(m', \tau') = ((t'', \tau_1'', \text{owner}[s''], \text{peer}[s'']), \tau_2'')$. Note that both $k_{pqc,m}$ in $G_{11,b}^{\neg \text{ms},r,k}$ and k in the OT-sEUF-CMA game are uniformly random keys, that $m \neq m'$ by the assumption of Case 2b1b, and that B_6 created a valid forgery exactly when $G_{11,b}^{\neg \text{ms},r,k}$ would not abort, so that

$$\begin{aligned} \Pr[G_{11,b}^{\neg \text{ms},r,k} \text{ does not abort on Line 86} \wedge \\ (t^*, \tau_1^*, \text{peer}[sID^*], \text{owner}[sID^*]) \neq (t'', \tau_1'', \text{owner}[s''], \text{peer}[s''])] = \text{Adv}_{\text{MAC}^{(pqc)}}^{\text{OT-sEUF-CMA}}(B_6) \end{aligned}$$

and thus by the difference lemma

$$\begin{aligned} \left| \Pr[G_{11,b}^{\neg \text{ms},r,k} \Rightarrow 1 \wedge (t^*, \tau_1^*, \text{peer}[sID^*], \text{owner}[sID^*]) \neq (t'', \tau_1'', \text{owner}[s''], \text{peer}[s''])] \right. \\ \left. - \Pr[G_{12,b}^{\neg \text{ms},r,k} \Rightarrow 1] \right| \leq \text{Adv}_{\text{MAC}^{(pqc)}}^{\text{OT-sEUF-CMA}}(B_6). \end{aligned} \quad (46)$$

We combine disjoint Cases 2b1a and 2b1b by taking the sum over Eqs. (45) and (46), we get

$$|\Pr[G_{11,b}^{\neg \text{ms},r,k} \Rightarrow 1] - \Pr[G_{12,b}^{\neg \text{ms},r,k} \Rightarrow 1]| \leq \text{Adv}_{\text{MAC}^{(pqc)}}^{\text{OT-sEUF-CMA}}(B_6). \quad (47)$$

We established that the initiator session s'' always aborts in $G_{12,b}^{\neg \text{ms},r,k}$. While this sufficed in Case 2a1, here we require one additional gamehop. In $G_{13,b}^{\neg \text{ms},r,k}$ we replace k_{sess} with a uniformly

random value. Since $G_{12,b}^{\text{ms},r,k}$ has established that $\widetilde{k_{pqc}}$ does not occur in the view of the adversary via session s'' , $k_{pqc,s}$ of the test session acts as a one-time pad in $G_{13,b}^{\text{ms},r,k}$, so that

$$\Pr[G_{12,b}^{\text{ms},r,k} \Rightarrow 1] = \Pr[G_{13,b}^{\text{ms},r,k} \Rightarrow 1] \quad (48)$$

which means that k_0^* and k_1^* are uniformly random keys in $G_{13,b}^{\text{ms},r,k}$, and

$$\Pr[G_{13,0}^{\text{ms},r,k} \Rightarrow 1] = \Pr[G_{13,1}^{\text{ms},r,k} \Rightarrow 1]. \quad (49)$$

Combining Eqs. (41) to (44) and (47) to (49) for Case 2b1, we get

$$\begin{aligned} & |\Pr[G_{7,0}^{\text{ms},r} \Rightarrow 1 \wedge \exists sID' \neq sID^* : \mathbf{kdfIn}[sID^*] = \mathbf{kdfIn}[sID']] \\ & \quad - \Pr[G_{7,1}^{\text{ms},r} \Rightarrow 1 \wedge \exists sID' \neq sID^* : \mathbf{kdfIn}[sID^*] = \mathbf{kdfIn}[sID']]| \\ & \leq 2(n_s - 1) \left(n_s \mu(\text{Secret}_{stat}) + \frac{2q_H}{\sqrt{2^{\ell_{stat}}}} + \text{Adv}_{\text{MAC}(pqc)}^{\text{OT-sEUF-CMA}}(B_6) \right) \end{aligned} \quad (50)$$

CASE 2B2. NO KDF INPUT MATCH EXISTS.

If no KDF input match exists, then $\mathbf{kdfIn}[sID] \neq \mathbf{kdfIn}[sID^*]$ for all $sID \neq sID^*$. We encode this in $G_{8,b}^{\text{ms},r,\neg k}$ in Fig. 16, so that

$$\Pr[G_{7,b}^{\text{ms},r} \Rightarrow 1 \wedge \forall sID \neq sID^* : \mathbf{kdfIn}[sID] \neq \mathbf{kdfIn}[sID^*]] = \Pr[G_{8,b}^{\text{ms},r,\neg k} \Rightarrow 1]. \quad (51)$$

Game $G_{9,b}^{\text{ms},r,\neg k}$ replaces k_{pqc} with a random value $\widetilde{k_{pqc}}$. Since this step replaces a single QROM output, identical to the gamehop from $G_{10,b}^{\text{ms},r,k}$ to $G_{11,b}^{\text{ms},r,k}$, we have

$$\begin{aligned} |\Pr[G_{8,b}^{\text{ms},r,\neg k} \Rightarrow 1] - \Pr[G_{9,b}^{\text{ms},r,\neg k} \Rightarrow 1]| &= |\Pr[G_{10,b}^{\text{ms},r,k} \Rightarrow 1] - \Pr[G_{11,b}^{\text{ms},r,k} \Rightarrow 1]| \\ &\leq 2q_H \sqrt{P_C} = \frac{2q_H}{\sqrt{2^{\ell_{stat}}}}. \end{aligned} \quad (52)$$

Game $G_{10,b}^{\text{ms},r,\neg k}$ replaces k_{sess} with a random value. Since $k_{pqc,s}$ acts as a one-time pad on $k_{qkd,s}$, this does not change the view of the adversary, so that

$$\Pr[G_{9,b}^{\text{ms},r,\neg k} \Rightarrow 1] = \Pr[G_{10,b}^{\text{ms},r,\neg k} \Rightarrow 1]. \quad (53)$$

Both k_0^* and k_1^* are uniformly random keys in $G_{10,b}^{\text{ms},r,\neg k}$, so that

$$\Pr[G_{10,0}^{\text{ms},r,\neg k} \Rightarrow 1] = \Pr[G_{10,1}^{\text{ms},r,\neg k} \Rightarrow 1]. \quad (54)$$

So that combining Eqs. (51) to (54) for Case 2b2, we get

$$\begin{aligned} & |\Pr[G_{7,0}^{\text{ms},i} \Rightarrow 1 \wedge \forall sID \neq sID^* : \mathbf{kdfIn}[sID] \neq \mathbf{kdfIn}[sID^*]] \\ & \quad - \Pr[G_{7,1}^{\text{ms},i} \Rightarrow 1 \wedge \forall sID \neq sID^* : \mathbf{kdfIn}[sID] \neq \mathbf{kdfIn}[sID^*]]| \leq \frac{4q_H}{\sqrt{2^{\ell_{stat}}}}. \end{aligned} \quad (55)$$

To conclude mutually exclusive Cases 2b1 and 2b2, we take the sum over Eqs. (50) and (55) and get

$$\begin{aligned} & |\Pr[G_{7,0}^{\text{ms},r} \Rightarrow 1] - \Pr[G_{7,1}^{\text{ms},r} \Rightarrow 1]| \\ & \leq 2(n_s - 1) \left(n_s \mu(\text{Secret}_{stat}) + \frac{2q_H}{\sqrt{2^{\ell_{stat}}}} + \text{Adv}_{\text{MAC}(pqc)}^{\text{OT-sEUF-CMA}}(B_6) \right) + \frac{4q_H}{\sqrt{2^{\ell_{stat}}}} \\ & \leq 2n_s \left(n_s \mu(\text{Secret}_{stat}) + \frac{2q_H}{\sqrt{2^{\ell_{stat}}}} + \text{Adv}_{\text{MAC}(pqc)}^{\text{OT-sEUF-CMA}}(B_6) \right) \end{aligned} \quad (56)$$

GAME $G_{\cdot,b}^{\neg\text{ms},r,\neg k}$		
01 $sID_{ctr} := 0$		
02 $sID^* := 0$		
03 $s' \leftarrow_{\$} [S]$	$\parallel G_{2,b}^{\neg\text{ms}}$	
04 $j' \leftarrow_{\$} [N]$	$\parallel G_{3,b}^{\neg\text{ms}}$	
05 for $i \in \{1, \dots, N\}$		
06 $(pk_i, sk_i) \leftarrow \text{KeyGen}_{stat}()$		
07 $\text{QkdInit}()$		
08 $(\tilde{c}_a, \tilde{k}_a) := \text{Encaps}_{stat}(pk_{j'})$	$\parallel G_{5,b}^{\neg\text{ms},r}$	
09 $\tilde{k}_a \leftarrow_{\$} \{0, 1\}^{\ell_{stat}}$	$\parallel G_{6,b}^{\neg\text{ms},r}$	
10 $\tilde{k}_{pqc} \leftarrow_{\$} \{0, 1\}^{\ell_{pqc}}$	$\parallel G_{9,b}^{\neg\text{ms},r,\neg k}$	
11 $\tilde{k}_{sess} \leftarrow_{\$} \{0, 1\}^{\ell_{sess}}$	$\parallel G_{10,b}^{\neg\text{ms},r,\neg k}$	
12 $b' \leftarrow A^{\circ}(pk_1, \dots, pk_N)$		
13 $matches := \text{FindMatches}(sID^*)$		
14 if $matches \neq \emptyset$: return 0	$\parallel G_{1,b}^{\neg\text{ms}}$	
15 if $\text{AttackPQC}(matches)$		
16 return 0		
17 if $s' \neq sID^*$: return 0	$\parallel G_{2,b}^{\neg\text{ms}}$	
18 if $j' \neq \text{peer}[sID^*]$: return 0	$\parallel G_{3,b}^{\neg\text{ms}}$	
19 if $\text{role}[sID^*] \neq \text{RESPONDER}$		
20 return 0	$\parallel G_{4,b}^{\neg\text{ms},r}$	
21 for $sID \in [S] \setminus \{sID^*\}$		
22 if $\text{kdfIn}[sID] = \text{kdfIn}[sID^*]$		
23 return 0	$\parallel G_{8,b}^{\neg\text{ms},r,\neg k}$	
24 return b'		
SEND-M1 (sID, m_1)		
25 $(i, j) := (\text{owner}[sID], \text{peer}[sID])$		
26 $(\rho, s) := (\text{role}[sID], \text{state}[sID])$		
27 if $i = \perp$ or $\rho \neq \text{RESPONDER}$ or $s \neq \perp$		
28 abort		
29 $\text{received}[sID] := m_1$		
30 $(c_b, pk_e) := m_1$		
31 $k'_b := \text{Decaps}_{stat}(sk_i, c_b)$		
32 $(c_a, k_a) \leftarrow \text{Encaps}_{stat}(pk_j)$		
33 if $sID = s'$		
34 $(c_a, k_a) := (\tilde{c}_a, \tilde{k}_a)$	$\parallel G_{5,b}^{\neg\text{ms},r}$	
35 $(c_e, k_e) \leftarrow \text{Encaps}_{eph}(pk_e)$		
36 if $sID = s'$		
37 $\text{kdfIn}[sID] = (k'_b, k_a, k_e)$	$\parallel G_{7,b}^{\neg\text{ms},r}$	
38 $k_{pqc} := \text{KDF}(k'_b, k_a, k_e)$		
39 if $sID = s'$		
40 $k_{pqc} := \tilde{k}_{pqc}$		$\parallel G_{9,b}^{\neg\text{ms},r,\neg k}$
41 $(kID, k_{qkd}) := \text{QKD-GET-KEY}(sID, \ell_{qkd})$		
42 $t := (m_1, (c_a, c_e, kID))$		
43 $(k_{pqc,m} \parallel k_{pqc,s}) := k_{pqc}$		
44 $(k_{qkd,m} \parallel k_{qkd,s}) := k_{qkd}$		
45 $k_{sess} := k_{pqc,s} \oplus k_{qkd,s}$		
46 if $sID = s'$		
47 $k_{sess} = \tilde{k}_{sess}$	$\parallel G_{10,b}^{\neg\text{ms},r,\neg k}$	
48 $\tau_1 = \text{MAC}_{k_{qkd,m}}^{(qkd)}((t, j, i))$		
49 $\tau_2 = \text{MAC}_{k_{pqc,m}}^{(pqc)}((t, \tau_1, j, i))$		
50 $m_2 := (c_a, c_e, kID, \tau_1, \tau_2)$		
51 $\text{sent}[sID] := m_2$		
52 $\text{sesKey}[sID] := k_{sess}$		
53 $\text{state}[sID] := \text{ACCEPT}$		
54 return m_2		
SEND-M2 (sID, m_2)		
55 $(i, j) := (\text{owner}[sID], \text{peer}[sID])$		
56 $(\rho, s) := (\text{role}[sID], \text{state}[sID])$		
57 if $i = \perp$ or $\rho \neq \text{INITIATOR}$: abort		
58 $\text{received}[sID] := m_2$		
59 $(c_a, c_e, kID, \tau'_1, \tau'_2) := m_2$		
60 $(k_b, sk_e, m_1) := s$		
61 $k'_a := \text{Decaps}_{stat}(sk_i, c_a)$		
62 if $i = j'$ and $c_a = \tilde{c}_a$		
63 $k'_a := \tilde{k}_a$	$\parallel G_{5,b}^{\neg\text{ms},r}$	
64 $k'_e := \text{Decaps}_{eph}(sk_e, c_e)$		
65 $\text{kdfIn}[sID] = (k_b, k'_a, k'_e)$	$\parallel G_{7,b}^{\neg\text{ms},r}$	
66 $k_{pqc} := \text{KDF}(k_b, k'_a, k'_e)$		
67 $k_{qkd} :=$		
$\text{QKD-GET-KEY-WITH-ID}(sID, kID)$		
68 if $k_{qkd} = \perp$: abort		
69 $t := (m_1, (c_a, c_e, kID))$		
70 $(k_{pqc,m} \parallel k_{pqc,s}) := k_{pqc}$		
71 $(k_{qkd,m} \parallel k_{qkd,s}) := k_{qkd}$		
72 $k_{sess} := k_{pqc,s} \oplus k_{qkd,s}$		
73 $\tau_1 = \text{MAC}_{k_{qkd,m}}^{(qkd)}((t, i, j))$		
74 $\tau_2 = \text{MAC}_{k_{pqc,m}}^{(pqc)}((t, \tau_1, i, j))$		
75 if $\tau_1 \neq \tau'_1$ or $\tau_2 \neq \tau'_2$: abort		
76 $\text{sesKey}[sID] := k_{sess}$		
77 $\text{state}[sID] := \text{ACCEPT}$		

Fig. 16: Game $G_{\cdot,b}^{\neg\text{ms},r,\neg k}$: non-matching sessions, responder test session, no matching kdfIn

so that for Case 2b we combine Eqs. (37) to (40) and (56) to conclude

$$\begin{aligned} & |\Pr[G_{3,0}^{\text{ms}} \Rightarrow 1 \wedge \text{role}[sID^*] = \text{RESPONDER}] - \Pr[G_{3,1}^{\text{ms}} \Rightarrow 1 \wedge \text{role}[sID^*] = \text{RESPONDER}]| \\ & \leq 2\delta_{\text{stat}} + 2\text{Adv}_{\text{KEM}_{\text{stat}}}^{\text{IND-CCA}}(B_5) + 2n_s \left(n_s \mu(\text{Secret}_{\text{stat}}) + \frac{2q_H}{\sqrt{2^{\ell_{\text{stat}}}}} + \text{Adv}_{\text{MAC}^{(pqc)}}^{\text{OT-sEUF-CMA}}(B_6) \right). \end{aligned} \quad (57)$$

For the mutually exclusive events encoded in Cases 2a and 2b, we can take the sum over Eqs. (36) and (57) and see that

$$\begin{aligned} & |\Pr[G_{3,0}^{\text{ms}} \Rightarrow 1] - \Pr[G_{3,1}^{\text{ms}} \Rightarrow 1]| \\ & \leq 4\delta_{\text{stat}} + 4\text{Adv}_{\text{KEM}_{\text{stat}}}^{\text{IND-CCA}}(B_3) + 4n_s \left(n_s \mu(\text{Secret}_{\text{stat}}) + \frac{2q_H}{\sqrt{2^{\ell_{\text{stat}}}}} + \text{Adv}_{\text{MAC}^{(pqc)}}^{\text{OT-sEUF-CMA}}(B_4) \right) \end{aligned} \quad (58)$$

Combining Eqs. (14) to (16) and (58) lets us conclude Case 2, proving Lemma 5.4

$$\begin{aligned} & |\Pr[\text{IND-StAA-PQC}_0(\Pi, A) \Rightarrow 1 \wedge \text{matches} = \emptyset] - \Pr[\text{IND-StAA-PQC}_1(\Pi, A) \Rightarrow 1 \wedge \text{matches} = \emptyset]| \\ & \leq 8n_s n_{\text{pt}} \left(\delta_{\text{stat}} + \text{Adv}_{\text{KEM}_{\text{stat}}}^{\text{IND-CCA}}(B_3) + n_s \left(n_s \mu(\text{Secret}_{\text{stat}}) + \frac{2q_H}{\sqrt{2^{\ell_{\text{stat}}}}} + \text{Adv}_{\text{MAC}^{(pqc)}}^{\text{OT-sEUF-CMA}}(B_4) \right) \right). \end{aligned}$$

5.3 Security proof for QKD-based security, Theorem 5.5

Proof (QKD-based security). We consider security for the IND-AA-QKD game, as defined in Fig. 8, so that we are proving an upper bound for

$$|\Pr[\text{IND-AA-QKD}_0(\Pi, A) \Rightarrow 1] - \Pr[\text{IND-AA-QKD}_1(\Pi, A) \Rightarrow 1]|.$$

By the assumption that QKD is not (trivially) broken, the QKD key used by the test session is **HONEST**, so it is uniformly random and has not leaked to the adversary directly via an **QKD-LEAK** or an **QKD-OVERRIDE** query. The QKD key may have been delivered to one other session, so the main hurdle for proving Theorem 5.5 is to prove that the other session did not leak the QKD key. Since the QKD key is not part of the state and $k_{qkd,s}$ is independent from other values, this could only occur through a **REVEAL** query. If the other session is a completed matching session, then **REVEAL** would constitute a trivial attack. Otherwise the transcript of the other session differs from that of the test session, in that case the **INITIATOR** will reject (unless a $\text{MAC}^{(qkd)}$ was forged).

Game G_0^{qkd} equals IND-AA-QKD_b:

$$\Pr[\text{IND-AA-QKD}_b(\Pi, A) \Rightarrow 1] = \Pr[G_{0,b}^{\text{qkd}} \Rightarrow 1]. \quad (59)$$

Game $G_{1,b}^{\text{qkd}}$ adds Lines 50 and 81, tracking per session which QKD key IDs they use in the associative array **kidUsed**, which are initialized at $\text{kidUsed}[sID] = []$ for all sID . Since this is only doing some bookkeeping, we have

$$\Pr[G_{0,b}^{\text{qkd}} \Rightarrow 1] = \Pr[G_{1,b}^{\text{qkd}} \Rightarrow 1]. \quad (60)$$

Game $G_{2,b}^{\text{qkd}}$ asserts that the test session used just a single kID , and labels it kID^* . The Send queries of accepting sessions must be queried exactly once and in order: **SEND-INIT** and **SEND-M2** for an initiator and **SEND-M1** for a responder. This is enforced by the **state** variable, where we note that for an initiator this enforcement is implicit since Line 73 of **SendM2** aborts if the **state**

GAME $G_{\cdot,b}^{\text{qkd}}$	
01 $sID_{ctr} := 0$	
02 $sID^* := 0$	
03 $sID'_{qSent} \leftarrow \$ \{1, \dots, n_s\}$	$\parallel G_{3,b}^{\text{qkd}}$
04 $\widetilde{k_{qkd}} \leftarrow \$ \{0, 1\}^{\ell_{sess}}$	$\parallel G_{4,b}^{\text{qkd}}$
05 $sID'_{qRecv} \leftarrow \$ \{1, \dots, n_s\} \setminus \{sID'_{qSent}\}$	$\parallel G_{7,b}^{\text{qkd,r,mt}}$
06 $\widetilde{k_{sess}} \leftarrow \$ \{0, 1\}^{\ell_{sess}}$	$\parallel G_{9,b}^{\text{qkd}}$
07 for $i \in \{1, \dots, n_{pt}\}$	
08 $(pk_i, sk_i) \leftarrow \text{KeyGen}_{stat}()$	
09 $\text{QkdInit}()$	
10 $b' \leftarrow A^{\circ}(pk_1, \dots, pk_{n_{pt}})$	
11 $matches := \text{FindMatches}(sID^*)$	
12 if $\text{TrivialQKD}(matches)$	
13 return 0	
14 if $ \text{kidUsed}[sID^*] \neq 1$	
15 return 0	$\parallel G_{2,b}^{\text{qkd}}$
16 $kID^* := \text{kidUsed}[sID^*]$	$\parallel G_{2,b}^{\text{qkd}}$
17 if $sID'_{qSent} \neq \text{qSentSid}[kID^*]$	
18 return 0	$\parallel G_{3,b}^{\text{qkd}}$
19 if $\text{qSentSid}[kID^*] = []$	
20 return 0	$\parallel G_{5,b}^{\text{qkd,r}}$
21 if $sID'_{qRecv} \neq \text{qRecvSid}[kID^*]$	
22 return 0	$\parallel G_{7,b}^{\text{qkd,r,mt}}$
23 return b'	
SEND-INIT (sID)	
24 $(i, j) := (\text{owner}[sID], \text{peer}[sID])$	
25 $(\rho, s) := (\text{role}[sID], \text{state}[sID])$	
26 if $i = \perp$ or $\rho \neq \text{INITIATOR}$ or $s \neq \perp$: abort	
27 $(c_b, k_b) := \text{Encaps}_{stat}(pk_j)$	
28 if $sID = sID'_{qRecv}$	
29 $\text{kb} := k_b$	$\parallel G_{8,b}^{\text{qkd,r,mt}}$
30 $(pk_e, sk_e) := \text{KeyGen}_{eph}()$	
31 $m_1 := (c_b, pk_e)$	
32 $\text{sent}[sID] := m_1$	
33 $\text{state}[sID] := (k_b, sk_e, m_1)$	
34 return m_1	
SEND-M1 (sID, m_1)	
35 $(i, j) := (\text{owner}[sID], \text{peer}[sID])$	
36 $(\rho, s) := (\text{role}[sID], \text{state}[sID])$	
37 if $i = \perp$ or $\rho \neq \text{RESPONDER}$ or $s \neq \perp$	
38 abort	
39 $\text{received}[sID] := m_1$	
40 $(c_b, pk_e) := m_1$	
41 $k'_b := \text{Decaps}_{stat}(sk_i, k_b)$	
42 if $sID = sID'_{qSent}$	
43 $k'_b := \text{kb}$	$\parallel G_{8,b}^{\text{qkd,r,mt}}$
44 $(c_a, k_a) \leftarrow \text{Encaps}_{stat}(pk_j)$	
45 $(c_e, k_e) \leftarrow \text{Encaps}_{eph}(pk_e)$	
46 if $sID = sID'_{qSent}$	
47 $(\text{ka}, \text{ke}) := (k_a, k_e)$	$\parallel G_{8,b}^{\text{qkd,r,mt}}$
48 $k_{pqc} := \text{KDF}(k'_b, k_a, k_e)$	
49 $(kID, k_{qkd}) := \text{QKD-GET-KEY}(sID, \ell_{qkd})$	
50 $\text{kidUsed}[sID] += [kID]$	$\parallel G_{1,b}^{\text{qkd}}$
51 if $sID = sID'_{qSent}$	
52 $k_{qkd} := \widetilde{k_{qkd}}$	$\parallel G_{4,b}^{\text{qkd}}$
53 $t := (m_1, (c_a, c_e, kID))$	
54 if $sID = sID'_{qSent}$	
55 $\text{tresp} = t$	$\parallel G_{5,b}^{\text{qkd,r},\neg\text{mt}}$
56 $(k_{qkd,m} \parallel k_{qkd,s}) := k_{qkd}$	
57 $(k_{pqc,m} \parallel k_{pqc,s}) := k_{pqc}$	
58 $k_{sess} := k_{qkd,s} \oplus k_{pqc,s}$	
59 if $sID = sID'_{qSent}$	
60 $k_{sess} := \widetilde{k_{sess}}$	$\parallel G_{9,b}^{\text{qkd}}$
61 $\tau_1 = \text{MAC}_{k_{qkd,m}}^{(qkd)}((t, j, i))$	
62 $\tau_2 = \text{MAC}_{k_{pqc,m}}^{(pqc)}((t, \tau_1, j, i))$	
63 $m_2 := (c_a, c_e, \tau_1, \tau_2)$	
64 $\text{sent}[sID] := m_2$	
65 $\text{sesKey}[sID] := k_{sess}$	
66 $\text{state}[sID] := \text{ACCEPT}$	
67 return m_2	
SEND-M2 (sID, m_2)	
68 $(i, j) := (\text{owner}[sID], \text{peer}[sID])$	
69 $(\rho, s) := (\text{role}[sID], \text{state}[sID])$	
70 if $i = \perp$ or $\rho \neq \text{INITIATOR}$: abort	
71 $\text{received}[sID] := m_2$	
72 $(c_a, c_e, kID, \tau'_1, \tau'_2) := m_2$	
73 $(k_b, sk_e, m_1) := s$	$\parallel \text{or abort}$
74 $k'_a := \text{Decaps}_{stat}(sk_i, c_a)$	
75 $k'_e := \text{Decaps}_{eph}(sk_e, c_e)$	
76 if $sID = sID'_{qRecv}$	
77 $(k'_a, k'_e) := (\text{ka}, \text{ke})$	$\parallel G_{8,b}^{\text{qkd,r,mt}}$
78 $k_{pqc} := \text{KDF}(k_b, k'_a, k'_e)$	
79 $k_{qkd} :=$	
QKD-GET-KEY-WITH-ID (sID, kID)	
80 if $k_{qkd} = \perp$: abort	
81 $\text{kidUsed}[sID] += [kID]$	$\parallel G_{1,b}^{\text{qkd}}$
82 if $sID'_{qSent} = \text{qSentSid}[kID]$	
83 $k_{qkd} := \widetilde{k_{qkd}}$	$\parallel G_{4,b}^{\text{qkd}}$
84 $t := (m_1, (c_a, c_e, kID))$	
85 if $sID'_{qSent} = \text{qSentSid}[kID]$ and $\text{tresp} = t$	
86 abort	$\parallel G_{6,b}^{\text{qkd,r},\neg\text{mt}}$
87 if $sID'_{qSent} = \text{qSentSid}[kID]$ and $\text{tresp} \neq t$	
88 abort	$\parallel G_{6,b}^{\text{qkd,r,mt}}$
89 $(k_{qkd,m} \parallel k_{qkd,s}) := k_{qkd}$	
90 $(k_{pqc,m} \parallel k_{pqc,s}) := k_{pqc}$	
91 $k_{sess} := k_{qkd,s} \oplus k_{pqc,s}$	
92 if $sID'_{qSent} = \text{qSentSid}[kID]$	
93 $k_{sess} := \widetilde{k_{sess}}$	$\parallel G_{9,b}^{\text{qkd}}$
94 $\tau_1 = \text{MAC}_{k_{pqc,m}}^{(qkd)}((t, i, j))$	
95 $\tau_2 = \text{MAC}_{k_{pqc,m}}^{(pqc)}((t, \tau_1, i, j))$	
96 if $sID'_{qSent} = \text{qSentSid}[kID]$	
97 abort	$\parallel G_{7,b}^{\text{qkd,r},\neg\text{mt}}$
98 if $\tau_1 \neq \tau'_1$ or $\tau_2 \neq \tau'_2$: abort	
99 $\text{sesKey}[sID] := k_{sess}$	
100 $\text{state}[sID] := \text{ACCEPT}$	

Fig. 17: Game G^{qkd} : QKD is secure.

does not hold a triple. In the protocol a session queries the QKD oracle only once (a responder in Line 49 and an initiator in Line 79), so that each session uses *at most* one QKD key, and each accepting session uses *exactly* one QKD key. Thus the check on Line 14 will always be false, so that

$$\Pr[G_{0,b}^{\text{qkd}} \Rightarrow 1] = \Pr[G_{1,b}^{\text{qkd}} \Rightarrow 1]. \quad (61)$$

Next we consider the other session that shares the QKD key with the test session, if it exists. First, define $sID_{qSent} = \text{qSentSid}[kID^*]$. Note that this session always exists ($sID_{qSent} \neq \perp$): only an honest query to **QKD-GET-KEY** can set $\text{qStatus}[kID^*]$ to **HONEST**, and this also sets $\text{qSentSid}[kID^*]$. Second, if $\text{qRecvSid}[kID^*] \neq []$, then define sID_{qRecv} such that $[sID_{qRecv}] = \text{qRecvSid}[kID^*]$. Note that $|\text{qRecvSid}[kID^*]| \leq 1$ even if $sID_{qRecv} \neq sID^*$, because $\text{qRecvSid}[kID^*]$ is only updated by **QKD-GET-KEY-WITH-ID**(sID, kID^*) if $\text{qKey}[kID^*] \neq \perp$, and an **HONEST** key will be deleted after the first call to **QKD-GET-KEY-WITH-ID**(sID, kID^*). This proves that there is at most one completed matching session.

Each responder session calls **QKD-GET-KEY** once and increments kID_{ctr} , so there is only one honest responder session that contains kID^* in **sent**. Since the QKD key is **HONEST**, it is only delivered to one initiator session. Therefore there is at most one honest initiator session that received a key from **QKD-GET-KEY-WITH-ID**(sID, kID^*), implying there is at most one accepting initiator session with kID^* in **received**.

Game $G_{3,b}^{\text{qkd}}$ makes a guess sID'_{qSent} for sID_{qSent} and returns 0 if the guess was incorrect. Line 17 checks this by comparing the guess sID'_{qSent} against $\text{qSentSid}[kID^*]$, which equals sID_{qSent} by definition. Since $\Pr[sID'_{qSent} = sID_{qSent}] = 1/n_s$, we have

$$\Pr[G_{2,b}^{\text{qkd}} \Rightarrow 1] = n_s \Pr[G_{3,b}^{\text{qkd}} \Rightarrow 1]. \quad (62)$$

Game $G_{4,b}^{\text{qkd}}$ samples $\widetilde{k_{qkd}}$ and replaces k_{qkd} as sampled by the QKD oracle with $\widetilde{k_{qkd}}$ in sessions sID_{qSent} and (if it exists) in sID_{qRecv} . The predicate on Line 82 ($sID'_{qSent} = \text{qSentSid}[kID^*]$) is equivalent to the predicate $sID = sID_{qRecv}$, except it will be false for all sessions if sID_{qRecv} does not exist. The equivalence holds since $\text{qStatus}[kID^*] = \text{HONEST}$, and thus the QKD key is delivered to at most one initiator session. For any other adversary queries **SEND-M2** containing kID^* , **QKD-GET-KEY-WITH-ID** will return $\perp$ and those sessions will abort on Line 80. Since all we did was move the uniform sampling from the oracle to the game, we have

$$\Pr[G_{3,b}^{\text{qkd}} \Rightarrow 1] = \Pr[G_{4,b}^{\text{qkd}} \Rightarrow 1]. \quad (63)$$

Game $G_{9,b}^{\text{qkd}}$ replaces k_{sess} with the uniformly random sampled $\widetilde{k_{sess}}$. Note that $k_{qkd,s}$ acts as a one-time pad on $k_{pqc,s}$, meaning that this gamehop is indistinguishable if $k_{qkd,s}^*$ (the value of $k_{qkd,s}$ used by the test session) is independent of the adversary view (without the test session key). We will prove this by case distinction.

CASE 1: sID_{qRecv} DOES NOT EXIST. In this case we assume $\text{qRecvSid}[kID^*] = []$: no honest session received the test session QKD key. Then $k_{qkd,s}^*$ only exists in session sID_{qSent} (and it must hold that $sID_{qSent} = sID^*$), so that

$$\Pr[G_{4,b}^{\text{qkd}} \Rightarrow 1 \wedge \text{qRecvSid}[kID^*] = []] = \Pr[G_{9,b}^{\text{qkd}} \Rightarrow 1]. \quad (64)$$

CASE 2: sID_{qRecv} EXISTS. In this case we assume $\text{qRecvSid}[kID^*] = [sID_{qRecv}]$. This is encoded in game $G_{5,b}^{\text{qkd},r}$, so that

$$\Pr[G_{4,b}^{\text{qkd}} \Rightarrow 1 \wedge \text{qRecvSid}[kID^*] = [sID_{qRecv}]] = \Pr[G_{5,b}^{\text{qkd},r} \Rightarrow 1]. \quad (65)$$

We prove $G_{5,b}^{\text{qkd},r}$ is indistinguishable from $G_{9,b}^{\text{qkd}}$ by looking at several subcases, based on whether or not the transcripts and/or tags are equal. Let $t_{qRecv} = ((c_b, pk_e), (c'_a, c'_e, kID^*))$ be the partial transcript of sID_{qRecv} , and $t_{qSent} = ((c'_b, pk'_e), (c_a, c_e, kID^*))$ be the partial transcript of sID_{qSent} . Similarly let τ_1, τ_2 be the tags sent by sID_{qSent} , and let τ'_1, τ'_2 be the tags as received by sID_{qRecv} .

CASE 2A: TRANSCRIPTS DIFFER. This case assumes $t_{qRecv} \neq t_{qSent}$. This is encoded in game $G_{6,b}^{\text{qkd},r,\neg\text{mt}}$, so that

$$\Pr[G_{5,b}^{\text{qkd},r} \Rightarrow 1 \wedge t_{qRecv} \neq t_{qSent}] = \Pr[G_{6,b}^{\text{qkd},r,\neg\text{mt}} \Rightarrow 1]. \quad (66)$$

Game $G_{7,b}^{\text{qkd},r,\neg\text{mt}}$ always aborts session sID_{qRecv} . Note that $G_{6,b}^{\text{qkd},r,\neg\text{mt}}$ and $G_{7,b}^{\text{qkd},r,\neg\text{mt}}$ are identical unless $G_{6,b}^{\text{qkd},r,\neg\text{mt}}$ does not abort session sID_{qRecv} on Line 98. Then by the difference lemma, we have

$$\left| \Pr[G_{6,b}^{\text{qkd},r,\neg\text{mt}} \Rightarrow 1] - \Pr[G_{7,b}^{\text{qkd},r,\neg\text{mt}} \Rightarrow 1] \right| \leq \Pr[G_{6,b}^{\text{qkd},r,\neg\text{mt}} \text{ does not abort } sID_{qRecv} \text{ on Line 98}],$$

This event would require at least $\tau_1 = \tau'_1$ to hold in sID_{qRecv} , which lets us bound this by a reduction to the OT-sEUF-CMA security of $\text{MAC}^{(qkd)}$, using the following adversary $B_7 = (B_{7,0}, B_{7,1})$: B_7 runs game $G_{6,b}^{\text{qkd},r,\neg\text{mt}}$, with the following modifications. On Line 61, if $sID = sID'_{qSent}$, then instead of computing τ directly, $B_{7,0}$ outputs $m = (t_{qSent}, \text{peer}[sID_{qSent}], \text{owner}[sID_{qSent}])$ and uses the response τ . Just before Line 98, if $sID_{qSent} = \text{qSentSid}[kID]$ (ie. if $sID = sID_{qRecv}$), $B_{7,1}$ outputs $(m', \tau') = ((t_{qRecv}, \text{owner}[sID_{qRecv}], \text{peer}[sID_{qRecv}]), \tau'_1)$. Then $m \neq m'$, and B_7 created a successful forgery exactly when sID_{qSent} accepts τ'_1 . Thus

$$\Pr[G_{6,b}^{\text{qkd},r,\neg\text{mt}} \text{ does not abort } sID_{qRecv} \text{ on Line 98}] \leq \text{Adv}_{\text{MAC}^{(qkd)}}^{\text{OT-sEUF-CMA}}(B_7)$$

and therefore

$$\left| \Pr[G_{6,b}^{\text{qkd},r,\neg\text{mt}} \Rightarrow 1] - \Pr[G_{7,b}^{\text{qkd},r,\neg\text{mt}} \Rightarrow 1] \right| \leq \text{Adv}_{\text{MAC}^{(qkd)}}^{\text{OT-sEUF-CMA}}(B_7). \quad (67)$$

Since $G_{7,b}^{\text{qkd},r,\neg\text{mt}}$ always aborts sID_{qRecv} , $k_{qkd,s}^*$ only exists in sID_{qSent} , so that:

$$\Pr[G_{7,b}^{\text{qkd},r,\neg\text{mt}} \Rightarrow 1] = \Pr[G_{9,b}^{\text{qkd}} \Rightarrow 1]. \quad (68)$$

We combine Eqs. (66) to (68) to conclude that for Case 2a:

$$|\Pr[G_{5,b}^{\text{qkd},r} \Rightarrow 1 \wedge t_{qRecv} \neq t_{qSent}] - \Pr[G_{9,b}^{\text{qkd}} \Rightarrow 1]| \leq \text{Adv}_{\text{MAC}^{(qkd)}}^{\text{OT-sEUF-CMA}}(B_7). \quad (69)$$

CASE 2B: TRANSCRIPTS ARE THE SAME. In this case: $t_{qRecv} = t_{qSent}$. Game $G_{6,b}^{\text{qkd},r,\text{mt}}$ encodes this:

$$\Pr[G_{5,b}^{\text{qkd},r} \Rightarrow 1 \wedge t_{qRecv} = t_{qSent}] = \Pr[G_{6,b}^{\text{qkd},r,\text{mt}} \Rightarrow 1]. \quad (70)$$

By definition of sID_{qSent} and sID_{qRecv} , both sessions use the same k_{qkd} , and thus the same $k_{qkd,m}$. Note that $(\text{owner}[sID_{qSent}], \text{peer}[sID_{qSent}]) = (\text{peer}[sID_{qRecv}], \text{owner}[sID_{qRecv}])$, because QKD-GET-KEY-WITH-ID checked this. We conclude that both sessions have the same key and message input to $\text{MAC}^{(qkd)}$, so they compute the same tag τ_1 . If the adversary changed $\tau'_1 \neq \tau_1$, then the initiator will abort, therefore:

$$\Pr[G_{6,b}^{\text{qkd},r,\text{mt}} \Rightarrow 1 \wedge \tau'_1 \neq \tau_1] = \Pr[G_{9,b}^{\text{qkd}} \Rightarrow 1]. \quad (71)$$

Next we prove that if $t_{qRecv} = t_{qSent}$, then almost certainly (by correctness of the KEMs) both sessions compute the same k_{pqc} . Game $G_{7,b}^{\text{qkd},r,\text{mt}}$ makes a uniformly random guess sID'_{qRecv} for

sID_{qRecv} and aborts if the guess was incorrect. Since the guess is correct with probability $1/(n_s - 1)$, we have

$$\Pr[G_{6,b}^{qkd,r,mt} \Rightarrow 1 \wedge \tau'_1 = \tau_1] = (n_s - 1) \Pr[G_{7,b}^{qkd,r,mt} \Rightarrow 1 \wedge \tau'_1 = \tau_1]. \quad (72)$$

Game $G_{8,b}^{qkd,r,mt}$ records the encapsulated secrets in sID_{qRecv} and replaces the decapsulation outputs in sID_{qSent} with the recorded values, and vice versa. By the correctness of the two static KEMs and the one ephemeral KEM, we have

$$|\Pr[G_{7,b}^{qkd,r,mt} \Rightarrow 1 \wedge \tau'_1 = \tau_1] - \Pr[G_{8,b}^{qkd,r,mt} \Rightarrow 1 \wedge \tau'_1 = \tau_1]| \leq 2\delta_{stat} + \delta_{eph}. \quad (73)$$

We emphasize that the KEM correctness is a statistical property and not a computational one, since the probability in Definition 2.1 is independent of any adversary.

In $G_{8,b}^{qkd,r,mt}$, both sID_{qSent} and sID_{qRecv} compute the same k_{pqc} , and thus the same $k_{pqc,m}$. Since $t_{qRecv} = t_{qSent}$, $\tau'_1 = \tau_1$, and $(\text{peer}[sID_{qSent}], \text{owner}[sID_{qSent}]) = (\text{owner}[sID_{qRecv}], \text{peer}[sID_{qRecv}])$, both key and message input to $\text{MAC}^{(pqc)}$ are the same in both sessions. That means that if the adversary sends $\tau'_2 \neq \tau_2$, then sID_{qRecv} will reject with certainty since $\text{MAC}^{(pqc)}$ is assumed to be canonical.⁷ On the other hand, if $\tau'_2 = \tau_2$, then sID_{qSent} and sID_{qRecv} are matching sessions and the adversary may not query **REVEAL** on either session. In either case, $k_{qkd,s}$ does not leak through any session, so that

$$\Pr[G_{8,b}^{qkd,r,mt} \Rightarrow 1 \wedge \tau'_1 = \tau_1] = \Pr[G_{9,b}^{qkd} \Rightarrow 1]. \quad (74)$$

We combine Eqs. (72) to (74) to conclude that

$$|\Pr[G_{6,b}^{qkd,r} \Rightarrow 1 \wedge \tau'_1 = \tau_1] - \Pr[G_{9,b}^{qkd} \Rightarrow 1]| \leq n_s(2\delta_{stat} + \delta_{eph}), \quad (75)$$

summing Eqs. (71) and (75) we get

$$|\Pr[G_{6,b}^{qkd,r} \Rightarrow 1] - \Pr[G_{9,b}^{qkd} \Rightarrow 1]| \leq n_s(2\delta_{stat} + \delta_{eph}), \quad (76)$$

and combining Eqs. (70) and (76) we conclude Case 2b

$$|\Pr[G_{5,b}^{qkd,r} \Rightarrow 1 \wedge t_{qRecv} = t_{qSent}] - \Pr[G_{9,b}^{qkd} \Rightarrow 1]| \leq n_s(2\delta_{stat} + \delta_{eph}). \quad (77)$$

Then we conclude Case 2 by considering disjoint Cases 2a and 2b by taking the sum of Eqs. (69) and (77) and combining with Eq. (65):

$$\begin{aligned} & \left| \Pr[G_{4,b}^{qkd} \Rightarrow 1 \wedge \text{qRecvSid}[kID^*] = [sID_{qRecv}]] - \Pr[G_{9,b}^{qkd} \Rightarrow 1] \right| \\ & \leq n_s(2\delta_{stat} + \delta_{eph}) + \text{Adv}_{\text{MAC}^{(qkd)}}^{\text{OT-sEUF-CMA}}(B_7). \end{aligned} \quad (78)$$

We add Eqs. (64) and (78) to combine disjoint Cases 1 and 2:

$$\left| \Pr[G_{4,b}^{qkd} \Rightarrow 1] - \Pr[G_{9,b}^{qkd} \Rightarrow 1] \right| \leq n_s(2\delta_{stat} + \delta_{eph}) + \text{Adv}_{\text{MAC}^{(qkd)}}^{\text{OT-sEUF-CMA}}(B_7) \quad (79)$$

Since k_{sess} is uniformly random in $G_{9,b}^{qkd}$, we have

$$\Pr[G_{9,0}^{qkd} \Rightarrow 1] = \Pr[G_{9,1}^{qkd} \Rightarrow 1], \quad (80)$$

so that combining Eqs. (59) to (63) and (80) concludes Case 3, thereby proving Theorem 5.5

$$\begin{aligned} & \left| \Pr[\text{IND-AA-QKD}_0(\Pi, A) \Rightarrow 1] - \Pr[\text{IND-AA-QKD}_1(\Pi, A) \Rightarrow 1] \right| \\ & \leq 2n_s \left(n_s(2\delta_{stat} + \delta_{eph}) + \text{Adv}_{\text{MAC}^{(qkd)}}^{\text{OT-sEUF-CMA}}(B_7) \right). \end{aligned}$$

⁷ Statistical strong unforgeability would suffice but induces as small loss, however *computational* strong unforgeability is insufficient.

6 Conclusion and Future Work

In this paper we identify some gaps within state-of-the-art modelling of combined QKD and PQC authenticated key exchange, most notably we consider the omission of QKD key IDs in the models. We demonstrate that this omission allows an adversary to mix-and-match QKD keys of different sessions, which could lead to full leakage of the session keys. To enable the proper handling of key IDs in cryptographic contexts, we propose to model QKD as an oracle that explicitly models key IDs as input and output of oracle queries. The oracle provides queries for honest parties to get QKD keys, and it provides adversarial queries to model both passive and active attacks on QKD.

To demonstrate the usefulness of the QKD oracle, we construct a QKD/PQC-combined protocol that generates secure session keys if either QKD is secure or the PQC (or PQC/classical-combined) primitives are. We emphasize that our protocol preserves the information-theoretic security provided by QKD. Our solution uses nested MACs to prevent Dependent-Key attacks, a construction that is inspired by the XOR-then-MAC KEM-combiner, but which avoids some of the pitfalls we identified in its security proof.

FUTURE WORK. Our QKD oracle is modular and can be integrated models other than CK^+ : many variations on AKE models exist and (in principle) the oracle could be integrated into any of them, allowing formal modelling of other QKD/PQC-combined protocols. For example, it could be integrated into HAKE, so that the security for the Muckle family of protocols could be proven when QKD key IDs are included into the protocol.

One noteworthy advantage of the HAKE model over ours is that it captures Post-Compromise Security (PCS) [CCG16], which allows some security even after session compromise. PCS seems a meaningful property to have in some contexts, we view it as an interesting future direction to combine our QKD oracle with AKE models that model PCS (such as HAKE).

We identified an open problem in Section 3.1 about the construction of robust one-time strong unforgeable MACs for the XtM KEM combiner. Future work could explore the existence of such a MAC, or it might be possible to fix the proof for the XtM combiner.

Our QKD oracle returns uniformly random bitstrings, while real QKD can only guarantee statistical security: the output key is statistically close to uniformly random. The oracle could be adapted into a more accurate representation of reality if this QKD security parameter is reflected in its output.

The QKD oracle presented in this paper is designed to model *end-to-end QKD*. Many deployments of QKD instead secure point-to-point links in a larger network, and trusted repeater nodes are used to relay messages and/or keys. While the QKD oracle could be used to output relayed keys, we point out that this is a strong assumption on the security of both QKD and on the relaying protocol itself. Instead for relaying protocols, it would be more realistic to use the QKD oracle only to model point-to-point links, and to precisely state the trust assumptions that are required in order to guarantee security of the relaying protocol itself.

Composability of the QKD interface. In this work we aim to prevent Dependent-Key attacks *within one* AKE protocol. We emphasize, however, that attacks on (concurrently) *composed* protocols are not captured in game-based AKE models – in other words, our security model does not guarantee secure composability. Even if QKD is secure and the AKE protocol is secure, this does not always mean that its combination via the ETSI 014 QKD interface (described in Section 2.5) is secure.

To illustrate the concern, consider the following Dependent-Key attack *between different protocols*. Imagine a scenario where Alice runs some (secure) protocol Π , where she uses a (secure) QKD key retrieved via “Get key”. Bob runs two protocols, Π and Π' , and in both he retrieves the QKD key via “Get key with key IDs”. The adversary can take the key ID sent by Alice in protocol Π , and deliver it to Bob in protocol Π' . If Π' happens to be vulnerable, this may leak the QKD key to the

adversary, which could be sufficient to impersonate Bob in protocol Π . Note that neither QKD nor Π need to be broken by the adversary for this attack.

For most PQC protocols this issue does not result in real-world attacks, because proper operational security practices (such as using a key only for a single protocol and protocol-specific key derivation) ensure that a key is only used in a single protocol. In a typical QKD setup, however, these methods are often not deployed or not available.

To protect against such attacks, some binding between the QKD key and the protocol (and session) in which it is used seems to be necessary, which might even require a change in the specification of the QKD interface. It is essential future work to extend our model to capture attacks of this type in order to handle such composability issues.

References

- AAC⁺22. Gorjan Alagic, Daniel Apon, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, Daniel Smith-Tone, and Yi-Kai Liu. Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process. Technical Report NIST Internal or Interagency Report (NISTIR) 8413, National Institute of Standards and Technology, 9 2022. doi:10.6028/NIST.IR.8413-upd1.
- ABC⁺25. Gorjan Alagic, Maxime Bros, Pierre Ciadoux, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, Hamilton Silberg, Daniel Smith-Tone, and Noah Waller. Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process. Technical Report NIST Internal or Interagency Report (NISTIR) 8545, National Institute of Standards and Technology, 3 2025. doi:10.6028/NIST.IR.8545.
- ABS14. Janaka Alawatugoda, Colin Boyd, and Douglas Stebila. Continuous after-the-fact leakage-resilient key exchange. In Willy Susilo and Yi Mu, editors, *ACISP 14: 19th Australasian Conference on Information Security and Privacy*, volume 8544 of *Lecture Notes in Computer Science*, pages 258–273, Wollongong, NSW, Australia, July 7–9, 2014. Springer, Heidelberg, Germany. doi:10.1007/978-3-319-08344-5_17.
- ACD⁺25. Nick Aquina, Bruno Cimoli, Soumya Das, Kathrin Hövelmanns, Fiona Johanna Weber, Chigo Okonkwo, Simon Rommel, Boris Škorić, Idelfonso Tafur Monroy, and Sebastian Verschoor. A critical analysis of deployed use cases for quantum key distribution and comparison with post-quantum cryptography. *EPJ Quantum Technology*, 12(51):1–42, 5 2025. doi:10.1140/epjqt/s40507-025-00350-5.
- ADK⁺22. Nimrod Aviram, Benjamin Dowling, Ilan Komargodski, Kenneth G. Paterson, Eyal Ronen, and Eylon Yogev. Practical (post-quantum) key combiners from one-wayness and applications to TLS. Cryptology ePrint Archive, Report 2022/065, 2022. <https://eprint.iacr.org/2022/065>.
- ANS22. ANSSI. ANSSI views on the Post-Quantum Cryptography transition, 2022. URL: <https://cyber.gouv.fr/en/publications/anssi-views-post-quantum-cryptography-transition>.
- ART24. Nick Aquina, Simon Rommel, and Idelfonso Tafur Monroy. Quantum secure communication using hybrid post-quantum cryptography and quantum key distribution. In *2024 24th International Conference on Transparent Optical Networks (ICTON)*, pages 1–4, 7 2024. doi:10.1109/ICTON62926.2024.10648124.
- BB84. C. H. Bennett and G. Brassard. Quantum cryptography: Public key distribution and coin tossing. In *Proceedings of IEEE International Conference on Computers, Systems, and Signal Processing*, page 175, India, 1984. doi:10.1016/j.tcs.2014.05.025.
- BBB⁺25. Jaime S. Buruaga, Augustine Bugler, Juan P. Brito, Vicente Martin, and Christoph Striecks. Versatile quantum-safe hybrid key exchange and its application to MACsec. *EPJ Quantum Technology*, 12(1):1–27, 7 2025. doi:10.1140/epjqt/s40507-025-00382-x.
- BBF⁺18. Nina Bindel, Jacqueline Brendel, Marc Fischlin, Brian Goncalves, and Douglas Stebila. Hybrid key encapsulation mechanisms and authenticated key exchange. Cryptology ePrint Archive, Report 2018/903, 2018. <https://eprint.iacr.org/2018/903>.

- BBF⁺19. Nina Bindel, Jacqueline Brendel, Marc Fischlin, Brian Goncalves, and Douglas Stebila. Hybrid key encapsulation mechanisms and authenticated key exchange. In Jintai Ding and Rainer Steinwandt, editors, *Post-Quantum Cryptography - 10th International Conference, PQCrypto 2019*, pages 206–226, Chongqing, China, May 8–10, 2019. Springer, Heidelberg, Germany. doi:10.1007/978-3-030-25510-7_12.
- BCD20. Elaine Barker, Lily Chen, and Richard Davis. Recommendation for key-derivation methods in key-establishment schemes. Recommendation NIST SP 800-56C, National Institute of Standards and Technology, 8 2020. doi:10.6028/NIST.SP.800-56Cr2.
- BCK98. Mihir Bellare, Ran Canetti, and Hugo Krawczyk. A modular approach to the design and analysis of authentication and key exchange protocols (extended abstract). In *30th Annual ACM Symposium on Theory of Computing*, pages 419–428, Dallas, TX, USA, May 23–26, 1998. ACM Press. doi:10.1145/276698.276854.
- BCNP08. Colin Boyd, Yvonne Cliff, Juan Gonzalez Nieto, and Kenneth G. Paterson. Efficient One-Round Key Exchange in the Standard Model. ACISP 08: 13th Australasian Conference on Information Security and Privacy, 2008. doi:10.1007/978-3-540-70500-0_6.
- BDF⁺11. Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. Random oracles in a quantum world. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology - ASIACRYPT 2011*, volume 7073 of *Lecture Notes in Computer Science*, pages 41–69, Seoul, South Korea, December 4–8, 2011. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-25385-0_3.
- BDK⁺18. Joppe Bos, Leo Ducas, Eike Kiltz, T Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehle. CRYSTALS - Kyber: A CCA-Secure Module-Lattice-Based KEM. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367, 4 2018. doi:10.1109/EuroSP.2018.00032.
- BL17. Daniel J. Bernstein and Tanja Lange. Post-quantum cryptography. *Nature*, 549(7671):188–194, 9 2017. doi:10.1038/nature23461.
- BR94. Mihir Bellare and Phillip Rogaway. Entity authentication and key distribution. In Douglas R. Stinson, editor, *Advances in Cryptology - CRYPTO'93*, volume 773 of *Lecture Notes in Computer Science*, pages 232–249, Santa Barbara, CA, USA, August 22–26, 1994. Springer, Heidelberg, Germany. doi:10.1007/3-540-48329-2_21.
- BRS23. Sonja Bruckner, Sebastian Ramacher, and Christoph Striecks. Muckle+: End-to-End Hybrid Authenticated Key Exchanges. In Thomas Johansson and Daniel Smith-Tone, editors, *Post-Quantum Cryptography*, pages 601–633, Cham, 2023. Springer Nature Switzerland. doi:10.1007/978-3-031-40003-2_22.
- BSP⁺24. Christopher Battarbee, Christoph Striecks, Ludovic Perret, Sebastian Ramacher, and Kevin Verhaeghe. Quantum-Safe Hybrid Key Exchanges with KEM-Based Authentication. arXiv:2411.04030 [cs.CR], 11 2024. doi:10.48550/arXiv.2411.04030.
- CCG16. Katriel Cohn-Gordon, Cas J. F. Cremers, and Luke Garratt. On post-compromise security. In Michael Hicks and Boris Köpf, editors, *CSF 2016: IEEE 29th Computer Security Foundations Symposium*, pages 164–178, Lisbon, Portugal, June 27–1, 2016. IEEE Computer Society Press. doi:10.1109/CSF.2016.19.
- CK01. Ran Canetti and Hugo Krawczyk. Analysis of key-exchange protocols and their use for building secure channels. In Birgit Pfitzmann, editor, *Advances in Cryptology - EUROCRYPT 2001*, volume 2045 of *Lecture Notes in Computer Science*, pages 453–474, Innsbruck, Austria, May 6–10, 2001. Springer, Heidelberg, Germany. doi:10.1007/3-540-44987-6_28.
- CK02. Ran Canetti and Hugo Krawczyk. Universally composable notions of key exchange and secure channels. In Lars R. Knudsen, editor, *Advances in Cryptology - EUROCRYPT 2002*, volume 2332 of *Lecture Notes in Computer Science*, pages 337–351, Amsterdam, The Netherlands, April 28 – May 2, 2002. Springer, Heidelberg, Germany. doi:10.1007/3-540-46035-7_22.
- DHP20a. Benjamin Dowling, Torben Brandt Hansen, and Kenneth G. Paterson. Many a mickle makes a muckle: A framework for provably quantum-secure hybrid key exchange. In Jintai Ding and Jean-Pierre Tillich, editors, *Post-Quantum Cryptography - 11th International Conference, PQCrypto 2020*, pages 483–502, Paris, France, April 15–17, 2020. Springer, Heidelberg, Germany. doi:10.1007/978-3-030-44223-1_26.

- DHP20b. Benjamin Dowling, Torben Brandt Hansen, and Kenneth G. Paterson. Many a mickle makes a muckle: A framework for provably quantum-secure hybrid key exchange. Cryptology ePrint Archive, Report 2020/099, 2020. <https://eprint.iacr.org/2020/099>.
- DSL⁺22. Oskar van Deventer, Nicolas Spethmann, Marius Loeffler, Momtchil Peev, Andreas Poppe, Stefan Rass, Michael Sauerwein, Peter Schindler, Damien Stucki, Nino Walenta, Helmut Weier, Christiane Wichmann, and Anton Zeilinger. Towards European Standards for Quantum Technologies. *EPJ Quantum Technology*, 9(1):33, 11 2022. doi:10.1140/epjqt/s40507-022-00150-1.
- Eke91. Artur K. Ekert. Quantum cryptography based on bell’s theorem. *Phys. Rev. Lett.*, 67:661–663, Aug 1991. doi:10.1103/PhysRevLett.67.661.
- ETS19. ETSI. Quantum Key Distribution (QKD); Protocol and data format of REST-based key delivery API. Standard GS QKD 014, European Telecommunications Standards Institute, 2 2019. URL: https://www.etsi.org/deliver/etsi_gs/QKD/001_099/014/01.01_01_60/gs_qkd014v010101p.pdf.
- ETS20. ETSI. Quantum Key Distribution (QKD); Application Interface. Standard GS QKD 0004, European Telecommunications Standards Institute, 8 2020. URL: https://www.etsi.org/deliver/etsi_gs/QKD/001_099/004/02.01_01_60/gs_qkd004v020101p.pdf.
- Fed21. Federal Office for Information Security. Quantum-safe cryptography – fundamentals, current developments and recommendations, 2021. URL: <https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/Brochure/quantum-safe-cryptography.html>.
- FSXY12. Atsushi Fujioka, Koutarou Suzuki, Keita Xagawa, and Kazuki Yoneyama. Strongly secure authenticated key exchange from factoring, codes, and lattices. In Marc Fischlin, Johannes Buchmann, and Mark Manulis, editors, *PKC 2012: 15th International Conference on Theory and Practice of Public Key Cryptography*, volume 7293 of *Lecture Notes in Computer Science*, pages 467–484, Darmstadt, Germany, May 21–23, 2012. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-30057-8_28.
- FSXY13. Atsushi Fujioka, Koutarou Suzuki, Keita Xagawa, and Kazuki Yoneyama. Practical and post-quantum authenticated key exchange from one-way secure key encapsulation mechanism. In Kefei Chen, Qi Xie, Weidong Qiu, Ninghui Li, and Wen-Guey Tzeng, editors, *ASIACCS 13: 8th ACM Symposium on Information, Computer and Communications Security*, pages 83–94, Hangzhou, China, May 8–10, 2013. ACM Press. doi:10.1145/2484313.2484323.
- GHP18. Federico Giacon, Felix Heuer, and Bertram Poettering. KEM combiners. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018: 21st International Conference on Theory and Practice of Public Key Cryptography, Part I*, volume 10769 of *Lecture Notes in Computer Science*, pages 190–218, Rio de Janeiro, Brazil, March 25–29, 2018. Springer, Heidelberg, Germany. doi:10.1007/978-3-319-76578-5_7.
- GPH⁺24. Lydia Garms, Taofiq K. Paraíso, Neil Hanley, Ayesha Khalid, Ciara Rafferty, James Grant, James Newman, Andrew J. Shields, Carlos Cid, and Maire O’Neill. Experimental Integration of Quantum Key Distribution and Post-Quantum Cryptography in a Hybrid Quantum-Safe Cryptosystem. *Advanced Quantum Technologies*, 7(4):2300304, 2 2024. doi:10.1002/qute.202300304.
- HKSU20. Kathrin Hövelmanns, Eike Kiltz, Sven Schäge, and Dominique Unruh. Generic authenticated key exchange in the quantum random oracle model. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020: 23rd International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 12111 of *Lecture Notes in Computer Science*, pages 389–422, Edinburgh, UK, May 4–7, 2020. Springer, Heidelberg, Germany. doi:10.1007/978-3-030-45388-6_14.
- ITU20. ITU-T. Key combination and confidential key supply for quantum key distribution networks. Recommendation ITU-T X.1714, International Telecommunication Union, 10 2020. URL: <http://handle.itu.int/11.1002/1000/14453>.
- Kra03. Hugo Krawczyk. SIGMA: The “SIGN-and-MAC” approach to authenticated Diffie-Hellman and its use in the IKE protocols. In Dan Boneh, editor, *Advances in Cryptology – CRYPTO 2003*, volume 2729 of *Lecture Notes in Computer Science*, pages 400–425, Santa Barbara, CA, USA, August 17–21, 2003. Springer, Heidelberg, Germany. doi:10.1007/978-3-540-45146-4_24.
- Kra05. Hugo Krawczyk. HMQV: A high-performance secure Diffie-Hellman protocol. In Victor Shoup, editor, *Advances in Cryptology – CRYPTO 2005*, volume 3621 of *Lecture Notes in Computer*

- Science*, pages 546–566, Santa Barbara, CA, USA, August 14–18, 2005. Springer, Heidelberg, Germany. doi:10.1007/11535218_33.
- Kra10. Hugo Krawczyk. Cryptographic extraction and key derivation: The HKDF scheme. In Tal Rabin, editor, *Advances in Cryptology – CRYPTO 2010*, volume 6223 of *Lecture Notes in Computer Science*, pages 631–648, Santa Barbara, CA, USA, August 15–19, 2010. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-14623-7_34.
- LLM07. Brian A. LaMacchia, Kristin Lauter, and Anton Mityagin. Stronger security of authenticated key exchange. In Willy Susilo, Joseph K. Liu, and Yi Mu, editors, *ProvSec 2007: 1st International Conference on Provable Security*, volume 4784 of *Lecture Notes in Computer Science*, pages 1–16, Wollongong, Australia, November 1–2, 2007. Springer, Heidelberg, Germany. doi:10.1007/978-3-540-75670-5_1.
- MSU13. Michele Mosca, Douglas Stebila, and Berkant Ustaoglu. Quantum key distribution in the classical authenticated key exchange framework. In Philippe Gaborit, editor, *Post-Quantum Cryptography - 5th International Workshop, PQCrypto 2013*, pages 136–154, Limoges, France, June 4–7, 2013. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-38616-9_9.
- RCV⁺23. Carlos Rubio García, Abraham Cano Aguilera, Juan Jose Vegas Olmos, Idelfonso Tafur Monroy, and Simon Rommel. Quantum-Resistant TLS 1.3: A Hybrid Solution Combining Classical, Quantum and Post-Quantum Cryptography. In *2023 IEEE 28th International Workshop on Computer Aided Modeling and Design of Communication Links and Networks (CAMAD)*, pages 246–251, 11 2023. doi:10.1109/CAMAD59638.2023.10478407.
- RDM⁺24. Sara Ricci, Patrik Dobias, Lukas Malina, Jan Hajny, and Petr Jedlicka. Hybrid Keys in Practice: Combining Classical, Quantum and Post-Quantum Cryptography. *IEEE Access*, 12:23206–23219, 2024. doi:10.1109/ACCESS.2024.3364520.
- SCN⁺24. Secure Information Technology Center Austria, Centre for Cybersecurity Belgium, National Cyber and Information Security Agency Czech Republic, Centre for Cyber Security Denmark, Information System Authority Estonia, Finnish transport and Communication Agency, French National Agency for the Security of Information Systems, Federal Office for Information Security Germany, National Cyber Security Authority Hellenic Republic, National Cyber Security Centre Ireland, National Cybersecurity Agency Italy, Ministry of Defense Latvia, National Cyber Security Centre Ministry of Defense Lithuania, High Commission for National Protection Luxemburg, Netherlands National Communication Security Agency, Ministry of Interior and Kingdom Relations Netherlands, National Cyber Security Centre Ministry of Security and Justice Netherlands, Research and Academic Research Center Poland, Government Information Security Office Slovenia, and National Cryptologic Center Spain. Securing Tomorrow, Today: Transitioning to Post-Quantum Cryptography, 11 2024. URL: <https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Crypto/PQC-joint-statement.pdf>.
- Sho94. Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *35th Annual Symposium on Foundations of Computer Science*, pages 124–134, Santa Fe, NM, USA, November 20–22, 1994. IEEE Computer Society Press. doi:10.1109/SFCS.1994.365700.
- Sho04. Victor Shoup. Sequences of games: a tool for taming complexity in security proofs. Cryptology ePrint Archive, Paper 2004/332, 2004. URL: <https://eprint.iacr.org/2004/332>.
- SPP⁺24. Juan Morales Sáez, Antonio Pastor Perales, Rafael Cantó Palancar, Diego R. Lopez, Jesús Folgueira Chavarria, Vicente Martín Ayuso, and Juan Pedro Brito Méndez. Current status, gaps, and future directions in quantum key distribution standards: Implications for industry. In *2024 International Conference on Quantum Communications, Networking, and Computing (Q CNC)*, pages 341–345, 2024. doi:10.1109/QCNC62729.2024.00059.
- SSW20. Peter Schwabe, Douglas Stebila, and Thom Wiggers. Post-quantum TLS without handshake signatures. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020: 27th Conference on Computer and Communications Security*, pages 1461–1480, Virtual Event, USA, November 9–13, 2020. ACM Press. doi:10.1145/3372297.3423350.
- TLLM18. Piotr K. Tysowski, Xinhua Ling, Norbert Lütkenhaus, and Michele Mosca. The engineering of a scalable multi-site communications system utilizing quantum key distribution (QKD). *Quantum Science and Technology*, 3(2):024001, 1 2018. doi:10.1088/2058-9565/aa9a5d.

- Unr15. Dominique Unruh. Revocable Quantum Timed-Release Encryption. *Journal of the ACM*, 62(6):49:1–49:76, 12 2015. doi:[10.1145/2817206](https://doi.org/10.1145/2817206).
- WC81. Mark N. Wegman and Larry Carter. New hash functions and their use in authentication and set equality. *Journal of Computer and System Sciences*, 22:265–279, 1981.